OCA

OPEN CHIPLET ATLAS

The System Architecture Specification

Version 0.7.6, 02/10/2026: Under Development

Table of Contents

Welcome . . . 1
Important Notice . . . 1
1. Changes and Feedback . . . 2
1.1. Feedback . . . 2
2. Overview . . . 3
2.1. Interoperability Challenges for Chiplets . . . 3
2.2. Goal of Open Chiplet Atlas Architecture . . . 4
2.3. Scope of OCA Architecture . . . 4
2.3.1. Layered View of OCA Architecture . . . 4
2.3.2. Integration Examples . . . 6
2.4. Design Principles for OCA Architecture . . . 8
2.5. Interoperability and Flexibility . . . 8
2.6. Document Organization . . . 8
3. Foundations of the OCA Architecture . . . 9
3.1. Chiplet Roles . . . 9
3.2. Chiplet Topology . . . 9
4. System Layer . . . 12
4.1. Communication Channels . . . 12
4.1.1. Application Channels . . . 12
4.1.2. Non-Application Channels . . . 13
4.1.3. Inter-chiplet Deadlock Prevention . . . 13
4.2. Addressing . . . 15
4.2.1. Address Domains . . . 15
4.2.2. Address Translation . . . 15
4.2.2.1. Address Translation Channel . . . 16
4.2.2.2. Address Translation Channel Properties . . . 16
4.2.3. Address Region Allocation . . . 17
4.2.4. Inter-chiplet Routing of a Request-Response Pair . . . 17
4.2.4.1. Inter-Chiplet Routing Across Domain Boundaries . . . 17
4.2.4.2. Inter-Chiplet Routing in a Shared Routing Domain . . . 19
4.3. Boot . . . 19
4.3.1. SiP Setup for Boot . . . 19
4.3.2. Boot Flow . . . 21
4.3.3. Boot Report . . . 23
4.3.4. Boot Channel . . . 24
4.4. System Management . . . 24
4.4.1. Interface between System Management Software and Primary SMC . . . 25
4.4.2. Inter-SMC Communication . . . 25
4.4.3. System Management Channel . . . 25
4.5. System Time Management . . . 25
4.5.1. System Time Counter . . . 25
4.5.2. System Time Management Channel . . . 26
4.6. Security . . . 26
4.6.1. Security Domains . . . 26
4.6.2. Secure Lifecycle Control . . . 27
4.6.3. Secure Key Management . . . 27

4.6.4. Security Channel . . . 28
4.6.5. OCA Security Implications . . . 28
4.7. Interrupts . . . 28
4.7.1. MSI Configuration . . . 29
4.7.1.1. MSI Message Control Register . . . 29
4.7.1.2. MSI Table . . . 29
4.7.1.3. MSI Pending Bits Array (PBA) . . . 30
4.7.2. Interrupt Channel . . . 30
4.8. Design for Debug . . . 30
4.8.1. Cross Triggering . . . 30
4.8.2. Timestamp Synchronization . . . 31
4.8.3. Trace Dump . . . 31
4.9. Design for Test . . . 31
4.9.1. Serial Test Channel . . . 31
4.9.2. Parallel Test Channel . . . 31
4.10. Functional Safety . . . 32
4.10.1. Interrupts . . . 33
4.10.1.1. Interrupt Self Test . . . 34
4.10.1.2. Interrupt Forwarding . . . 34
4.10.1.3. Interrupt Response and Clearing . . . 35
4.10.2. FuSa Messaging Channel . . . 35
4.10.2.1. Heartbeat and Self Test . . . 36
4.11. Reliability Availability and Serviceability . . . 36
4.11.1. Error Detection . . . 36
4.11.2. Error Response . . . 37
4.11.3. Error Report . . . 37
4.11.3.1. Error Record . . . 37
4.11.3.2. Error Containment . . . 39
5. Protocol and Transport Layers . . . 41
5.1. Mapping of Channels, Protocols, and Transport Mechanisms . . . 41
5.2. Protocols . . . 43
5.2.1. AXI over UCIe . . . 43
5.2.2. CHI-C2C . . . 43
5.2.3. Open Chiplet Configuration Protocol . . . 43
5.2.3.1. OCCP Protocol Overview . . . 43
5.2.3.2. OCCP Header Format . . . 43
5.2.3.3. Responses with Error . . . 44
5.2.3.4. GET_VERSION (MsgID: 0x0) . . . 45
5.2.3.5. Base Application (AppID: 0x0) . . . 46
5.2.3.6. Boot application (AppID: 0x1) . . . 49
5.2.4. OCCP I3C/I2C Binding . . . 52
5.2.4.1. I3C Addressing . . . 52
5.2.4.2. I3C Message Format . . . 52
5.2.4.3. I3C Message Response . . . 54
5.2.4.4. I3C Message Ordering . . . 54
5.2.5. Open Chiplet Time Synchronization (OCTS) . . . 54
5.2.5.1. System Timer Preset . . . 54
5.2.5.2. System Timer Credit . . . 54
5.2.5.3. TMR_SYNC_LOAD . . . 54

5.2.5.4. TMR_CNT_CREDIT . . . 54
5.2.5.5. Software Requirements . . . 55
5.2.6. Open Chiplet Cross Triggering (OCCT) . . . 55
5.2.6.1. Integration Examples . . . 57
5.2.7. Open Chiplet Packetized Scan (OCPS) . . . 59
5.2.7.1. Inter-Chiplet Packetized Scan Interface . . . 59
5.2.7.2. SiP Packetized Scan Interface . . . 60
5.2.7.3. Test Packet Routing . . . 60
5.2.7.4. Scan Data Synchronization . . . 60
5.3. Transport Mechanisms . . . 61
5.3.1. UCIe and BoW . . . 61
5.3.2. I3C . . . 61
5.3.3. I2C . . . 61
5.3.4. Mix and Match of I3C and I2C . . . 62
5.4. Interfaces . . . 62
5.5. Traffic Attribute . . . 64
5.5.1. Quality of Service (QoS) . . . 64
5.5.2. Performance Observability . . . 65
6. Software Layer . . . 66
6.1. Open Chiplet Management Interface (OCMI) . . . 66
6.2. Open Chiplet Configuration Data Structure (OCCDS) . . . 66
6.2.1. Common Connection Properties . . . 66
6.2.2. Functional Domain Specific Properties . . . 66
6.2.3. An Example of OCCDS . . . 67
6.3. Software Image Format (SIF) . . . 68
6.3.1. OCA Identifier . . . 69
6.3.2. OCA_SW_PKG_VER . . . 69
6.3.3. TARGET_CHIPLET_ID . . . 69
6.3.4. PACKAGE_VERSION . . . 70
6.3.5. SW_PACKAGE_CONFIGURATION . . . 70
6.3.6. PKG_SIZE . . . 70
6.3.7. VENDOR BUNDLE . . . 71
6.3.8. PACKAGE_HASH . . . 71
6.4. Tooling and Software Support . . . 72
7. Profiles . . . 73
7.1. Automotive Profile . . . 73
7.1.1. System Layer for Automotive Chiplets . . . 73
7.1.2. Transport Layer and Protocol Layer for Automotive Chiplets . . . 74
7.2. Data Center Profile . . . 74
7.2.1. System Layer of Data Center Chiplets . . . 75
7.2.2. Transport Layer and Protocol Layer for Data Center Chiplets . . . 75
7.3. Data Center Profile Using Bunch of Wires . . . 76
7.3.1. System Layer of Data Center Chiplets . . . 76
7.3.2. Transport Layer and Protocol Layer for Data Center Chiplets . . . 77
Appendix A: **Interim Specification License** . . . 79
Appendix B: Terminology . . . 82
B.1. Initialism . . . 82
B.2. Nomenclature . . . 82
B.3. Conventions . . . 83

Appendix C: **High Level Final License Terms (for Comment Only)** 84

Welcome

The Open Chiplet Atlas™ (OCA) System Architecture enables chiplets from different vendors to work together in an open ecosystem.

Today's integration challenges arise from incompatible solutions. The OCA System Architecture provides a unified architecture for chiplet connectivity, boot, management, and software operation.

Built on industry standards, this architecture ensures interoperability for diverse use cases.

Let's make chiplets interoperable.

Important Notice

This is a draft specification. Changes are expected. We are publishing it early to invite your feedback and collaboration. An open chiplet ecosystem cannot be built by one company as it takes a community.

By reading and using this draft, you agree that the legal terms in Appendix A apply. Please review that section for the full conditions.

Chapter 1. Changes and Feedback

Revision	Date	Descriptions
0.5	2025.05.09	Problem space defined; Interoperability architecture defined for addressing, boot, system management, system time management, security, interrupts, debug, DFT, FuSa, bus protocols, and transport schemes; Automotive profile defined.
0.5.5	2025.07.07	Updates in the automotive profile and channel properties. Elaborated on deadlock prevention, testing, and scan.
0.6	2025.08.22	Added the RAS section, the DC profile, and the legal terms; removed reference to base, clarified required functional domains in each profile, polishing and formatting for public release.
0.7	2025.09.22	Codified I2C and its compatibility with I3C in the interface section; Corrected FUSA requirement tag for the FUSA channels; Defined chiplet configuration data structure; Updated OCCDS and OCCP sections per feedback; Clarifications and edits based on feedback from BOS, AheadComputing, and other partners; Captured a problem statement from a partner; Replaced the recommendation tag of "REC" with "OPT" to avoid confusion with "REQ"; An update of Software Image Format.
0.7.5	2025.12.23	Updated the legal terms; updated the xtrig requirements; Added requirements on inter-chiplet routing of request+response per partner feedback; reflected the new profile-dependent parameters in both automotive and data center profiles; updated the Open Chiplet Atlas terminologies; incorporated feedback from from Samsung, from Prof. Li at University of California, Riverside, and others.
0.7.6	2026.02.10	Introduced a data center profile based on BOW.

1.1. Feedback

Feedback is welcome for this document. Please send any comments to feedback@openchipletatlas.org with the following supplementary information:

- Document title: Open Chiplet Atlas System Architecture Specification.
- Version.
- Page number.
- Rule identifier, if applicable.
- A concise explanation of your comments.

Chapter 2. Overview

Chiplets are small, specialized integrated circuits designed to function as individual components. Chiplets are integrated to form a System in Package (SiP) that behaves like a traditional monolithic system, especially from a high-level software perspective. Unlike traditional monolithic integrated circuits where all functions are manufactured on a single piece of silicon, chiplets may be designed, fabricated, and assembled separately. This approach allows for greater flexibility, improved yields, higher cost efficiency, and shorter time to market.

2.1. Interoperability Challenges for Chiplets

While a chiplet-based SiP offers advantages, it also brings new challenges to SiP development, especially interoperability among various chiplets. There is a growing demand to standardize chiplet interfaces and assure interoperability so that the advantages are realized.

- **Interconnect**: Chiplets need to establish connections and efficient communication across all layers including physical, transport, and protocol. Any misalignment can result in communication failure. Factors like high speed, low latency, energy efficiency, packaging feasibility, and cost need to be considered.
- **Addressing**: Chiplets define address maps and on-chip networks independently, making address map overlap inevitable. This presents challenges for cross-chiplet memory access, as the target chiplets need to interpret addresses originating from a different address map and route traffic appropriately. The situation becomes more complex when a single transaction traverses multiple chiplets before reaching its destination.
- **Boot**: While some chiplets maintain standalone boot solutions, most rely on another chiplet for early boot configuration, firmware/software loading, and boot status reporting. This necessitates alignment among chiplets from different vendors on communication, boot setup, and boot flow.
- **System Management**: System management commands regarding chiplet states, component states, power, voltage, clock, reset, performance, etc. need to travel between chiplets and be serviced correctly in a timely manner. System management control loops, like thermal control loops and current control loops, can span across multiple chiplets. Chiplets need to enable communication between system management agents using standard interfaces. Task partitioning for control loops also need to be seamless while considering hierarchy and latency.
- **Security**: Chiplets can have different security architectures, with varying definitions of secure boot, security domains, secure lifecycle control, and system/application security support. These differences pose challenges for seamless integration, as misalignment can introduce security vulnerabilities. Additionally, some security applications require complex cross-chiplet collaboration. For example, device attestation in a SiP may lead to a hierarchical attestation structure, complicating device attestation due to inconsistencies in security mechanisms across chiplets.
- **Interrupts**: Chiplets rely on interrupts to alert each other. Challenges arise due to differing implementations. Variations in interrupt types, like message-based versus wire-based, along with differences in payload format, routing mechanisms, and source configuration can lead to incompatibilities. Without careful coordination, these discrepancies can hinder seamless communication and system integration.
- **Debug and Test**: All chiplets in a package require debug and test support. However, not all have an external debug/test access port. Routing the debug/test traffic between chiplets from different vendors is challenging and can be even more so with varying security lifecycle controls from different chiplets. Inconsistencies in cross-trigger mechanisms, synchronized timestamps, and trace dumping further complicate system-wide debug standardization.
- **Functional Safety (FuSa)**: Interoperability for functional safety in chiplets is not easy to achieve due to diverse architecture, topologies, and strategies that vary by application and industry. The requirement that interoperability solutions themselves need to be robust and interference-free is another challenge in itself.
- **RAS**: Different manufacturers design RAS architectures uniquely. Individual chiplets in a SiP can have specific error handling implementations. Despite any variations, all chiplets need to collaborate to accurately identify and report errors, manage them effectively, and limit the impact of failures. Main hurdles to interoperability lie in establishing common communication protocols for RAS functions between chiplets and agreeing on fundamental definitions of error severity. Overcoming these challenges is essential to achieve RAS

interoperability and enabling advanced, profile-tailored RAS capabilities.

- **Software**: Software interoperability across chiplets is challenging due to independently defined hardware/software interfaces and inter-software communication protocols from different vendors. Without standardized interfaces, software reusability and integration become complex and inefficient. Automating SiP configuration requires a standardized data structure, but reaching consensus on its content and format across vendors remains difficult. Loading software images from different chiplet vendors is necessary yet inconsistencies in data representation and configuration mechanisms pose significant barriers to seamless integration.

2.2. Goal of Open Chiplet Atlas Architecture

The goal of the Open Chiplet Atlas™ (OCA) architecture is to provide an open and thorough solution to interoperability between chiplets in a SiP. It is designed to support IPs and chiplets from different CPU ecosystems and vendors. It comprehensively addresses the challenges in different function domains across multiple layers. OCA-architecture-compliant chiplets can be seamlessly integrated in a “plug-and-play” manner. This entails the following aspects:

1. **Plug**: Chiplets are physically connected enabling high-speed, low-latency, energy-efficient, and robust die-to-die (D2D) signal transfer, with standardized die placement, package routing, thermal management, and power delivery. Chiplets can "babble" with each other.
2. **Talk**: Chiplets use standard transport mechanisms like UCIe and BoW for data transfer. Chiplets use standard transaction protocols to construct and interpret data packets to "speak the same language".
3. **Play**: Chiplets follow system-level interoperation rules so once assembled, they collaborate automatically on traffic routing, boot, system management, security, interrupts, functional safety, RAS, debug, and testing. Chiplets not only implement standard hardware support for different system-level features, but also run software on different chiplets using standard software interfaces. This enables chiplets to "work" like a monolithic SoC, especially from high-level software perspective.

Following the OCA architecture, once chiplets from different vendors are assembled, the hardware functions seamlessly out of the box. The accompanying software interacts naturally, requiring minimal adjustments to operate smoothly.

Ultimately, the architecture to support creation of an OCA Ecosystem. In this ecosystem, chiplet designers have access to design IPs with clearly defined interoperable interfaces. Package designers benefit from a chiplet marketplace, where they can select various chiplets from different vendors and assemble them to create System-in-Packages.

2.3. Scope of OCA Architecture

The OCA architecture focuses on the interoperability of chiplets, defining how chiplets interconnect, and how chiplet features interact with other chiplets. The architecture does not define the functionality of a chiplet feature itself, neither does it define the quality or competitiveness of a chiplet. For instance, the architecture specifies how chiplets work together for power management but does not mandate specific power-saving features for each chiplet. Similarly, the architecture outlines how chiplets collaborate to support RAS but does not require particular RAS features in individual chiplets.

2.3.1. Layered View of OCA Architecture

To achieve the “plug-and-play” capability, alignment on the layers of the chiplet design stack (physical, transport, protocol, system, and software) is required as shown in Figure 1.

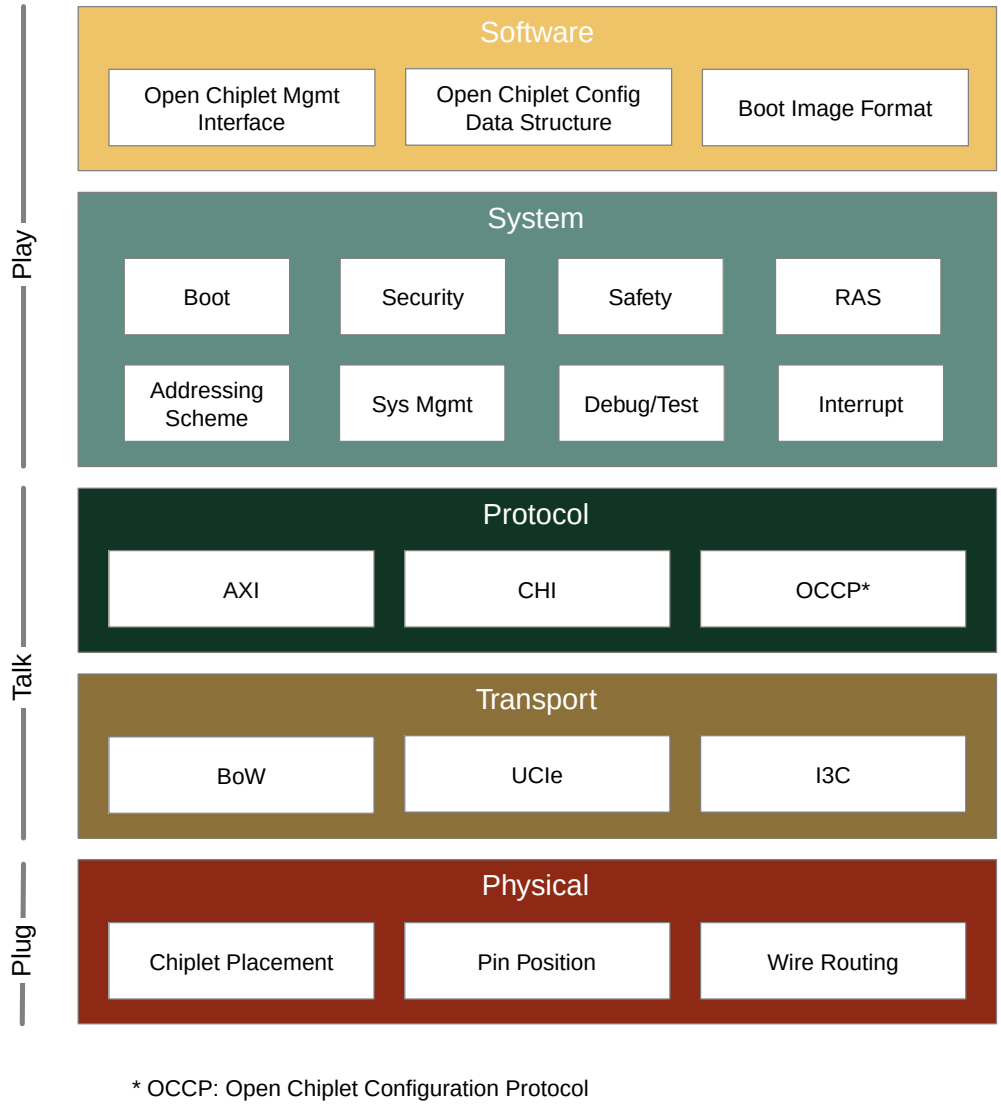

Figure 1. Layered view of Chiplet Interoperability

Below are detailed explanations of the five architectural layers:

1. Physical Layer - Defines physical integration rules on chiplet placement, orientation, wire routing, etc. The physical layer is staged for later OCA specification releases.
2. Transport Layer - Enables data transfer between chiplets. Examples include UCIe, BoW, and I3C.
3. Protocol Layer - Defines standard mechanisms for interpreting data packets and mapping them to local fabrics or buses, like AXI and CHI.
4. System Layer - Ensures chiplets work together by creating a coherent system for boot, management, security, safety, RAS, QoS, interrupts, debugging, and testing.
5. Software Layer - Defines standard configuration data structure and software interfaces.

Layers are designed with the objectives of plug, talk, and play. Standardizing the Physical Layer is crucial for ensuring chiplets can "plug" together seamlessly. Enabling "talk" between chiplets requires standardization at the Protocol and Transport Layers. For chiplets to "play" together effectively, alignment at the System and Software Layers is essential.

While some transport and protocol solutions, such as UCIe and AMBA protocols over UCIe, have been widely studied, the System and Software layers have received far less attention. Moreover, the lack of a systematic approach has left the definitions of the Transport and Protocol layers incomplete. The OCA architecture 1.0 takes a comprehensive approach, addressing Transport, Protocol, System, and Software Layers to ensure interoperability.

The physical layer will be defined as a new chapter in a subsequent release after the initial architecture 1.0. Until then it is up to the chiplet designer to create a physical layer solution that meets the needs of their chiplet and the SiP

integrator to ensure seamless integration. There are indeed physical layers in UCIe and BoW while they are omitted for simplicity in this initial release and will be updated once more details are available in the physical layer.

2.3.2. Integration Examples

This section illustrates three chiplet integration examples, with the OCA architecture related components highlighted in color.

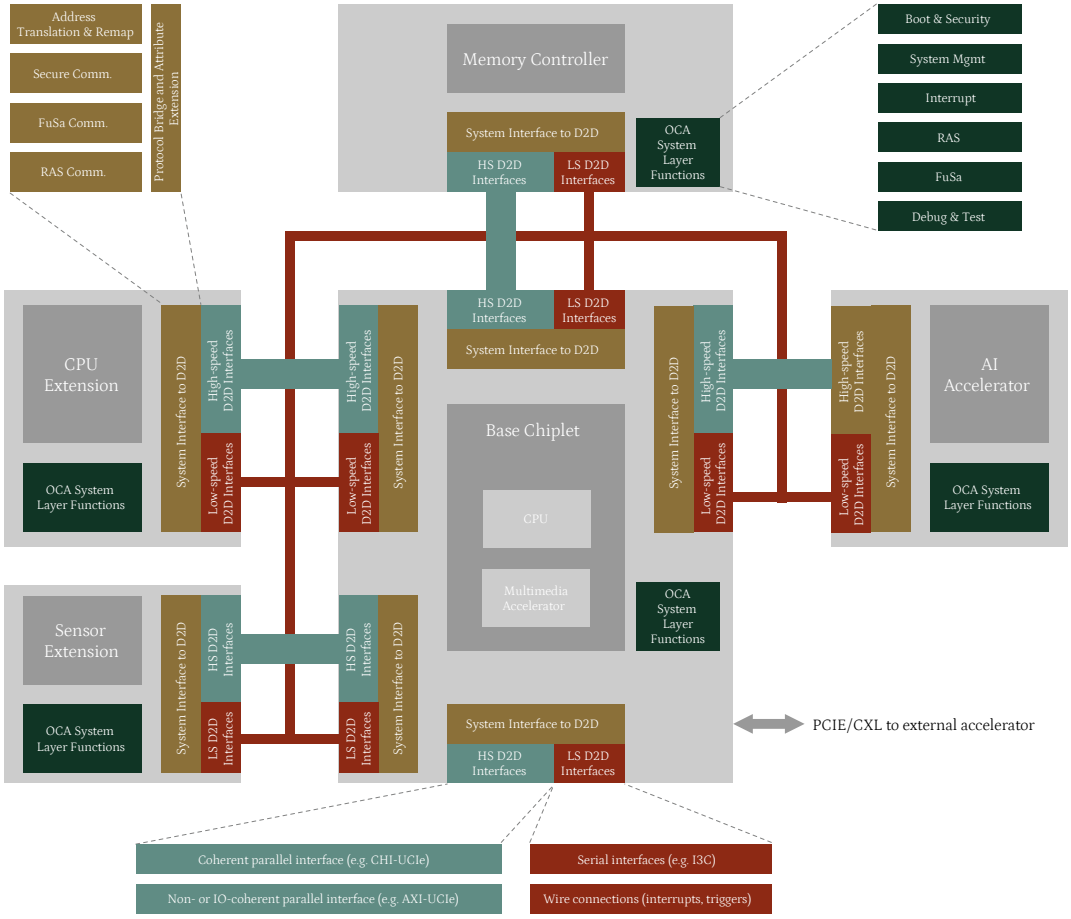

Figure 2. Extended Chiplet Integration with a Base Chiplet

Figure 2 shows an extended integration where a base chiplet implements basic functions sitting in the center with extension chiplets surrounding it as function extensions. The D2D interfaces along the southern edge can be used to connect other extension chiplets or base chiplets.

In general, the OCA architecture related components in one chiplet can be categorized in the following groups:

- **D2D Interfaces**: This maps to the Package Physics Layer and the Transport Layer as shown in Figure 1. It can be further divided into high-speed interfaces, like the UCIe connections, and low-speed but highly available interfaces, like the I3C connection.
- **System Interface to D2D**: This maps to the Protocol Layer which includes the bus protocol bridge and the domain specific traffic attribute as shown in Figure 1. For example, the security domain information is taken into account when mapping the AXI protocol to the UCIe streaming protocol.
- **OCA System Layer Functions**: This maps to the architecture definition at the System Layer as shown in Figure 1.

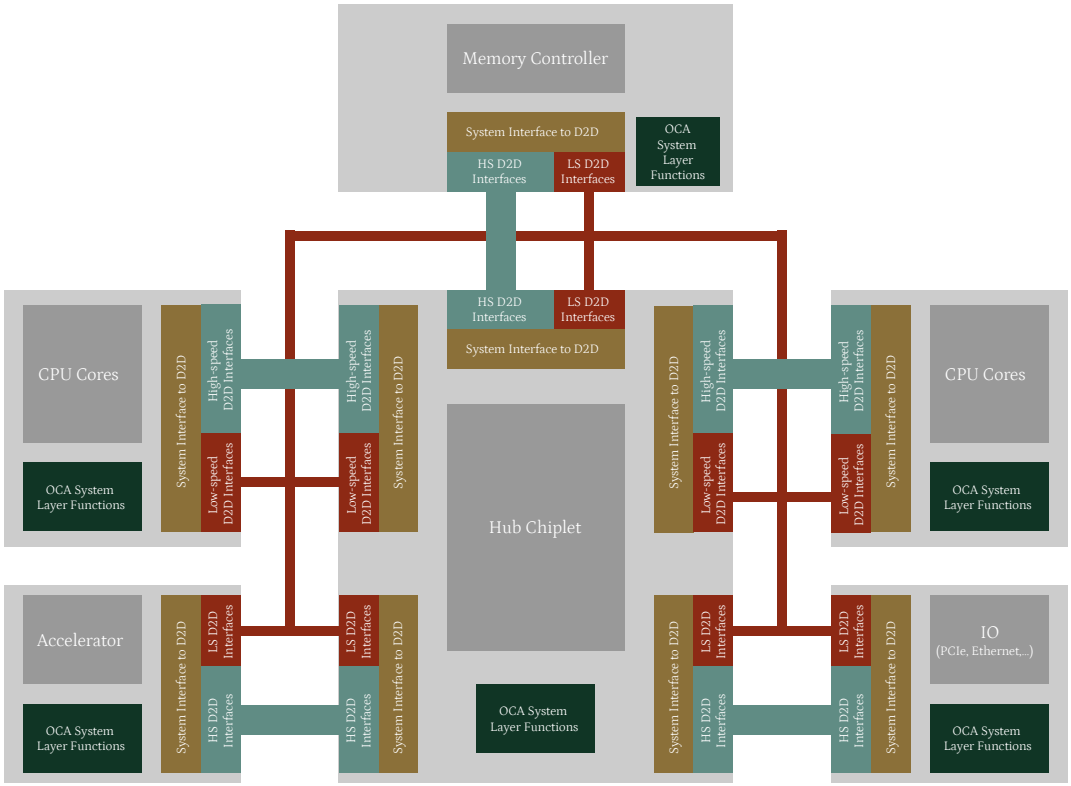

Figure 3. Centralized Chiplet Integration with A Hub

Figure 3 shows a centralized chiplet integration example with a central hub chiplet. Unlike the extended integration in Figure 2, the central hub chiplet focuses on connection. The CPU chiplets are not an extension, but the main CPU clusters.

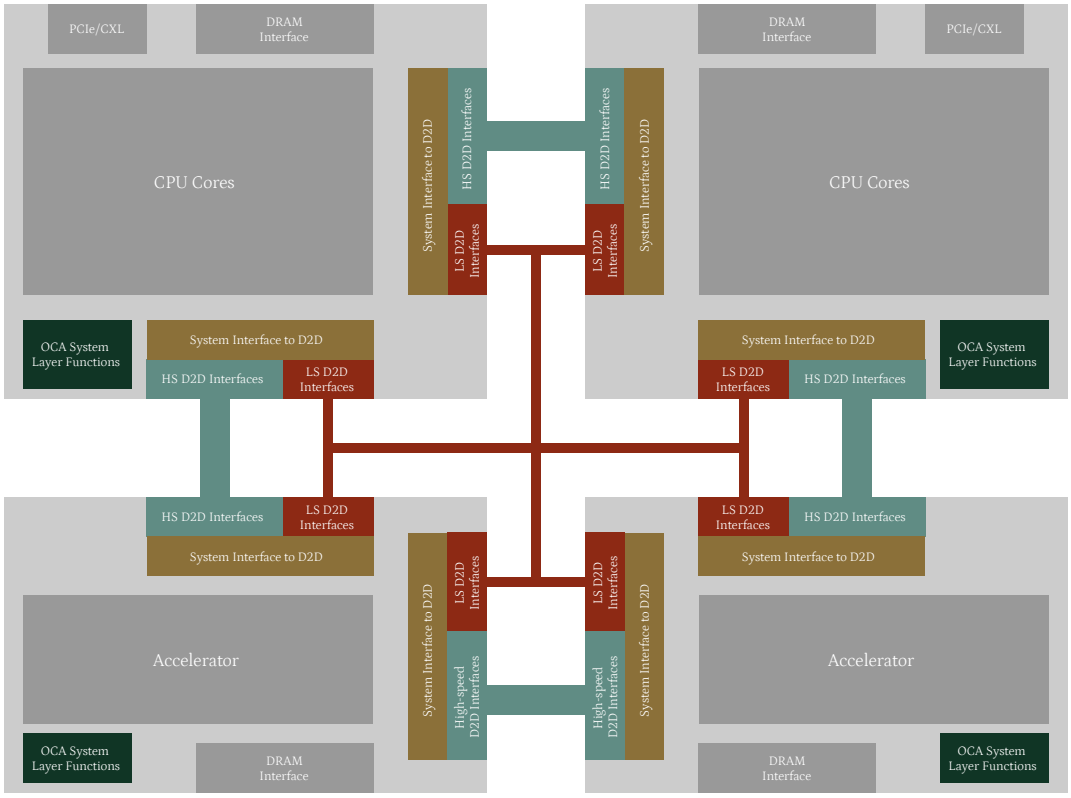

Figure 4. Distributed Chiplet Integration without A Hub

Figure 4 shows a distributed chiplet integration without a central base chiplet or hub chiplet.

2.4. Design Principles for OCA Architecture

The OCA architecture is defined based on the following principles:

- **Open**: The architecture an open architecture that fosters collaboration, reduces costs, and ensures interoperability. To avoid vendor lock-in, the OCA architecture only relies on technologies with open specifications.
- **Industry-standard based**: The architecture leverages existing industry-standard solutions wherever possible.
- **Accessible**: The architecture prefers solutions with widely available IPs.

2.5. Interoperability and Flexibility

Flexibility often comes at the cost of interoperability. The OCA architecture follows the following methods to strike a balance between the two:

- When narrowing down to a single solution is impractical, the architecture considers each viable option in its architecture. Interoperability in this case is among chiplets that adopt the same option. For example, while some chiplets may use UCIe as their transport mechanism and others may opt for BoW, the architecture does not mandate a single choice. Instead, it guarantees that chiplets can operate together if they all adopt identical and compatible protocol/transport/physical options. There are unfortunate incompatibilities even within the same transport mechanism, the OCA ecosystem intends to bridge some of the gaps.
- Different solutions may be favored by different use cases and market segments. To address this, the OCA architecture introduces the concept of profiles. Each profile is a refinement of the OCA basic definition through selection of functional domains and the assignment of parameter values, tailored for specific families of chiplets. Certain features are profile-dependent, meaning they are required only in specific profiles. Interoperability is not guaranteed between chiplets conforming to different profiles. If a chiplet is required to support multiple profiles it must meet the superset of all profile requirements, with individual profiles activated selectively as needed.

2.6. Document Organization

The remainder of this document is structured as follows:

- A general architecture is introduced in Chapter 3, Chapter 4, Chapter 5, and Chapter 6. The general architecture includes common functional domains, such as addressing, boot, and system management and usage-specific functional domains, such as RAS and functional safety.
- The profile-based definition (Chapter 7) refines the general architecture by applying profile-specified choices.
- Definition of the functional domains and the profiles always follows the layered definition.

Chapter 3. Foundations of the OCA Architecture

This chapter defines foundational assumptions in the OCA architecture.

3.1. Chiplet Roles

The OCA architecture supports a hierarchical system, where chiplets can assume different roles, some acting as primaries and others as secondaries. The primary chiplet serves as the main control element for a specific function domain. For each function domain, there is usually one primary chiplet in a package, while all other chiplets operate as secondaries. Some domains naturally have no primary chiplet, such as the Interrupt domain. Some others may have a primary chiplet outside of the package, and we will cover those cases in later revisions of this document.

Different function domains have different requirements for the primary chiplet. The following are definitions of the primary chiplet's properties required by each domain.

- Boot Primary: the chiplet boots first with access to necessary peripheral interfaces and the responsibility to bring up the rest of the SiP.
- System Management Primary: The chiplet oversees the system management tasks for the entire SiP and interfaces with the system software.
- System Time Management Primary: The chiplet holds the reference time for the SiP.
- Security Primary: The chiplet acts as the Root of Trust (RoT) for the SiP, ensuring SiP integrity and authenticity.
- Interrupt Primary: N/A
  - There is no Interrupt primary from an interoperability perspective. Interrupts are delivered to the target chiplet from the source without a central controller.
- Debug Primary: the chiplet interfaces to the external debugger and routes debug traffic to other chiplets.
- DFT Primary: The chiplet interfaces to an external tester and routes test traffic to other chiplets.
- RAS Primary: The chiplet acts as the top of the error aggregation tree.
- FuSa Primary: The chiplet acts as the top of the error aggregation tree.

It is common for a single chiplet to serve as the primary chiplet across all function domains, as demonstrated by the Base Chiplet in Figure 2. However, a SiP integrator may choose to designate different chiplets as the primary for distinct function domains, for example, assigning separate primary chiplets for Security/Boot/Debug/DFT and for System Management/System Time Management/RAS/FuSa.

[OPT-FND-1] A chiplet intended to serve as the central controller of the SiP should be designed to support the primary functions of all relevant domains.

It is the responsibility of the SiP integrator to designate the primary chiplet for each function domain. The SiP integrator should consider the need of the SiP and capabilities of the chiplets when making this decision.

It is not uncommon to multiply instantiate chiplets in a SiP. In such cases, the SiP integrator need the same chiplet to be able to serve as the primary for a function domain in one instance and as a secondary in another instance.

[OPT-FND-2] Chiplets should be designed with secondary capabilities and interfaces of the functional domains of interest to support multi-instantiation, enabling the SiP integrator to flexibly designate the appropriate primary for each function domain.

3.2. Chiplet Topology

The OCA architecture compliant chiplets in a package and their connections form a Directed Acyclic Graph. Each node in the graph is a chiplet. The edges indicate physical connections between chiplets. The direction of the edges

doesn't define uni- or bi-directional traffic. Instead, the direction specifies the management hierarchy. Following the direction of a directed edge, the graph moves from an upstream chiplet to a downstream chiplet. The upstream chiplet is at the same or higher management level than the downstream. The following are some terms the OCA architecture uses to describe topology related topics.

- Root: the topology root of a directed acyclic graph, which is also the primary chiplet in a function domain. Root and Primary are interchangeable in this document.
- Leaf: An end chiplet with no downstream chiplet.
- Non-leaf: An intermediate chiplet with at least one upstream chiplet and one downstream chiplet.
- Upstream and downstream: Upstream chiplets of chiplet *U* refers to all chiplets that have a directed path to chiplet *U*. Downstream chiplets of chiplet *U* refers to all chiplets that can be reached from chiplet *U* via a directed path.

The topology view is domain specific. The primary chiplet of a function domain always sits at the root of that function domain's topology view. Based on this structure, the management hierarchy is built, and the rooted graph is defined.

Figure 5 shows the System Management topology view and Security topology view of one SiP.

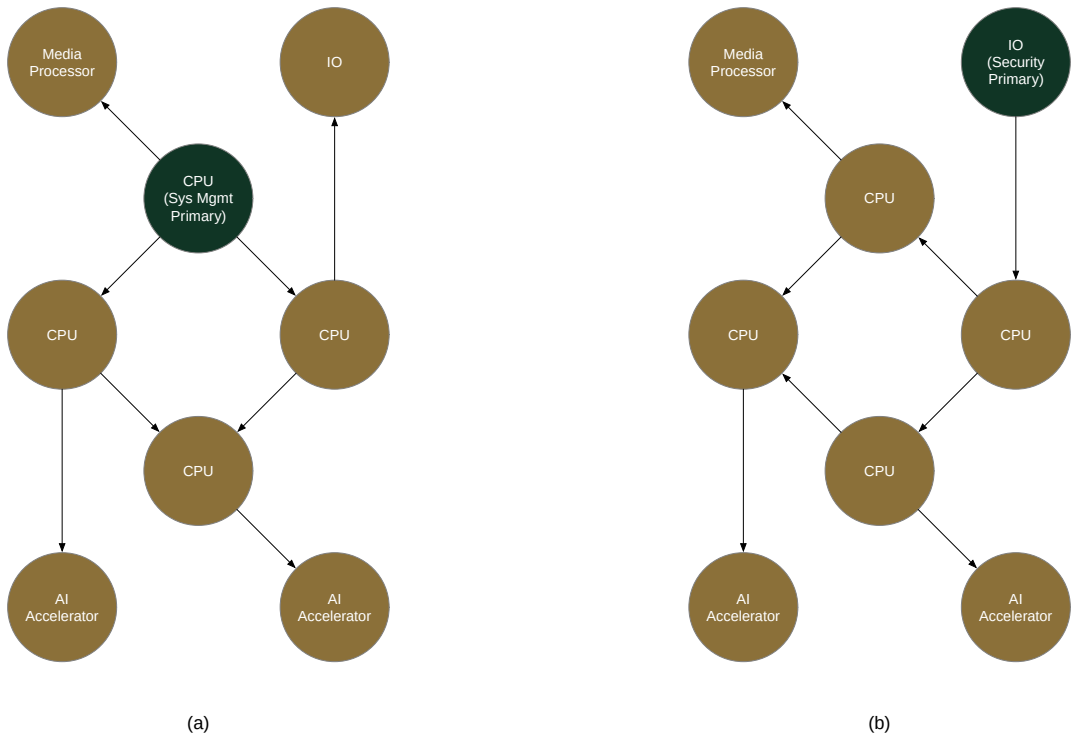

Figure 5. Examples of chiplet topology: 4 CPU chiplets are connected in a symmetric way, forming a CPU compute cluster. Accelerator and IO chiplets are connected to different CPU chiplets; (a) In System Management's topology view, one of the CPU chiplet is the primary; (b) In Security's topology view, the IO chiplet is the primary.

Figure 6 shows the topology view of another SiP, where the base chiplet at the center serves as the primary chiplet for both System Management and Security.

Information about the chiplet topology configuration is provided to software through a standardized data structure defined in Section 6.2. Because the topology is fixed after packaging, no discovery is required during boot time or run time. This data structure is provided by the SiP integrator.

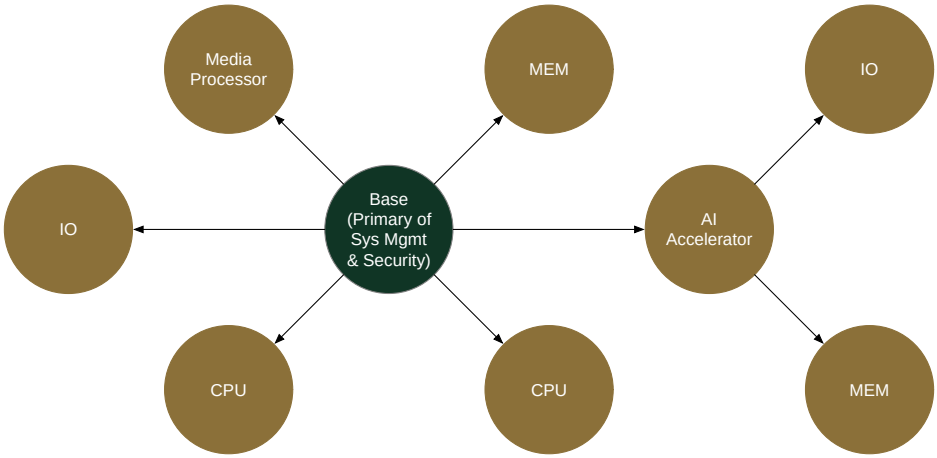

Figure 6. Example of chiplet topology: A base chiplet, as the primary chiplet for all function domains, is at the center of the SiP. CPU, accelerator, and IO chiplets are connected to the base chiplet.

Chapter 4. System Layer

This chapter defines a general architecture at the System Layer. It specifies basic functional domains of the OCA architecture, including Communication Channels, Addressing, Boot, System Management, Security, Interrupts, Design for Debug, and Design for Test. It also specifies additional functional domains such as FuSa and RAS.

4.1. Communication Channels

The OCA architecture uses Channels to describe abstracted and logical inter-chiplet communication pathways. This section defines Channels and corresponding architectural requirements. The architecture categorizes channels as application and non-application channels. Application channels support primary compute elements and high-speed I/O, while non-application channels are dedicated to specific functions like boot and system management.

A Channel is an architectural concept and not always equivalent to a physical connection. Some Channels may be merged and mapped to a single physical connection, while others may need to be split across multiple physical connections. Each physical connection operates with its own bus protocol (e.g., AXI, CHI) and transport mechanism (e.g., UCIe, BoW). Further details are provided in Chapter 5.

4.1.1. Application Channels

Application Channels connect components like CPUs, accelerators, and I/O devices to carry out the functional objectives of the SiP.

The OCA architecture categorizes Application Channels into the following types: **Coherent**, **IO-Coherent**, and **Non-Coherent**.

Coherent Channel: A communication pathway that ensures memory reads always retrieve the most up-to-date copy. It enables multiple compute chiplets to be interconnected, forming a coherent compute cluster. When a chiplet needs to form a coherent compute cluster with chiplets from other vendors, a standardized Coherent Channel needs to be used to access the memory hierarchy, including caches and system memory located on or attached to other chiplets. In some cases a cluster is composed entirely of chiplets from the same vendor, allowing the vendor-specific interconnect implementations.

Non-Coherent Channel: A communication pathway that does not enforce memory coherence between connected components. Unlike coherent channels, it does not guarantee that memory reads will always reflect the most recent writes made by other components. Non-coherent Channels are typically used for peripheral communication or memory access where strict memory consistency across multiple compute elements is not required or guaranteed by software.

IO-Coherent Channel: A communication pathway that allows a source chiplet to participate in a target chiplet's memory coherence protocol. It ensures that data accessed by the source chiplet is kept coherent with the target chiplet's caches and memory. This means that the source chiplet can read and write shared memory without explicit software-managed cache flushes or invalidations.

Figure 7 illustrates how the SiP setup examples in Figure 5 and Figure 6 have application Channels implemented.

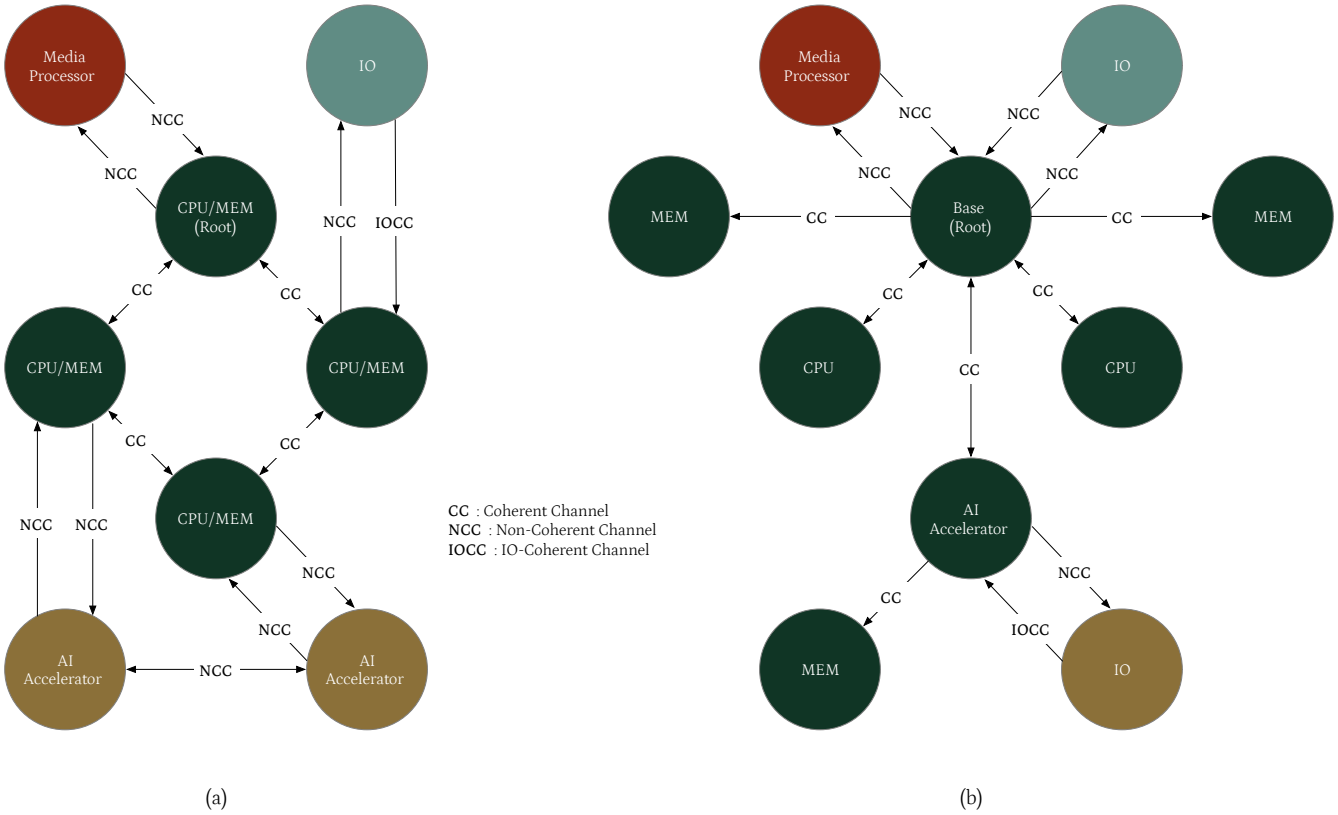

Figure 7. Examples of chiplet application channels: (a) 4 CPU chiplets are connected with the Coherent Channels to form a compute cluster. The Non-Coherent Channel is used for the CPU chiplet cluster to access the peripheral chiplets. The accelerators use the Non-Coherent Channel to access system memory. The IO chiplet uses IO-Coherent Channel to access system memory; (b) A base chiplet, 2 CPU chiplets, an AI accelerator, and memory chiplets are connected via the Coherent Channel to form a heterogeneous compute cluster. The Non-Coherent Channel is used for the heterogeneous compute cluster to access peripheral chiplets. Peripherals use the IO-Coherent Channel or the Non-Coherent Channel to talk with the heterogeneous compute cluster and system memory.

4.1.2. Non-Application Channels

Besides application channels, the OCA architecture supports the following Non-Application Channels:

- Boot Channel (Section 4.3.4)
- Address Translation Channel (Section 4.2.2.1)
- System Management Channel (Section 4.4.3)
- System Time Management Channel (Section 4.5)
- Security Channel (Section 4.6)
- Interrupt Channel (Section 4.7)
- RAS Channel (Section 4.11)
- Debug Channel (Section 4.8)
- Test Channel (Section 4.9)
- FuSa Messaging Channel (Section 4.10.2)

These Non-Application Channels are detailed in their domain specific definition.

4.1.3. Inter-chiplet Deadlock Prevention

Deadlock in a network occurs when packets become stuck in a circular dependency, where each packet is waiting for another to release a resource (like a buffer or link) that it needs to proceed, leading to a standstill. There are in general two kinds of deadlocks:

1. Protocol-driven deadlocks (PDD): primarily when a protocol is translated, regardless of chiplet.
2. Topology-driven deadlocks (TDD): Overall topology and other chiplets are to be defined when a chiplet is designed.

Because protocol translation may happen on an inter-chiplet interface, and an inter-chiplet transaction may traverse through intra-chiplet fabric, some basic requirements are established as follows:

[REQ-FABRIC-1] Independent progress shall be available in the fabric between traffic classes.

Traffic class is a general terminology for different traffic groups per access type, QoS, sender/receiver, or other traffic attributes. A particularly vulnerable area arises during protocol translation, where traffic classes may be remapped, combined, or separated. Special attention must be given to these points to ensure that such transformations do not introduce deadlock risks.

[REQ-FABRIC-2] An outbound transaction shall not be made dependent on an inbound transaction.

In addition to the above requirements to prevent protocol-driven deadlock, the following requirements are required to prevent inter-chiplet deadlocks resulted from topology or routing.

[REQ-FABRIC-3] The routing of a transaction back to its originating component, a condition known as a U-turn or loop-back path, shall be prohibited.

To simplify the discussion and design, fabrics are classified into two buckets:

- Non-performance-sensitive fabrics prioritize low-complexity connectivity over high throughput and bandwidth.
- Performance-sensitive fabrics have more requirements on throughput and bandwidth with the primary goal to deliver optimal performance.

The ultimate goal of inter-chiplet topology and routing through it is to be acyclic for both kinds of fabrics. There is a chance for non-performance-sensitive fabrics to stay with simpler topologies and thus routing.

[OPT-FABRIC-1] Non-performance-sensitive inter-chiplet topology should be acyclic.

The topology selection, a bus or a tree topology, for example, rules out cycles physically to avoid main causes of routing deadlock. If this is not feasible, follow the requirements as follows.

[REQ-FABRIC-4] Each chiplet shall provide deadlock prevention, at a minimum through turn restrictions, and its programming interface to the SiP integrator, so that the inter-chiplet topology could be acyclic.

An inter-chiplet route starts when a transaction is injected into the fabric until it reaches its final destination on a remote chiplet. These requirements indicate that the following three segments of programming must be configured by the SiP integrator to prevent deadlock:

1. At the originating chiplet the SiP integrator must configure the route of an inter-chiplet transaction between the sender to an inter-chiplet port exit.
2. Within a pass-through chiplet the SiP integrator must configure the route from the entry port to an exit port.
3. At the destination chiplet the SiP integrator must configure the route a transaction takes from the entry port to its final receiver.

A turn restriction is a prohibited route where multiple routes are available. For example, there are three options (1, 2 and 3) in a 4-port router in which a transaction comes in from physical port 0.

The OCA architecture mandates turn restriction as the baseline mechanism for deadlock prevention. By programming both inter-chiplet routing and turn restrictions, a SiP integrator can ensure a deadlock-free topology that delivers reasonable performance.

However, advanced functionalities—including load balancing, QoS, dynamic re-routing, and support for extreme topologies depend on more sophisticated fabric capabilities like virtual channels or deadlock detection and avoidance. These advanced methods fall outside the scope of generic interoperability and are therefore defined on a product-specific or profile-specific basis.

4.2. Addressing

4.2.1. Address Domains

An address domain refers to a scope within which a set of addresses is used to access memory or other resources. A single chiplet may contain multiple address domains. An address domain can also span multiple chiplets. Address maps of different address domains are independent — the same address in different domains may refer to entirely different resources.

The following types of address domains are supported by the OCA architecture:

- **Local Address Domain**: An address domain confined to a single chiplet. This allows chiplet designers to define its address map independently, without concern for conflicts with other address domains. Address translation is required for related cross-chiplet traffic as they originate from a different address domain. Most address domains in a SiP fall into this category.
- **Shared Address Domain**: An address domain that spans multiple chiplets, using a unified address map in which each address is uniquely assigned across the participating chiplets. Inter-chiplet communication in the same shared address domain does not require address translation. For example, multiple CPU chiplets can be in a shared address domain, with each chiplet assigned non-overlapping address regions in the unified address map. Similarly, when an address translation cache is integrated into a device chiplet, the fabric in the device chiplet that carries the translated addresses operates in the same Shared Address Domain as the host chiplet(s).

[REQ-ADDR-1] Each chiplet shall clearly classify each of its address domains as either a Local Address Domain or a Shared Address Domain.

The address width of a Local Address Domain is implementation-defined.

[REQ-ADDR-2] When inter-chiplet traffic stays within the same Shared Address Domain, its address width shall be SAD_ADDR_WIDTH, which is profile-dependent.

[REQ-ADDR-3] The address space of a Shared Address Domain shall not exceed 2^SAD_ADDR_WIDTH bytes.

In some cases, a Shared Address Domain is used exclusively by chiplets from the same vendor, like when multiple identical CPU chiplets are integrated to form a large compute cluster. In this scenario, the Shared Address Domain can follow a proprietary design, with implementation-defined address width for inter-chiplet requests and domain size.

4.2.2. Address Translation

Address translation allows independent address map definition of different address domains.

[REQ-ADDR-4] When traffic moves from the source address domain to the target address domain, the target domain shall be responsible for performing address translation for incoming traffic.

[OPT-ADDR-1] Configuration of address translation should be performed by agents operating in the target address domain.

This approach decouples proprietary address translation mechanisms from the standardized interface used for address region allocation (Section 4.2.3). It also supports enhanced security by isolating translation control within the

target domain.

[REQ-ADDR-5] When inter-chiplet traffic crosses address domains, its address width shall be D2D_ADDR_WIDTH, which is profile-dependent. All address bits shall be made available to address translation.

Address domain boundaries do not necessarily align with chiplet boundaries, and a target address domain may extend into the source chiplet. In such cases, address translation at the entry point of the target address domain is physically located within the source chiplet. This scenario can be further categorized into the following sub-cases:

1. Self-Sufficient Address Translation: For example, a fully featured IOMMU handles address translation independently. In this case, only the translated traffic is transmitted over the D2D interface to the target chiplet.
2. Non-Self-Sufficient Address Translation: For example, address translation relies on a TLB-like translation cache, like the PCIe Address Translation Cache, while a fully featured IOMMU resides in the target chiplet. Here, the D2D connection carries both translated traffic and address translation requests.

Figure 8 illustrates an example of address translation across different address domains. Two CPU chiplets share a unified address domain, eliminating the need for address translation between them. Other chiplets, however, operate in independent local address domains. As traffic enters the CPUs' Shared Address Domain, dynamic address translation is performed by an IOMMU, while static address translation is handled through remapping. Address translation occurs whenever traffic crosses address domain boundaries. Additionally, the CPUs' shared address domain extends into the Accelerator chiplet, where an Address Translation Cache (ATC) is implemented. Memory access traffic from the Accelerator chiplet to the CPU chiplets is already translated, while the ATC forwards address translation requests to the IOMMU in the CPU chiplet.

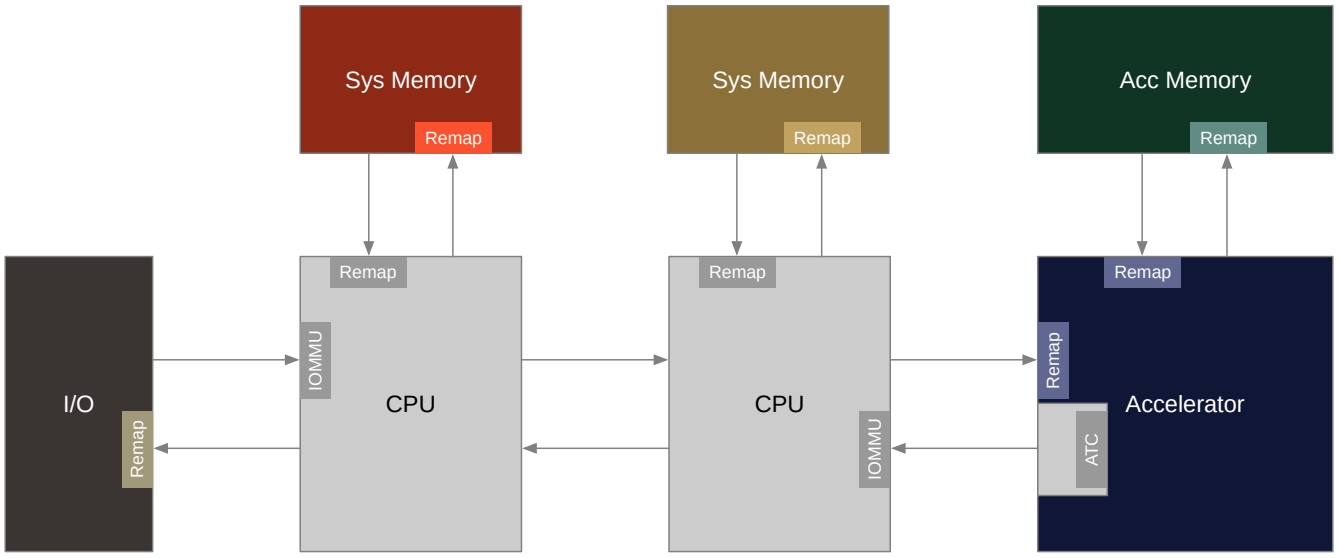

Figure 8. Example of address domains and address translation between them.

4.2.2.1. Address Translation Channel

Address Translation Channel is employed to handle address translation, invalidation, and page requests. The Address Translation Channel is bidirectional enabling it to send translation requests and page requests to the translation agent and to send translation responses and page request responses to the translation requester.

4.2.2.2. Address Translation Channel Properties

The properties of the address translation channel, like quantity, reset earliness, availability, error handling, etc., are usage defined. It is usually defined in synchronization with the supported channel(s) like the coherent channel.

4.2.3. Address Region Allocation

[OPT-ADDR-2] Inter-chiplet D2D ports should be allocated with the largest possible address spaces.

Address region allocation for the D2D ports is crucial for chiplets with static address maps. The required region size depends on unforeseen chiplets connected on the opposite side of the D2D connection. If a chiplet does not allocate a sufficiently large address region for the D2D port to accommodate the connected chiplets, it will lack a complete view of the resources in those chiplets.

Additionally, the address translation logic at the entry point of the target address domain must understand the address region allocation on the opposite side of the D2D connection.

[REQ-ADDR-6] The number of the address regions allocated to a D2D port in the source address domain shall be equal to or smaller than 4.

[REQ-ADDR-7] Each address region allocated to a D2D port in the source address domain shall be contiguous.

[OPT-ADDR-3] The base addresses of the address regions allocated to a D2D port in the source address domain should be provided to the agent that configures the address translation logic in the target domain via the Boot Channel (Section 4.3).

4.2.4. Inter-chiplet Routing of a Request-Response Pair

An inter-chiplet pair, which traverses chiplet boundaries, involves an outgoing request and an incoming response. For example, a CPU chiplet may issue a memory read request to a memory controller chiplet. In such cases, the response typically omits the original address, and the routing context used by the responder may be specific to its local chiplet context. This creates several challenges:

- The response must be routed back to the original requester, but the necessary routing context may not be available at the D2D interface.
- The lack of address information in the response complicates the identification and delivery of the response to the correct source.
- Existing mechanisms for handling response routing across chiplets are insufficient, and further work is required to address these issues.

Following the same concept of address domains, two kinds of routing domains are considered: Local Routing Domain and Shared Routing Domain.

- **Local Routing Domain**: A routing domain that is wholly contained within and exclusively managed by a single physical chiplet. All components within this domain reside on that one chiplet.
- **Shared Routing Domain**: A single, contiguous routing domain that spans two or more physical chiplets. Components on different chiplets within the same Shared Routing Domain can communicate without their transactions needing to cross a domain boundary.

[REQ-ADDR-8] Each chiplet shall clearly classify each of its routing domains as either a Local Routing Domain or a Shared Routing Domain.

4.2.4.1. Inter-Chiplet Routing Across Domain Boundaries

When crossing Local Routing Domain boundaries, each inter-chiplet pair of request and response must be uniquely identifiable and routable. This is easier for the request as it carries the full address and other routing information. However, the response typically lacks the original address, making it challenging to route it back to the requester. To address this, an inter-chiplet routing context is introduced and required in inter-chiplet pairs of request and response.

Inter-Chiplet Routing Context: The complete set of information required to route an inter-chiplet request and its corresponding response. This context must be sufficient to uniquely determine the transaction's original source and its final destination. The representation of this context is protocol-dependent and may be implemented in several ways:

- **Explicit Addressing**: The context is directly encoded in the packet header using distinct fields for identifying the source and destination (e.g., Source ID and Destination ID).
- **Inferred Addressing**: The context is represented by a unique transaction identifier (e.g., an AXI AWID or ARID). The routing fabric is then responsible for using this identifier to internally track and resolve the transaction's source and destination.

With at least either explicit or inferred addressing, the inter-chiplet pair can be routed correctly across Local Routing Domain boundaries. A protocol may choose to support both schemes, e.g., a source ID, a destination ID, and a transaction ID. Its NoC may employ all or some for added robustness.

[REQ-ADDR-9] When an inter-chiplet transaction crosses Local Routing Domains, its associated inter-chiplet routing context shall have a width of D2D_ROUTING_WIDTH, which is profile-dependent and protocol-dependent.

[REQ-ADDR-10] Chiplets forwarding the inter-chiplet pair shall use the inter-chiplet context for routing and ensure it is preserved in the packet until delivery to the original source.

The method for preserving the inter-chiplet routing context depends on the protocol and NoC implementation. For example, some NoCs reserve specific regions of routing context, such as source IDs and destination IDs, to represent chiplets. At the D2D interface, these IDs are remapped according to configurations set by the SiP owner, ensuring correct routing between chiplets. Other NoCs may use a unique transaction identifier, such as an AXI ID, to internally track the source and destination of each transaction. In this approach, the originating NoC assigns a unique transaction ID to each request, which is then preserved in the response as it returns to the original requester. Both NoCs use this transaction ID to route the response accurately back to its source.

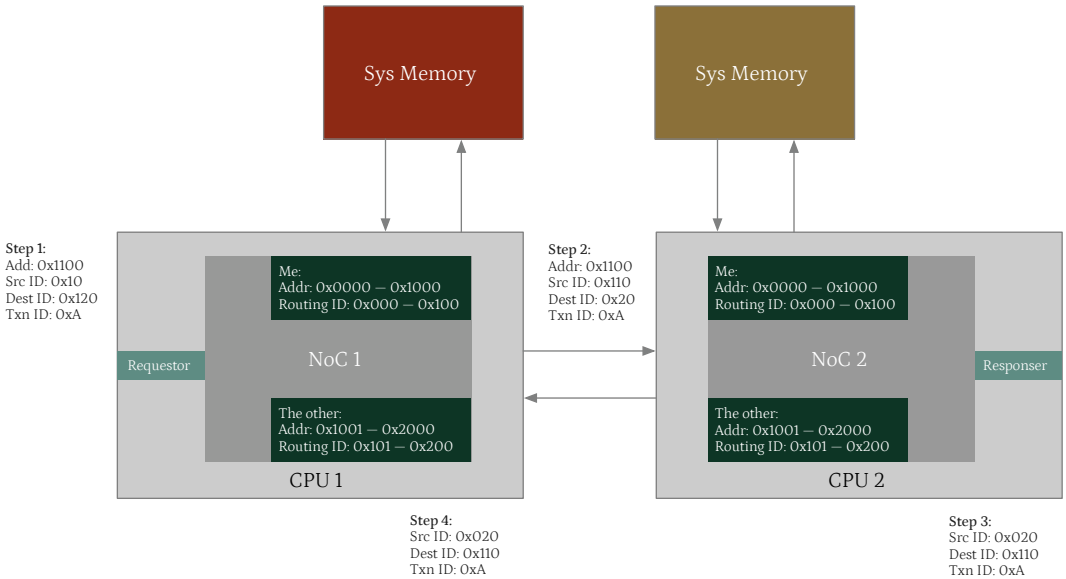

Figure 9. Inter-chiplet routing across Local Routing Domains.

Figure 9 illustrates inter-chiplet routing across Local Routing Domains and the handling of routing context in different NoC designs. In some implementations, each NoC may understand the other NoC as a range of routing IDs (i.e., routing context). Then the D2D interface may choose to remap the routing IDs statically, so that many-to-many communications are available between two routing domains. Some other implementations may use a unique transaction ID to track the source and destination of each transaction, ensuring accurate routing of responses back to their original requesters. AXI ID is a good example of such a transaction ID.

4.2.4.2. Inter-Chiplet Routing in a Shared Routing Domain

There may be an efficient routing method for closely coupled chiplets, such as multiple instances of the same chiplet in a System-in-Package (SiP). This allows them to communicate using a more direct, low-overhead addressing scheme instead of the more complex cross-domain routing.

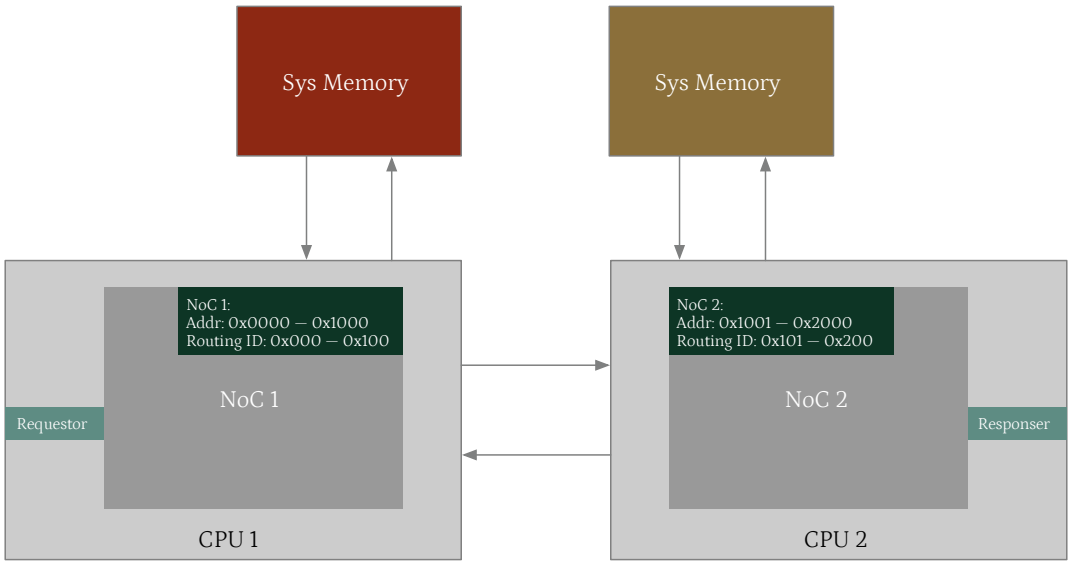

Figure 10. Inter-chiplet routing within a Shared Routing Domain.

Figure 10 shows an example of inter-chiplet routing between two CPU chiplets within a Shared Routing Domain. In this example, two CPU chiplets share a unified routing domain, allowing them to communicate directly without the need for complex inter-chiplet remapping or tracking. In such a domain the routing IDs (i.e., Routing Context) are globally understood by requestors and responders on both chiplets. When the address domain also aligns with the routing domain, this is very similar to a conventional monolithic CPU.

[REQ-ADDR-11] When an inter-chiplet pair of request and response stays within the same Shared Routing Domain, its routing conte width shall be SAD_ROUTING_WIDTH, which is profile-dependent and protocol-dependent.

[REQ-ADDR-12] The total routing space of a Shared Routing Domain shall not exceed 2^SAD_ROUTING_WIDTH addressable nodes.

4.3. Boot

Compared to monolithic SoCs, the boot process of a SiP must account for the following differences:

- Some chiplets are not self-sufficient for boot and rely on other chiplets to load their configuration and software.
- Assets, like intellectual property and secrets, may be owned by different entities.
- D2D communication needs to be established during the boot process.

The increased complexity and involvement of multiple stakeholders require a standardized boot flow to define owner responsibilities, inter-chiplet communication interfaces, and software interfaces between boot stages.

4.3.1. SiP Setup for Boot

A chiplet that is self-sufficient to load its configuration and software is called a self-boot chiplet. It has the capability and the necessary system setup to boot by itself. A self-boot chiplet can also help load configuration and software to other non-self-boot chiplets.

[REQ-BOOT-1] A SiP shall have the following boot setup:

- One and only one chiplet that is designated as the Boot Primary chiplet
- Zero or more chiplets that are configured as the Boot Secondary chiplets
- The Boot Channel for inter-chiplet communications

[REQ-BOOT-2] The Boot Primary chiplet shall be a self-boot chiplet that boots first in the SiP.

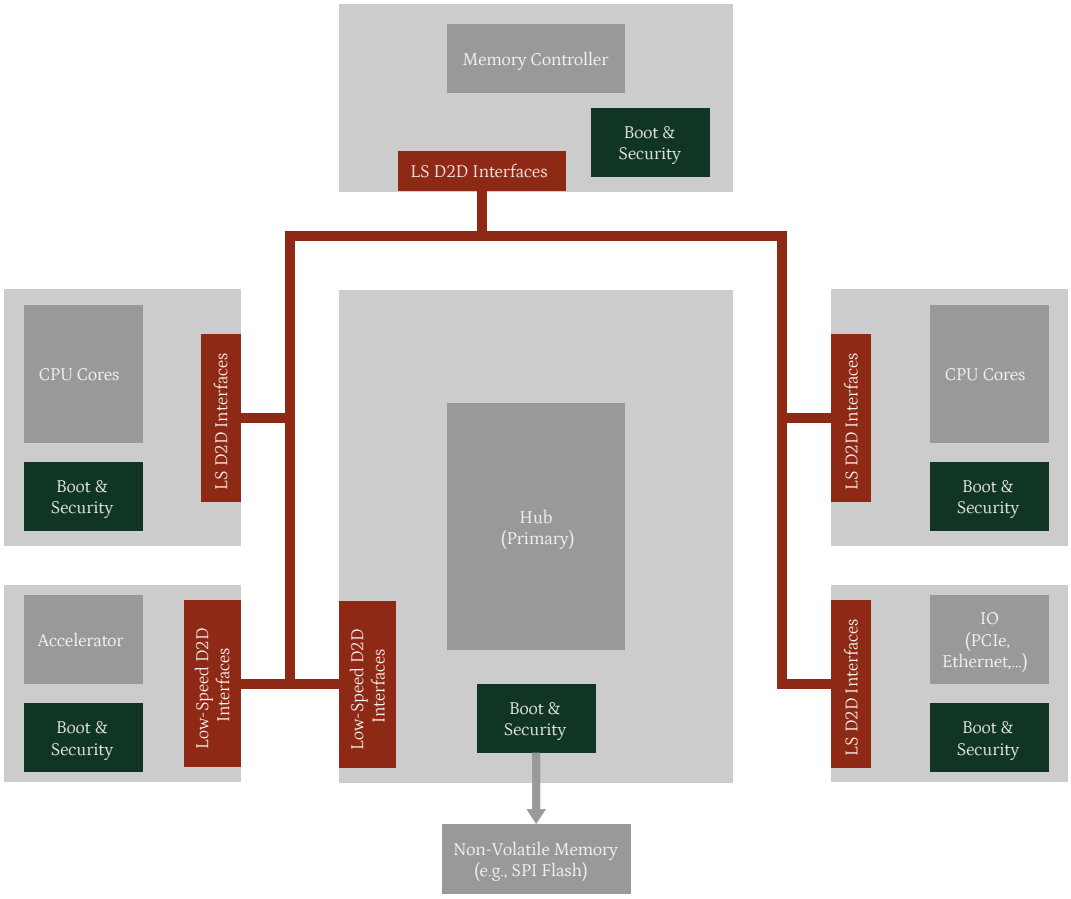

Figure 11. Flat Boot Setup

The Boot Primary chiplet can be the only self-boot chiplet in a SiP. In such a case, The Boot Primary chiplet is responsible for the boot-up of all the Boot Secondary chiplets. Figure 11 illustrates an example of this configuration, where a centralized Hub chiplet boots up first, with a non-volatile memory attached via an implementation-defined interface. Acting as the Boot Primary chiplet, this Hub chiplet is responsible for configuring and loading software or data to all Boot Secondary chiplets. During the early stages of the boot process, communication between the primary and secondary chiplets occurs over the Boot Channel running on the low-speed D2D connection.

A Boot Secondary chiplet can also be a self-boot chiplet and be responsible for the boot-up of some other Boot Secondary chiplets. This results in a multi-layer hierarchical boot structure within the SiP. Figure 12 illustrates such a hierarchical boot setup. In this example, one of the Accelerator chiplets functions as both a Boot Secondary chiplet and a self-boot chiplet. It is responsible for booting up the other Accelerator chiplet. A connection between the Boot Primary CPU chiplet and the self-boot Accelerator chiplet is still required for boot synchronization and boot status report.

[REQ-BOOT-3] The self-boot chiplet shall use a Boot Channel to communicate with other chiplets.

[REQ-BOOT-4] Each Boot Secondary chiplet shall connect to one and only one upstream self-boot chiplet.

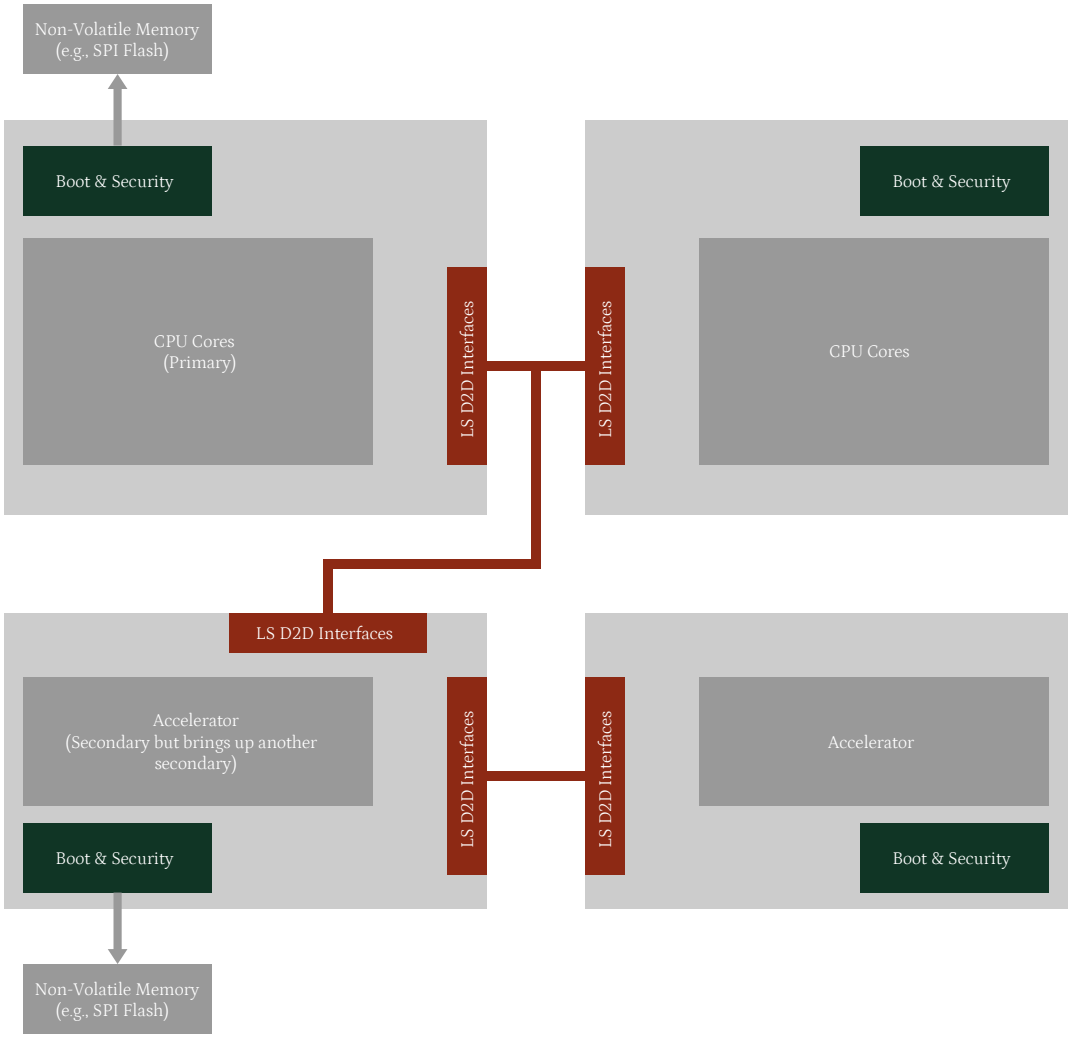

Figure 12. Hierarchical Boot Setup

4.3.2. Boot Flow

[OPT-BOOT-1] The boot flow of each chiplet should have a Chiplet Boot Stage and a SiP Boot Stage.

- **Chiplet Boot Stage**: The typical task for the Chiplet Boot Stage is to finish chiplet's local configurations and firmware loading to make it ready for further SiP bring up in the SiP Boot Stage. The Chiplet Boot Stage includes but is not limited to powering on chiplet's dedicated power rails, releasing subsystem resets, configuring PLLs, local fabric, and subsystem specific settings. If chiplet designer's IP needs protection, such as enforcing feature and performance constraints, this task also needs to be done at the Chiplet Boot Stage. Typically the boot ROM and the subsequent mutable software stages fall in this stage. The Boot Loader ROM (BL-ROM), and mutable software executed immediately after BL-ROM, called Boot Loader Chiplet (BL-CPL), are developed by chiplet designers. BL-CPL is optional depending on the needs of the chiplet designers.
- **SiP Boot Stage**: The objectives of the SiP Boot Stage include local chiplet configuration updates, bring-up of high-speed inter-chiplet connections, SiP configurations, and loading further software stages. The software running in the SiP Boot Stage is called Boot Loader SiP (BL-SIP). It is developed by SiP integrator.

Partitioning of chiplet and SiP boot stages considers the ownership difference between a chiplet and the SiP.

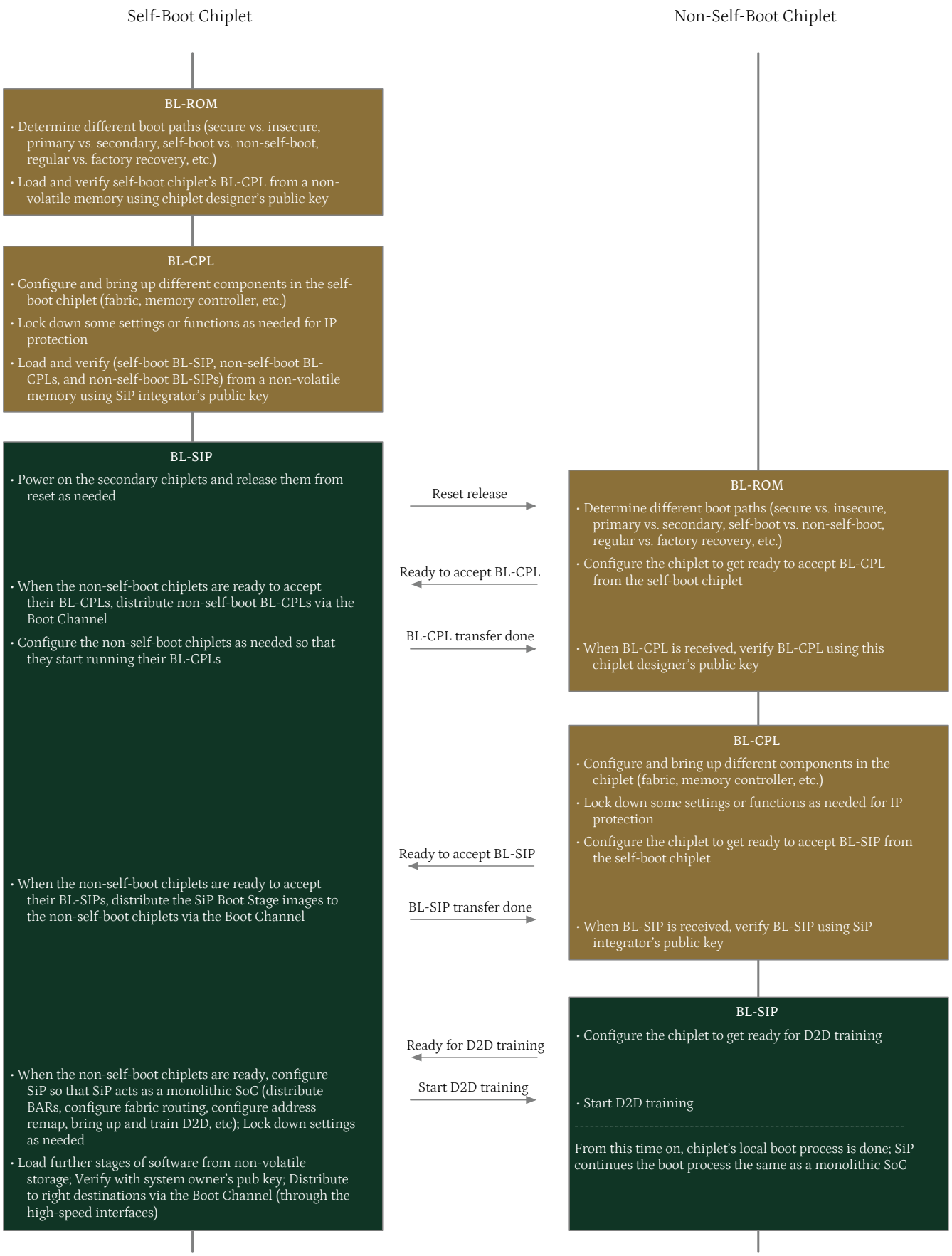

Figure 13. Boot Flow (the secondary chiplet requires local secure boot)

Figure 13 shows an example of the boot flow where the non-self-boot chiplet mandates secure boot. The self-boot chiplet's BL-SIP, BL-CPL, and BL-SIP of the non-self-boot chiplet, are bundled and signed with the SiP integrator's private key. In the bundle, the non-self-boot chiplet's BL-CPL is signed with the corresponding chiplet designer's private key.

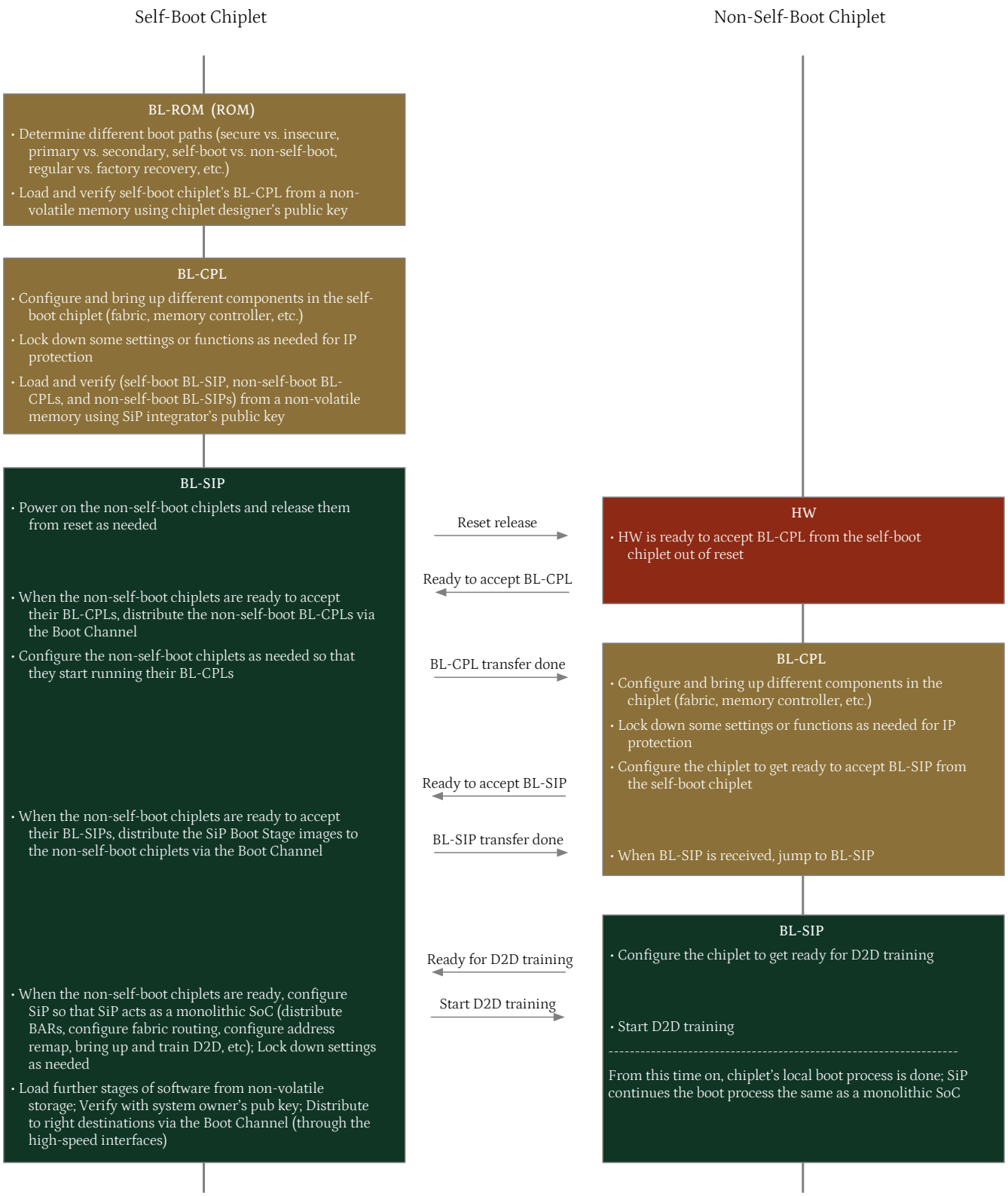

Figure 14. Boot Flow (the secondary chiplet doesn't need secure boot)

Figure 14 shows another example of boot flow where the non-self-boot chiplet lacks secure boot support. The self-boot chiplet's BL-SIP, BL-CPL, and BL-SIP of the non-self-boot chiplet, are bundled and signed with the SiP integrator's private key. In the bundle, the non-self-boot chiplet's BL-CPL is not signed by any private key.

4.3.3. Boot Report

Downstream chiplets in the boot topology use the Boot Channel to report boot status to upstream chiplets. All reports are eventually aggregated by the Boot Primary chiplet. The Boot Primary chiplet can further forward reports to a board-level controller, like a BMC, through implementation-defined interfaces.

[REQ-BOOT-5] When a boot report is required it shall be stored in the downstream chiplets and made available to the immediate upstream self-boot chiplet.

[REQ-BOOT-6] A Boot Secondary chiplet shall use the Boot Channel to notify its immediate upstream self-boot chiplet when a boot report is available.

[REQ-BOOT-7] The upstream self-boot chiplet shall retrieve boot reports via the Boot Channel. If the upstream self-boot chiplet is not the Boot Primary chiplet, it shall notify its immediate upstream self-boot chiplet through the Boot Channel.

The location of the report shall be provided as configuration data. For more information see Section 6.2.

4.3.4. Boot Channel

[REQ-BOOT-8] The Boot Channel shall be immediately ready to use when power and clock are supplied with no configuration required.

The term "no configuration" is to emphasize that the Boot Channel is one of the earliest channels that cannot rely on others for configuration. In reality, a couple of straps or GPIO pins are viable but we choose a strict definition to avoid any chicken-and-egg issue.

[REQ-BOOT-9] The Boot Channel shall start with a shared acyclic topology to simplify inter-chiplet deadlock avoidance.

[REQ-BOOT-10] The Boot Channel shall enable communications between a self-boot chiplet and its non-self-boot chiplets.

4.4. System Management

System management includes system state management, power management, performance management, reset management, clock management, hazard management, etc.

The OCA architecture uses a hierarchical system management structure, as shown in Figure 5(a) and Figure 6.

[REQ-SM-1] Each chiplet shall implement a System Management Controller (SMC) to handle chiplet level management tasks. SMC can be a physically standalone component or can be a service that shares physical resources with other services.

[REQ-SM-2] SMC in the System Management Primary chiplet (Primary SMC) shall also handle top-level management tasks.

[REQ-SM-3] Management tasks spanning multiple chiplets, like voltage control of shared voltage rails, shall be managed by the relevant SMC uppermost in the applicable part of the chiplet topology.

[REQ-SM-4] SMCs shall use the System Management Channel to communicate with each other.

The hierarchical management scheme prevents a system software agent, like an OS, from directly interacting with system management resources. It also eliminates the need for one chiplet to manage system resources within another chiplet.

This hierarchical approach can remain transparent to system software, presenting a flat view similar to that of a monolithic SoC. The System Management Primary chiplet serves as the central interface for system software, handling all communication and delegating tasks to downstream SMCs and local system management resources.

4.4.1. Interface between System Management Software and Primary SMC

Messaging protocol standards can be used as a layer of abstraction between the system management software and the Primary SMC. For example, the RISC-V architecture defines RISC-V Platform Management Interface (RPMI) while the ARM architecture defines System Control and Management Interface (SCMI).

[REQ-SM-5] The chiplet running the system management software shall be designated as the System Management Primary chiplet.

This ensures the interface between the system management software and the primary SMC to be an intra-chiplet interface, so that secondary SMCs can be CPU architecture agnostic.

4.4.2. Inter-SMC Communication

SMCs communicate to achieve the following goals:

- Distribute system software commands to downstream SMCs.
- Forward responses from downstream SMCs to the upstream SMC in reply to system software commands.
- Transmit telemetry data and state change notifications from downstream SMC to the upstream SMC.
- Send telemetry configurations, control loop budgets, hazard control commands, and other management instructions to downstream SMCs.

To ensure interoperability of system management firmware, a standardized firmware communication interface is essential. The architecture defines the Open Chiplet Management Interface (OCMI) to address this need. The definition of OCMI can be found in Section 6.1.

4.4.3. System Management Channel

The System Management Channel is an architectural channel designed to facilitate inter-SMC communication and support SiP configuration.

The System Management Channel shall have the following properties:

[REQ-SM-6] Each chiplet shall have the System Management Channel always available for inter-chiplet management communications.

4.5. System Time Management

Compared with monolithic SoCs, SiPs are more likely to have distributed system timers in different chiplets. A mechanism is needed to synchronize the distributed system timers.

System timers can be used by software. It can also be used by debug to generate timestamps. In comparison, software typically has higher requirements on the synchronization accuracy, demanding that software sees no time reversal when reading from different system timers.

More details on synchronization protocol are available in Section 5.2.5.

4.5.1. System Time Counter

[REQ-TM-1] Each system timer shall be equipped with a System Time Counter, which shall function as a 64-bit up counter.

[REQ-TM-2] The System Time Counter in the System Time Management Primary chiplet shall run at frequency SYS_TIMER_FREQ, which is profile-dependent. The counting step shall be 1.

[REQ-TM-3] The System Time Counter in a System Time Management Secondary chiplet shall run at frequency SYS_TIMER_FREQ divided by an integer. The counting step shall be the frequency divider.

The frequency divider may be 1 (i.e., no division), while it is common to have a frequency divider greater than 1 on System Time Secondary chiplets.

4.5.2. System Time Management Channel

[REQ-TM-4] System Time Counters shall use System Time Management Channel to get synchronized.

4.6. Security

The OCA architecture does not mandate specific security protections for chiplets, as they depend on the security assets and threat models, which vary by implementation. Instead, the architecture defines interoperability for general security protections, including security domain alignment (Section 4.6.1) and secure lifecycle coordination (Section 4.6.2). Certain OCA architecture requirements and recommendations also carry security implications, as discussed in Section 4.6.5.

4.6.1. Security Domains

Each chiplet may define its own security domains and associated security policies. These domains fall into two categories:

- **Local Security Domains** - Isolated security domains that operate independently from other security domains, including those in other chiplets, and therefore out of OCA's scope.
- **Shared Security Domains** - Cross-chiplet security domains that require explicit interoperability definitions:
  - **Shared Static Security Domains**: Security domains that are defined during hardware design or configured and fixed at boot time. These domains maintain persistent security boundaries throughout system operation. An example is a dedicated shared security domain among a group of standalone security processors. Protecting assets within a Shared Static Security Domain requires attaching a Static Security Identifier (SSID) to all traffic and enforcing SSID-based traffic filtering at the destination.
  - **Shared Dynamic Security Domains**: Security domains that exist within the system for limited periods of time and can be created or destroyed at runtime. An example is the security domain assigned to a virtual machine that runs on both the CPU and device. Protecting assets within Dynamic Security Domains by default relies on page table based address translation at the source. In cases where address translation is not available at the source of a traffic transaction, a Dynamic Security Identifier (DSID) is attached to the traffic and later used by the address translation logic like a security-aware IOMMU.

[REQ-SEC-1] Each chiplet shall clearly classify each of its security domains as either a Local Security Domain, a Shared Static Security Domain, or a Shared Dynamic Security Domain.

[REQ-SEC-2] If a chiplet needs to access remote assets within a Static Security Domain shared by another chiplet over certain communication Channels, the egress transaction requests on those Channels shall carry a SSID_WIDTH-bit Static Security Identifier (SSID). SSID_WIDTH is profile-dependent.

[REQ-SEC-3] If a chiplet contains assets within a Static Security Domain shared with other chiplets over certain communication Channels, those Channels shall carry an SSID_WIDTH-bit Static Security Identifier (SSID) in the ingress transaction requests. SSID_WIDTH is profile-dependent. A configurable SSID remapping mechanism shall be implemented at the D2D entry point of these Channels to flexibly map an arbitrary remote SSID to an arbitrary local SSID.

This allows alignment of security domains between chiplets while still allowing an individual chiplet to have its own security domain definition and encoding.

[REQ-SEC-4] If a chiplet needs to access remote assets within a Dynamic Security Domain shared by another chiplet over certain communication Channels, and the page table walker is embedded in the remote chiplet, the egress transaction requests on those Channels shall carry a DSID_WIDTH-bit Dynamic Security Identifier (DSID). DSID_WIDTH is profile-dependent.

[REQ-SEC-5] If a chiplet contains assets within a Dynamic Security Domain shared with other chiplets over certain communication Channels, and the page table walker is embedded locally, those Channels shall carry a DSID_WIDTH-bit Dynamic Security Identifier (DSID) in the ingress transaction requests over the D2D connection. DSID_WIDTH is profile-dependent.

There is no need for DSID remap as IOMMU already has the flexibility to map page tables based on the DSID(s).

4.6.2. Secure Lifecycle Control

A chiplet can have security assets that belong to the chiplet's designers only, but not to the SiP integrator or other chiplets' designers. This usually demands that chiplets implement their own secure lifecycle control. This distributed secure lifecycle control inevitably brings misalignment among different chiplets. The following are some examples:

- **Misaligned lifecycle states** - The secure lifecycle state typically governs debug and test enablement, as well as cryptographic key derivation. Misaligned lifecycle states can lead to inconsistent debug/test access and divergent key derivation policies. For instance, a chiplet in production state may derive production keys, while other chiplets are still in the development state, allowing debug traffic to enter the production chiplet through D2D connections and leak production keys.
- **Misaligned lifecycle control schemes** - Different chiplets in a SiP can have their own secure lifecycle control schemes. Some have finely grained lifecycle states, some have coarsely grained lifecycle states. Some may not have secure lifecycle control, making them dependent on others' secure lifecycle schemes.

Secure lifecycle control for the Local Security Domains within a chiplet is implementation-defined as local security domains don't require interoperability.

A SiP can have multiple chiplets with external debug and/or test access. These chiplets can route the debug/test traffic to their local functions or to other chiplets through the Test Channel (Section 4.9).

[OPT-SEC-1] SiP integrators should only expose a chiplet to external debug/test access when the chiplet has sufficient secure lifecycle control protection.

[REQ-SEC-6] If a chiplet with external debug/test access forwards debug/test traffic to other chiplets, it shall provide access control protections for both its local functions and other chiplets. The access control protection for latter shall allow its lifecycle control policy to be controlled by the SiP integrator only instead of by the chiplet designer.

When multiple chiplets join a Shared Security Domain, each chiplet can enable debug/test for the Shared Security Domain.

[REQ-SEC-7] A chiplet that joins Shared Security Domains shall allow the lifecycle control policy for the Shared Security Domains to be controlled by the SiP integrator only instead of by the chiplet designers.

As a result, chiplet designers need to avoid embedding chiplet-unique secrets in Shared Security Domains.

4.6.3. Secure Key Management

Each security domain typically includes cryptographic keys accessible only to itself or to higher-privilege domains. These keys are usually derived based on the domain's lifecycle state.

[OPT-SEC-2] Secret keys and key derivation functions for a Shared Security Domain shall be controlled by the SiP integrator only instead of by the chiplet designers.

4.6.4. Security Channel

OCA does not standardize a separate communication solution dedicated to Shared Security Domains. Instead, the Security Channel is integrated into other channels, such as the Coherent Channel, IO-Coherent Channel, Non-Coherent Channel, and Boot Channel, with SSID and DSID tags attached to the traffic to distinguish between security domains.

4.6.5. OCA Security Implications

The OCA security architecture currently does not mandate security protection for the inter-chiplet interconnect (Chapter 5). This implies that by default the trust boundary is the SiP boundary. If a threat model includes decapping of the SiP package, eavesdropping, or intercepting the inter-chiplet connections, additional security protections are needed. For example, encryption is needed to counteract eavesdropping, in which case both sides need to use the same algorithm, crypto mode, and key. These additional protections are not precluded by the current architecture and each chiplet and SiP owner is encouraged to evaluate their threat models and implement appropriate security protections.

The architecture does not mandate secure boot for every chiplet, but recommends that secure boot requirements be determined based on the system's threat model. If secure boot is required for the entire SiP (Section 4.3), this can be achieved by enforcing secure boot on the primary boot chiplet. Chiplets lacking local secure boot support may rely on the primary chiplet to authenticate their boot configuration and code. By default, this places the trust boundary at the SiP level. However, if the interconnect between the primary boot chiplet and secondary chiplets is not trustworthy, secure boot must be implemented on each chiplet individually.

The OCA architecture defines how security domains from different chiplets can be aligned (Section 4.6.1). This also implies that inter-chiplet interconnect is trustworthy, so are security domain IDs that travel over the interconnect. If the inter-chiplet interconnect is not trustworthy, alignment on security domains will need to rely on additional protections, like domain specific cryptographic solutions to protect confidentiality and integrity.

Some of these solutions come with interoperability challenges as well. If needed, they will be covered by future versions of OCA architecture.

4.7. Interrupts

The default mechanism in the OCA architecture for transferring inter-chiplet interrupts is the Message Signaled Interrupt (MSI). Some specific purposes may require wire-based interrupts, like FuSa and RAS. These domain-specific wire-based interrupts are addressed in their respective sections.

Each OCA MSI message shall be a memory write with a DWORD (32-bit) data payload to an interrupt file in the interrupt recipient through the Interrupt Channel. The memory write path goes across chiplet boundaries. It can also go through chiplets. The address of the memory write is configurable by software, which shall consider the address translation along the path.

The MSI requester is responsible for synchronization between data traffic and MSI messages.

Configuration of the OCA MSI uses the PCIe MSI-X definition as a reference, with modifications to accommodate the chiplet model. Implementation determines the scope of a function, with each function utilizing dedicated tables within its own address space to manage MSI.

The OCA MSI configuration information of various chiplets in a SiP is not collected through enumeration but directly provided as a data structure. It can be constructed after all chiplets are assembled in the SiP.

Certain domains require inter-chiplet interrupts to non-CPU components, such as the security processor and the system management controller. The interrupts are handled differently and are addressed in their respective sections.

4.7.1. MSI Configuration

OCA MSI configuration contains three addressable data structures, including Message Control Register, MSI Table, and Pending Bit Array (PBA).

[OPT-INT-1] Chiplets that are capable of generating MSIs should implement Message Control Register, MSI Table, and PBA as defined in Section 4.7.1.1, Section 4.7.1.2, and Section 4.7.1.3. The base addresses of these data structures should be 4KB-aligned in the interrupt source chiplet's address region.

The base addresses of these data structures are implementation-defined. They are provided as configuration data to the host software. Details can be found in Section 6.2.

4.7.1.1. MSI Message Control Register

Figure 15 shows the Message Control Register. Compared with PCIe MSI-X, the OCA interrupt puts the Message Control Register in the interrupt initiator's address space instead of the configuration space in the interrupt recipient's address space. The OCA interrupt changes the layout of the register.

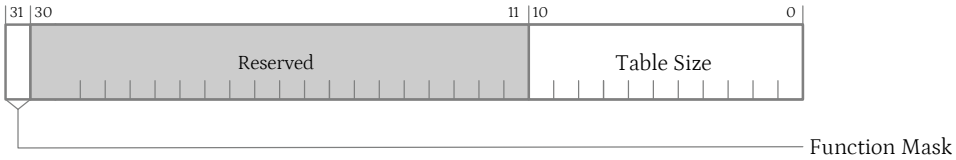

Figure 15. Message Control Register

Bit Location	Register Description	Attributes
10:0	**Table Size** - System software reads this field to determine the MSI Table Size N, encoded as N-1. For example, a returned value of 000 0000 0011b indicates a table size of 4. This information can also be provided as part of the configuration data structure together with the address of the register.	RO
30:11	**Reserved** - Always returns 0 on a read. Write operations have no effect.	RsvdP
31	**Function Mask** - If Set, all vectors associated with the function are masked regardless of their per-vector mask bit values.+ If Clear, each vector's mask bit determines whether the vector is masked or not.+ Setting or Clearing the MSI Function Mask bit has no effect on the value of the per-vector Mask bits. Default value of this bit is 0b.	RW

Table 1. Message Control Register

4.7.1.2. MSI Table

The OCA MSI Table follows PCIe MSI-X Table's definition but simplifies the Vector Control Register such that only the Mask Bit is kept.

The MSI Table structure, Message Address Register, Message Upper Address Register, and Message Data Register follow PCIe MSI-X Table definition exactly. Details can be found in Section 7.7.2 of PCI Express Base Specification Revision 6.1.

Table 2 defines details of the Vector Control Register in each MSI table entry. Unlike the PCIe MSI-X's Vector Control Register, the OCA MSI Vector Control Register only implements the per vector mask bit.

Bit Location	Register Description	Attributes
0	**Mask Bit** - When this bit is Set, the function is prohibited from sending a message using this MSI Table entry. However, any other MSI entries programmed with the same vector is still able to send an equivalent message unless they are also masked. Default value of this bit is 1b (entry is masked). Default value of this bit is 1b (entry is masked)	RW
31:1	**Reserved** - Always returns 0 on a read. Write operations have no effect.	RsvdP

Table 2. Vector Control Register for MSI Table Entries

4.7.1.3. MSI Pending Bits Array (PBA)

The OCA MSI Table follows PCIe MSI-X PBA's definition exactly. Details can be found in Section 7.7.2 of PCI Express Base Specification Revision 6.1.

4.7.2. Interrupt Channel

The Interrupt Channel serves as the architectural mechanism for facilitating inter-chiplet MSI transfers. It can integrate with data movement channels. When posted requests are supported, the interrupt requester can opt to send MSIs as posted writes. In such cases, the Interrupt Channel must ensure compliance with ordering rules relative to data traffic.

4.8. Design for Debug

The OCA debug architecture covers cross triggering, debug timestamp synchronization, and the trace dumping mechanism

4.8.1. Cross Triggering

Cross-triggering is essential for signaling events across a system, including between chiplets in a SiP. Cross-triggering schemes offer flexibility throughout the system hierarchy. They can be configured in a flat structure across the system, or to isolate specific parts of the system — enabling one cross-triggering behavior in an isolated portion, while supporting different behaviors at higher levels. These options allow users to tailor cross-triggering implementations to their specific needs.

[REQ-DBG-1] Each chiplet shall support XTRIG_CNT bidirectional cross triggers. XTRIG_CNT is profile-dependent but shall be greater than or equal to 1.

Each cross trigger represents one event, such as debug signaling between hardware units to coordinate debug across the entire system. Such events can be very specific and are decoded by debug units instantiated throughout the system.

[REQ-DBG-2] Each chiplet shall rely on Cross Trigger Channel to signal events across the SiP.

[REQ-DBG-3] The Cross Trigger Channel shall provide a path for cross triggers to be sent from any arbitrary chiplet to all other chiplets in the same SiP. It shall be physically separated from other Channels to minimize the inter-Channel interference.

4.8.2. Timestamp Synchronization

The resolution of time sync for functional purposes has higher standards than debug timestamp. Debug time stamp synchronization leverages the solution defined in Section 4.5.

4.8.3. Trace Dump

When debug traces are expected to be dumped to local on-chip memory or directly attached memory or storage, the trace-dump solution is implementation-defined.

[REQ-DBG-4] When trace dump involves more than one chiplet, it shall use the Application Channels, Coherent Channel, IO-Coherent Channel, or Non-Coherent Channel, to dump traces to memory or storage.

4.9. Design for Test

Chiplets do not always have their Test Access Ports (TAPs) exposed as a package interface. A more practical and cost-efficient approach is to have chiplets with exposed TAPs to route test traffic to other chiplets via in-package networks. To enable interoperability, the Test Channel is defined in this section.

4.9.1. Serial Test Channel

[REQ-TEST-1] Each chiplet shall support a serial test channel capable of receiving test traffic from either an external tester or an upstream chiplet. Based on routing information in the test traffic, the chiplet shall direct it either to local on-die test and debug functions or forward it to downstream chiplets.

The serial port used to receive test traffic from an external tester or upstream chiplet is referred to as the Serial Primary Test Access Port (SPTAP). The port used to forward test traffic to another chiplet is called the Serial Secondary Test Access Port (SSTAP).

[REQ-TEST-2] Each chiplet shall implement a SPTAP and SSTAP_CNT SSTAPs, where SSTAP_CNT is profile-dependent but shall be at least one.

When each chiplet has only one SSTAP, the resulting test network forms a daisy chain. This configuration minimizes the number of pins required for test ports but results in the greatest network depth.

Alternatively, SSTAP_CNT can match the number of high-speed die-to-die (D2D) ports, forming a tree topology. This reduces network depth but increases the number of pins allocated for test ports. Some of these ports may remain unused, especially when multiple D2D ports connect to the same chiplet.

4.9.2. Parallel Test Channel

Chiplets that support parallel scan run tests in parallel saving test time. The following recommendations ensure interoperability between parallel test solutions.

[REQ-TEST-3] Chiplets that support parallel scan shall support a parallel test channel capable of receiving test traffic from either an external tester or an upstream chiplet. Based on configuration by the serial test channel, the chiplet shall direct parallel test packets either to local on-die test and debug functions or forward them to downstream chiplets.

The parallel port receiving test traffic from an external tester or upstream chiplet is the Parallel Primary Test Access Port (PPTAP). The port forwarding test traffic to other chiplets is the Parallel Secondary Test Access Port (PSTAP).

[REQ-TEST-4] Chiplets that support parallel scan shall implement a PPTAP and PSTAP_CNT PSTAPs, where

PSTAP_CNT is profile-dependent. Parallel scan chiplets shall have at least one PPTAP and PSTAP.

If a chiplet has only one PPTAP and PSTAP, the resulting test network forms a daisy chain. This configuration minimizes the number of pins required for test ports but results in the greatest network depth.

Alternatively, PSTAP_CNT can match the number of high-speed die-to-die (D2D) ports, forming a tree topology. This reduces network depth but increases the number of pins allocated for test ports. Not all ports must be used, for example when multiple D2D ports connect to the same chiplet.

4.10. Functional Safety

In applications like automotive, robotics, and factory automation, failures can lead to catastrophic outcomes. These applications bring additional Functional Safety (FuSa) requirements. This section defines interoperability requirements for FuSa features, as well as the FuSa requirements associated with the logic introduced to ensure interoperability.

OCA employs a hierarchical approach for FuSa management. From an OCA FuSa perspective, chiplets in a package and their connections form a Directed Tree, a subset of the Directed Acyclic Graph as defined in Section 3.2. In this topology, the upstream chiplet receives and processes safety information from the downstream chiplets.

[**REQ-FUSA-1**] Each chiplet intended to support safety applications shall implement a Safety Entity to handle local safety-related tasks. The local Safety Entity aggregates both the internal safety information from its own chiplet and the safety information received from downstream chiplets, then forwards the combined information to the upstream chiplet.

Eventually, the Safety Entity in the FuSa Primary chiplet (Primary Safety Entity) aggregates all safety information in a SiP.

Figure 16 shows an example of the FuSa topology in an automotive SiP. Each chiplet has its own Safety Entity, while the primary Safety Entity sits at the root of the tree. The primary Safety Entity's role is to collect all information regarding the SiP's ability to continue operating fault-free, respond by transitioning the SiP into a safe, or fault detected, state when necessary, and communicate this status externally. When a secondary chiplet encounters an internal fault, like an internal bus timeout, that could impact the system's safe operation, it needs to signal the fault to the primary Safety Entity. The appropriate response to such a fault is often highly system-specific, like resetting the affected chiplet or transitioning the entire system to a safe state, making it the responsibility of the primary Safety Entity to handle the fault in an application-specific manner.

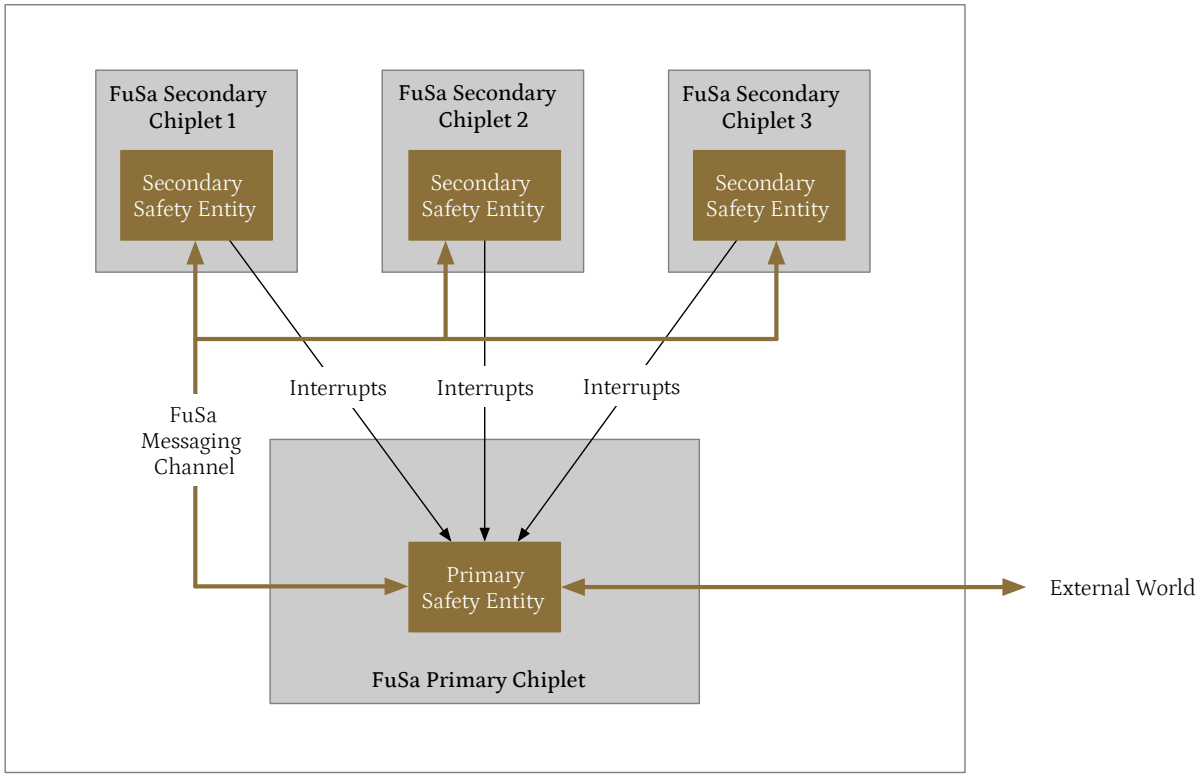

Figure 16. Safety Entity Topology

In some cases, the Primary Safety Entity resides in a Base chiplet, where the essential functions of the SiP are implemented. This Primary Safety Entity may either handle faults internally or forward them to an external safety Microcontroller Unit (MCU). In some other cases, the Primary Safety Entity is located within a dedicated safety MCU inside the SiP, where faults are directly managed and addressed.

FuSa interoperability involves standard interrupts that allow downstream chiplets with FuSa requirements to notify upstream FuSa chiplets of FuSa events (Section 4.10.1). It also includes a standard FuSa Messaging Channel, enabling FuSa chiplets to exchange detailed information about FuSa events or commands (Section 4.10.2).

4.10.1. Interrupts

The OCA FuSa chooses point-to-point interrupts for simplicity and reliability. Each interrupt is a wire that connects to another chiplet. An interrupt signal is asserted high to indicate an active interrupt or low to indicate that there is no interrupt. The physical timing of these interrupt signals is not important, they are level sensitive, not clocked.

[REQ-FUSA-2] Each FuSa chiplet shall source two interrupts and send them to an upstream FuSa chiplet:

- **Fatal Error Interrupt**: The Fatal Error Interrupt is a level-sensitive, active high, point-to-point interrupt that indicates a fatal error has occurred in the chiplet and it can no longer be relied upon to operate safely. If the chiplet autonomously goes offline (thermal shutdown or unexpected reset), this interrupt shall be driven high. This interrupt shall always be driven high during reset to indicate that the function is unavailable.
- **Non-Fatal Error/Status Interrupt**: The Non-Fatal Error/Status Interrupt is a level-sensitive, active high, point-to-point interrupt that indicates that a non-fatal error or a status event has occurred.

Examples of fatal errors include uncorrectable ECC errors or system hangs. When this interrupt is asserted, the safety host should query the signaling chiplet via the FuSa Messaging Channel to determine the cause of the issue.

Examples of non-fatal errors include corrected ECC errors, while examples of status events may include the completion of self-tests. When the non-fatal/status interrupt is asserted, the FuSa primary chiplet should query the signaling chiplet via the FuSa messaging channel to determine the cause of the event. It is expected that the target chiplet remains operational and capable of responding. Non-fatal errors may escalate to fatal errors if not addressed promptly. Although the mechanisms and time constraints for handling such situations are outside the scope of this

specification, it is expected that the safety host will respond with sufficient speed, as defined by the system's safety concept, to address these faults.

4.10.1.1. Interrupt Self Test

The interrupt lines themselves may be subject to faults like becoming open or shorted within the package. Most safety standards require that such faults be detectable within a defined time frame. To support this, it is mandatory that these interrupts be testable via software.

Interrupt self-test allows each wired interrupt to be a single wire, minimizing the impact on in-package routing.

[REQ-FUSA-3] The primary Safety Entity and the secondary Safety Entities shall perform periodic self-tests to ensure that there are no permanent faults, like short and open, in the interrupt wires.

- The primary Safety Entity shall initiate self-tests with downstream secondary Safety Entities via the FuSa Messaging Channel. Upon receiving the self-test interrupt, the primary Safety Entity shall respond via the FuSa Messaging Channel to clear the interrupt.
- The secondary Safety Entities shall support software control to set wired interrupts high and clear them in response to self-test commands from the primary Safety Entity over the FuSa Messaging Channel.

Chiplets should implement measures to ensure that if a real interrupt occurs during a self-test, this information can be communicated to the primary Safety Entity.

4.10.1.2. Interrupt Forwarding

Aggregating and forwarding safety interrupts between chiplets before reaching the primary Safety Entity may be necessary for two reasons:

- The designated FuSa primary chiplet does not have enough interrupt inputs to support all chiplets in the package.
- The physical layout of the SiP makes routing all interrupts directly to the FuSa Primary chiplet difficult.

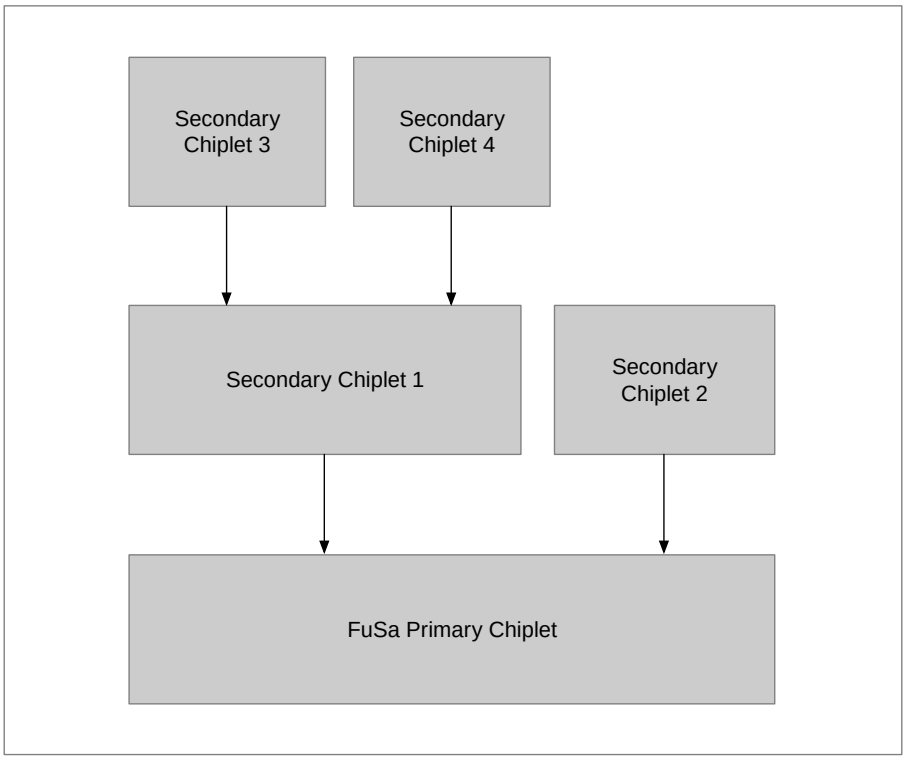

Figure 17. Interrupt Forwarding

Forwarding refers to the simple relay of error interrupts. If a fatal or non-fatal/status input interrupt is detected on a chiplet that is not designated as the primary Safety Entity, the equivalent output interrupt from that chiplet is asserted. This can occur either through direct hardware connection or through software, where the Safety Entity detects the input interrupt and sets its own output interrupt.

When the ultimate destination for error interrupts, the primary Safety Entity, receives an interrupt, it queries the next chiplet in the report chain to determine the nature of the error using the FuSa Messaging Channel. That chiplet will report that the forwarded interrupt was triggered by another chiplet, prompting the primary Safety Entity to move on and eventually query the original source.

For example, in Figure 17, if Secondary Chiplet 3 asserts a Fatal Interrupt, Secondary Chiplet 1 will assert a Fatal Interrupt to the FuSa Primary chiplet. The Primary chiplet will first query Secondary Chiplet 1 to determine the cause of the Fatal Interrupt, and Secondary Chiplet 1 will respond that the source is Secondary Chiplet 3. The FuSa Primary chiplet will then directly query Secondary Chiplet 3. The FuSa view of the SiP topology, including the connection of interrupts, is included in the configuration data structure, as specified in Section 6.2.

[REQ-FUSA-4] Each FuSa chiplet shall be capable of sinking FUSA_FATAL_CNT fatal interrupts. The value of FUSA_FATAL_CNT is profile-dependent but shall be greater than or equal to one.

[REQ-FUSA-5] Each input fatal interrupt shall be configurable via local software and/or messages from the primary safety entity to trigger the output fatal error.

[REQ-FUSA-6] Each OCA profile compliant FuSa chiplet shall be capable of sinking FUSA_NONFATAL_CNT non-fatal interrupts. FUSA_NONFATAL_CNT is profile-dependent but shall be greater than or equal to one.

[REQ-FUSA-7] Each input Non-Fatal Interrupt shall be configurable, via software and/or messages from the Primary Safety Entity, to trigger the output Non-Fatal Error.

The previous requirement ensures that an interrupt from any FuSa chiplets is always sent to the FuSa Primary chiplet, like via a daisy chain. To shorten the path, chiplets can support more input interrupt pairs. For example, chiplets that may act as the FuSa Primary chiplet can support sufficient incoming interrupts to remove stacking. For the same purpose, chiplets intended for stacking other chiplets can implement more than one input interrupt pair.

[REQ-FUSA-8] When a FuSa chiplet aggregates interrupts from its downstream FuSa chiplets, it shall record the aggregation information and provide it to the primary Safety Entity when queried via the FuSa Messaging Channel, allowing the primary Safety Entity to precisely identify the source of an interrupt.

4.10.1.3. Interrupt Response and Clearing

When a secondary Safety Entity has an interrupt to report from its own chiplet, it shall assert the appropriate output interrupt. When a Secondary Safety Entity receives an incoming interrupt, it forwards it toward the FuSa Primary chiplet (Section 4.10.1.2). When the primary Safety Entity receives an input interrupt, it will use the FuSa Messaging Channel to query the source of the interrupt. There may be multiple possible interrupt sources, depending on the number of connected input interrupts, but the system topology should be known in advance. The responding chiplet will either report the reason for the interrupt or identify the next source in the chain if Interrupt Forwarding is being used. It is the responsibility of the primary Safety Entity to signal the secondary Safety Entities when an interrupt may be cleared.

[REQ-FUSA-9] After sending the request to clear the last pending interrupt, the primary Safety Entity shall recheck the interrupt line after a configurable delay. If the fatal or non-fatal interrupt line remains asserted, it indicates that additional faults have occurred.

4.10.2. FuSa Messaging Channel

The FuSa Messaging Channel is a communication pathway that connects various Safety Entities in a SiP.

[REQ-FUSA-10] The FuSa Messaging Channel shall be physically separate from other Channels to minimize inter-Channel interference.

[REQ-FUSA-11] The Safety Entities on different chiplets shall use the FuSa Messaging Channel for inter-chiplet communication.

The following is a summary of the previously described use cases of the FuSa Messaging Channel:

- Self-test initiation
- Interrupt source query and response
- Interrupt cause query and response

4.10.2.1. Heartbeat and Self Test

The FuSa Messaging Channel itself may be subject to faults, like being open or shorted. Most safety standards dictate that faults be detectable within a certain time frame. To support this, each chiplet is required to implement a heartbeat (periodic predefined message) that the primary Safety Entity can use to verify the integrity of the FuSa Messaging Channel. The time interval for these tests is defined by the system. The self-test message and response are specified in [oca_ocsp].

4.11. Reliability Availability and Serviceability

This section outlines the Reliability, Availability, and Serviceability (RAS) requirements defined as a functional domain within the OCA System Architecture. The OCA RAS establishes a framework for supporting RAS features and paradigms, enabling interoperability between hardware and software components.

In multi-vendor SiPs, varying RAS mechanisms across chiplets can create interoperability challenges. Each chiplet may implement fault mitigation and error reporting based on different architectural standards, such as ARM RAS, RISC-V RERI, or Intel MCA. The OCA RAS specifies a common set of behaviors and capabilities to ensure operability and provide essential features for an effective SiP-level RAS solution.

The OCA RAS defines requirements in the following areas:

1. Error Detection
2. Error Response
3. Error Reporting
4. Error Containment

Chiplets are responsible for error detection, response, recording, signaling, and optional containment in hardware or firmware. RAS Agents supplement these capabilities by providing additional error response, containment, and recovery actions not handled by the chiplet's hardware, extending support to the system and end user.

The OCA RAS focuses on the interoperability aspects of chiplet error handling, which is necessary but not sufficient for a complete RAS solution. For example, intra-chiplet actions taken upon receiving a viral error are outside the scope of this document, but remain critical for a robust RAS architecture.

4.11.1. Error Detection

Error detection identifies faults that can manifest as errors. The Error Detection section defines requirements for detecting errors related to the interfaces and communication channels only; it does not provide requirements for detecting errors in the chiplet's internal logic.

[REQ-RAS-1] A communication channel that employs a high-performance and high-bandwidth transport shall detect

two or more random errors in each packet received.

Some example support includes FEC in BoW and CRC16 in UCIe.

[REQ-RAS-2] A communication channel that employs a high-performance and high-bandwidth transport shall support reduced width lanes or redundant lanes (RAS_NUM_REDUNDANT_LANES) to replace failed or failing lanes without re-training or resetting. The RAS_NUM_REDUNDANT_LANES value is profile dependent.

[REQ-RAS-3] All channel protections shall be checked at the channel egress on chiplet interfaces.

[OPT-RAS-1] The channel protection **check** values should be passed through chiplet interfaces with the fields they protect (e.g. address, control, and data fields) as additional fields in the packet. Receiving chiplets ingress interface checks, discards, or passes the **check** values without modification or loss of fidelity.

[OPT-RAS-2] All application channels should have on chiplet interfaces separate protection domains for data and non-data fields across and within a packet.

The recommendation above allows uncorrected errors to assert poison for the data portion of the packet. This contains uncorrected data and increases recovery opportunities.

[OPT-RAS-3] When an application channel on chiplet interfaces does not pass **check** fields, a packet should be checked as close as possible to where the **check** fields are dropped.

This is to minimize the window of vulnerability where uncorrected errors may propagate without detection.

4.11.2. Error Response

Error Response includes a variety of mechanisms to correct errors, replay packets, and recover from errors. The OCA RAS defines requirements for error response in communication channels between chiplets only and does not provide requirements for detecting intra-chiplet errors.

[REQ-RAS-4] A communication channel that employs a high-performance and high-bandwidth transport shall correct one-bit or more detected errors.

The correction may be in-line for Forward Error Correcting (FEC) codes or via packet level replay.

4.11.3. Error Report

The Error Report consists of an Error Record and an Error Signaling. The Error Record is a data structure that contains information about the error event, such as the error category, error source, and other relevant information. The Error Signal is a message that indicates the presence of an error event, the error event's severity and optionally other information deemed relevant to the error event and the hardware response to the error event. The OCA RAS specification defines requirements for both the error record and the error signal. The OCA RAS specification does not define requirements for the error report in the chiplet's internal logic.

4.11.3.1. Error Record

The Error Record's purpose is to convey key information about the error event and possible actions that have already been taken or that should be taken by any agent consuming the error record.

The OCA RAS lists key attributes of a supported error record, but does not specify a specific format and behavior for the error record. Therefore, it is uncertain if error records exist within an OCA RAS compliant SiP. The RAS Agent must be prepared and capable of dealing with this variability.

[REQ-RAS-5] The error categories shall include, at a minimum, `UnCorrected` `UnContained` and `Other`.

[REQ-RAS-6] Error records shall be read and write accessible to RAS Agents initiating access within the chiplet, external to the chiplet, and external to the SiP (e.g. BMC).

[OPT-RAS-4] An `UnCorrected` error category should be derived from the broader `Other` category, and more specific categories such as `UnCorrected Deferred Consumed` and `UnCorrected Deferred Produced` should be additionally supported for finer granularity.

[OPT-RAS-5] A `Corrected` error category should be derived from the broader `Other` category, and more specific categories such as `Corrected Persistent` and `Corrected Scrubbed` should be additionally supported for finer granularity.

[OPT-RAS-6] When support for a finer-grained error category is not available, the broader category should be used.

[OPT-RAS-7] All corrected errors of an application channel that exceed a programmable threshold (RAS_REC_COR_THR) within a programmable time interval (RAS_REC_COR_TIME), an error event should be reported with the `Corrected Persistent` error event, indicating that it may be persistent.

4.11.3.1.1. Error Signaling

To improve scalability of RAS signaling and alleviate package wiring, every chiplet is asked to support aggregating RAS errors from its inter-chiplet ports and export the most severe error category as an error signal group.

This enables various inter-chiplet topologies, such as a daisy chain in Figure 18 and a star topology in Figure 19. This flexibility is very crucial when a SiP has multiple instantiations of a chiplet.

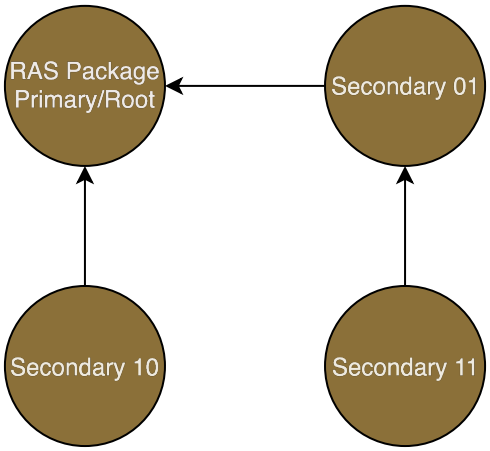

Figure 18. Daisy Chain Signal Example

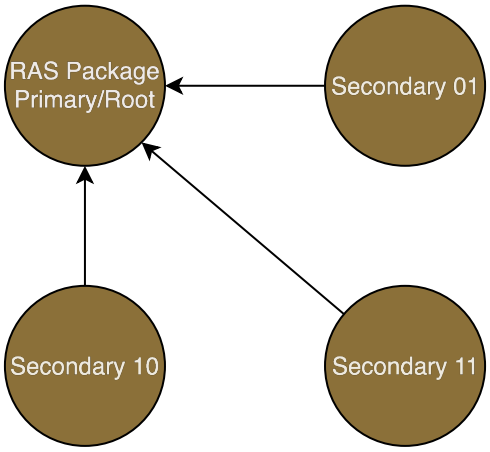

Figure 19. Individual Signal Example

Error signals can be delivered through two complementary mechanisms: direct hardwired point-to-point signaling and message-based communication. Direct hardwired signaling provides immediate and reliable notification of

critical errors to the RAS Agent. For less critical or lower-severity errors, message-based methods, such as virtual wires, offer a scalable way to communicate more detailed information.

[REQ-RAS-7] Every error category shall be signaled via message-based communication. `UnCorrected UnContained` shall be additionally signaled as a hard wire on chiplet interfaces.

[OPT-RAS-8] The `UnCorrected UnContained` signal should have both a high asserted and low asserted complement to provide redundancy and an error detection mechanism.

[OPT-RAS-9] Error signals should correspond directly to the supported error categories.

[REQ-RAS-8] Error signals received from other chiplets and those generated within the chiplet shall be reduced and combined into a single signal group for output from the chiplet.

A signal group is a set of signals corresponding to the supported error categories.

4.11.3.2. Error Containment

Error containment takes many forms, but the OCA RAS defines requirements for data poison and viral containment solutions and their behaviors.

4.11.3.2.1. Data Poison Containment and Deferral

Data Poison is a framework to indicate that a particular address or range of addresses contain corrupted data. Data Poison is essential in deferring uncorrected error event's severity and response from the producer (e.g. cache, memory), through propagators (e.g. fabric) to the consumer (e.g. core, end-point). Severity and response deferral creates opportunities to limit the effect of uncorrected errors and improve overall availability by leveraging information typically only available to the consumer to recover from the error.

The producer records and signals a lower severity error event and "tags" the affected data.

The propagator passes the poison indication without losing it from source to destination.

The consumer records and signals a high severity error while minimizing the affected state, including key recovery details (e.g. context, address, precise) to enable recovery actions such as:

1. Discard the memory and restart the application.
2. Terminate the application while maintaining the OS/VM.
3. Terminate the OS/VM while maintaining the rest of the system.

[REQ-RAS-9] Each application channel shall carry a poison indication on a chiplet interface without loss of fidelity from input to output for all data packets.

Please note that not all available solutions deliver the optimal result of poison. However, the OCA RAS establishes this requirement to guarantee interoperability and encourage future enhancements in chiplet designs.

[REQ-RAS-10] When an application channel detects poison asserted but the channel does not support poison on a chiplet interface, the chiplet's ingress logic shall report an `UnCorrected UnContained` error event.

[OPT-RAS-10] Upon an uncorrectable error in the data protection domain, poison should be asserted for the affected data transfers and an `UnCorrectable Deferred Produced` error should be reported.

4.11.3.2.2. Viral Containment

Viral is a framework to indicate a knife's edge of data that is "bad" – data received before viral observation is "good"

while subsequently received data is considered to be corrupted. Uncorrected errors can be contained via data poison and related mechanisms, other uncorrected errors (e.g. tag errors in a Cache, fabric address/control errors) require a coherent domain wide solution. The Viral containment solution begins at the uncorrected error detection which propagates the Viral indication to interface logic BEFORE the state affected by the error propagates to another interface or to non-volatile storage. When the viral indication is received at a non-volatile storage or potential non-volatile storage interface that does not support the viral indication, all writes are dropped at that interface to prevent the contaminated data from reaching permanent storage.

The producer records and signals a high severity error event and drives viral to its interface coincident or before a corruption reaches that interface.

The propagator passes the viral indication without any input to any output coincident or before a potential corruption reaches an outbound interface.

The consumer is any interface that MAY have non-volatile storages associated with it (e.g. NIC, PCIe device) or cannot pass the Viral indication in a timely manner. The consumer drops all subsequent writes while allowing reads to complete. Viral is typically cleaned with an event/response that re-initializes the interface logic, clears the viral indication, and re-initializes the affected domain's state.

[**REQ-RAS-11**] Each application channel shall carry a viral indication on a chiplet interface without loss of fidelity for all packet types.

All packet types must support viral, as corruption may spread across different transactions and types (e.g., snoop responses), and it is insufficient to drive viral with only some packets since an uncorrected and uncontained error may corrupt both data and non-data transactions.

Similarly to poison, not all available solutions deliver the optimal result of viral. However, the OCA RAS establishes this requirement to guarantee interoperability and encourage future enhancements in chiplet designs.

[**OPT-RAS-11**] When a channel observes a viral indication on its input, the viral indication should be propagated to all output interfaces coincident with or faster than the input channel packet that is viral.

Chapter 5. Protocol and Transport Layers

This section specifies a general architecture for the Protocol and Transport Layers. The OCA architecture leverages existing industry-standard solutions wherever possible. This is especially true at the Protocol and Transport Layers. UCIe and BoW are two broadly discussed high-bandwidth transport mechanisms. There are also standardized low-bandwidth transport mechanisms like I3C and I2C. At the Protocol Layer, mapping PCIe Flit Mode, CXL 256B Flit Mode, and CXL 64B Flit Mode to UCIe transport scheme are natively supported by UCIe. AMBA CHI Chip-to-Chip (CHI-C2C) can also run over UCIe.

The standards mentioned previously do not yet offer a complete solution for chiplet interoperability. For example, the protocols operating over low-bandwidth transport mechanisms are still not well defined. Similarly, there is currently no standardized method for running the widely adopted AMBA AXI protocol over UCIe or BoW. To bridge these gaps and provide a comprehensive interoperability solution, this section defines the missing protocols.

The OCA architecture also remains open to incorporating additional open protocols and transport mechanisms with open specifications as needed.

As a general architectural definition, this section does not constrain the architecture to a single protocol or transport mechanism per channel, which is done in the profile definitions (Chapter 7).

5.1. Mapping of Channels, Protocols, and Transport Mechanisms

Connected chiplets need to map each of the architectural Channels specified in Chapter 4 to the same bus protocol, transport mechanism, and physical interface. For some Channels, the mapping changes depending on availability of physical interfaces.

[REQ-IC-1] The Coherent Channel shall use COH_PRO as the bus protocol and COH_TRN as the transport scheme. COH_PRO and COH_TRN are profile-dependent.

When COH_TRN is unavailable, the Coherent Channel is unavailable.

[REQ-IC-2] The IO-Coherent Channel shall use NCOH_PRO as the bus protocol and NCOH_TRN as the transport scheme. NCOH_PRO and NCOH_TRN are profile-dependent.

When the NCOH_TRN is unavailable, the IO-Coherent Channel is unavailable.

[REQ-IC-3] The Non-Coherent Channel shall use NCOH_PRO as the bus protocol, NCOH_TRN as the transport scheme. NCOH_PRO and NCOH_TRN are profile-dependent.

When the NCOH_TRN is unavailable, the Non-Coherent Channel is unavailable.

[OPT-IC-1] During early boot stages, the Boot Channel should use Open Chiplet Configuration Protocol (OCCP) as its communication protocol.

OCCP defines the protocol for an upstream chiplet, like the Boot Primary, to initiate transactions to the downstream chiplets. It also defines the protocol for a downstream to notify its immediate upstream self-boot chiplet when a boot report is available. Section 5.2.3 describes OCCP in detail.

[REQ-IC-4] During early boot stages, the Boot Channel shall use BOOT_TRN as the transport scheme. BOOT_TRN is profile-dependent.

After the high-speed D2D connections are established, boot software can choose to map the Boot Channel to other

protocols, transport schemes, and physical interfaces. This remapping is implementation-defined.

[OPT-IC-2] When the high-speed D2D connection is unavailable, the System Management Channel should use OCCP (Section 5.2.3) as the protocol.

[REQ-IC-5] When the high-speed D2D connection is unavailable, the System Management Channel shall use SM_TRN as the transport scheme. SM_TRN is profile-dependent.

[REQ-IC-6] When the high-speed D2D connection is available, by default, the System Management Channel shall use the high-speed D2D connection using NCOH_PRO as bus protocol, NCOH_TRN as the transport scheme. Meanwhile, the System Management Channel shall still be able to use the SM_TRN path.

[REQ-IC-7] The System Time Management Channel shall use the Open Chiplet Time Synchronization (OCTS) scheme as the solution to both the protocol and the transport.

Section 5.2.5 describes OCTS in detail.

[OPT-IC-3] When the high-speed D2D connection is unavailable, the Security Channel should use OCCP (Section 5.2.3) as the protocol.

[REQ-IC-8] When the high-speed D2D connection is unavailable, the Security Channel shall use BOOT_TRN as the transport scheme. BOOT_TRN is profile-dependent.

[REQ-IC-9] When the high-speed D2D connection is available, by default, the Security Channel shall use the high-speed D2D connection using NCOH_PRO as bus protocol and NCOH_TRN as the transport scheme. Meanwhile, the Security Channel shall still be able to use the BOOT_TRN path.

[REQ-IC-10] When the Non-Coherent Channel or IO-Coherent Channel exists, the Interrupt Channel shall use NCOH_PRO as the protocol and NCOH_TRN as the transport scheme.

[REQ-IC-11] When only the Coherent Channel exists, the Interrupt Channel shall use COH_PRO as the bus protocol and COH_TRN as the transport scheme.

When neither the NCOH_TRN path nor the COH_TRN path is available, the Interrupt Channel is unavailable.

[REQ-IC-12] The Address Translation Channel shall be paired with the memory access channel (Coherent, IO-Coherent, or Non-Coherent Channels). The corresponding COH_PRO and NCOH_PRO shall support memory access traffic, address translation requests, and page request messages.

[REQ-IC-13] The Cross Trigger Channel shall use the Open Chiplet Cross Trigger (OCTS) scheme as the solution to both the protocol and transport.

[REQ-IC-14] The Serial Test Channel shall use IEEE 1838 as the protocol and JTAG as the transport scheme.

[REQ-IC-15] The Parallel Test Channel shall use Open Chiplet Packetized Scan (OCPS) as the solution to both the protocol and transport.

[REQ-IC-16] The Safety Interrupt Channel shall use two wires to deliver two level based interrupts (fatal and non-fatal).

[OPT-IC-4] The Safety Message Channel should use Open Chiplet Safety Protocol (OCSP) as the protocol.

[oca_ocsp] describes OCSP in detail.

[REQ-IC-17] The Safety Message Channel shall use FUSA_TRN as the transport scheme. FUSA_TRN is profile-dependent.

5.2. Protocols

5.2.1. AXI over UCIe

Please visit openchiplets.org for the OCA AXI over UCIe specification.

5.2.2. CHI-C2C

The OCA architecture adopts AMBA CHI Chip-to-Chip (CHI-C2C) as an option for COH_PRO, the coherent bus protocol.

5.2.3. Open Chiplet Configuration Protocol

The Open Chiplet Configuration Protocol (OCCP) is a standardized protocol for communication between chiplets in a System-in-Package (SiP) during the boot process. It is designed to allow a primary chiplet to configure and boot secondary chiplets in a SiP. It is designed to be simple, extensible, and to provide a common interface for booting secondary chiplets. The OCCP protocol is designed to be used over I3C/I2C, but it can also be used over other transport mechanisms.

5.2.3.1. OCCP Protocol Overview

OCCP expects a single strongly ordered traffic class to be used between the initiating device and the target device. This is used to send requests and receive responses. The OCCP protocol is designed to be simple, extensible, and to provide a common interface for booting secondary chiplets. The OCCP protocol is designed to be used over I3C/I2C, but it can also be used over other transports such as UCIe sideband or shared memory.

All messages have responses, and the responses are sent back to the initiating device in order. The target must be ready to process another message after sending a response, and the initiating device must be ready to receive the response before sending another message.

5.2.3.2. OCCP Header Format

All OCCP messages, request or response, share a common 32-bit header as shown in Table 3.

Bit Location	Field Name	Type	Description
7-0	AppID	uint	**Application ID** - This identifies what the 'application' that the message is a part of. All messages within an application are related. Some of these will be reserved for implementation specific applications.
15-8	MsgID	uint	**Message ID** - This identifies the specific message within the application. A message conveys specific information or requests an action to be taken by the target. Some portion of the message id space is reserved for implementation specific messages. All applications must support MsgID 0x0, which is the GET_VERSION message. This allows the initiating device to discover what applications are supported by the target, and what versions of those applications are supported.

Bit Location	Field Name	Type	Description
20-16	Flags	uint	**Flag bits** • For request messages, bit[20:16] are reserved for future use. • For response messages, bit[16] is used as Error indicator, bit[20:17] are reserved for future use.
31-21	MsgLen	uint	**Message Length** - Length of the message body in bytes, excluding the header. This should be long enough to allow for efficient transport of large amounts of data while being small enough to keep the header size small. The length is in bytes, and is not aligned to any specific boundary.

Table 3. OCCP Request Message Header

5.2.3.3. Responses with Error

The response header has the Error bit to indicate that an error occurred while processing the request. If the Error bit is set, then the response shall only contain a 32-bit error code as data payload. The message length also tells if an error code is present or not. If the Error bit is not set then the response will contain the expected data or be empty if not data is expected in the response.

Bit Location	Field Name	Type	Description
7-0	AppID	uint	Same as the request when applicable
15-8	MsgID	uint	Same as the request when applicable
20-16	Flags	uint	0x1
31-21	MsgLen	uint	4
39-32	MesTyp	uint	**Message Type** - The type of the message, like error, warning, and status. Same definition as the status report. Since this is an error response, encoding is always 0xf.
47-40	Source	uint	**Source** - The encoding is implementation defined.

Bit Location	Field Name	Type	Description
63-48	Error	uint	Error value • 0x1: Invalid AppID. The message contained an AppID that is not recognized or supported by the target. This can happen if the initiating device is trying to use an application that the target does not support. • 0x2: Invalid MsgID. The message contained a MsgID that is not recognized or supported by the target. This can happen if the initiating device is trying to use a message that the target does not support, or if the MsgID is not valid for the given AppID. • 0x3: Invalid header. The message length is not valid or the header is malformed. This can happen if the initiating device sends a message with a length that is not valid for the given AppID and MsgID, or if the header is not formatted correctly. • 0x4: Corrupt header. The header of the message fails the integrity check that is defined in an OCCP transport binding (Section 5.2.4). • 0x5: Corrupt data. The data of the message fails the integrity check that is defined in an OCCP transport binding (Section 5.2.4). • 0x6: Invalid address. The address passed in the request is not valid. • 0x7: Unsupported status ID. The status ID passed in the get_status request is not supported by the target. Some applications may not support returning a status, so requesting status from those applications will return this error. • 0x8: Corrupt transport. The transport mechanism's native integrity check fails. • Others: reserved.

Table 4. Response Message with Error

When a request header is corrupted, the target device will not be able to properly form an AppID and MsgID for the response. In this case, the response message follows definition of Table 4 with AppID and MsgID both being 0xff. This is the only case where the AppID and MsgID are not the same as the request.

5.2.3.4. GET_VERSION (MsgID: 0x0)

The GET_VERSION message requests the version of the OCCP an Application protocol supported by the target. All applications shall implement the GET_VERSION message with MsgID of 0x0, which allows the initiating device to determine the version of the OCCP application protocol that the target supports. This is useful for determining if the target supports the features that the initiating device needs to use, and allows for future extensions of the protocol without breaking existing implementations. Supported Applications can be discovered by sending a GET_VERSION message using the desired AppID, and the response will include the version of that application or an error if that application is not supported by the target. This allows the initiating device to discover what applications are supported by the target, and what versions of those applications are supported.

Bit Location	Field Name	Type	Description
7-0	AppID	uint	Application ID
15-8	MsgID	uint	0x0
20-16	Flags	uint	0x0
31-21	MsgLen	uint	0

Table 5. GET_VERSION Request

The GET_VERSION response without error is defined in Table 6, and follow the 'semantic versioning' format of major.minor.patch.

Bit Location	Field Name	Type	Description
7-0	AppID	uint	Application ID
15-8	MsgID	uint	0x0
20-16	Flags	uint	0x0
31-21	MsgLen	uint	4
39-32	MajVer	uint	Major version
47-40	MinVer	uint	Minor version
63-48	PatVer	uint	Patch version

Table 6. GET_VERSION Response

The GET_VERSION response with error is defined in Table 4.

5.2.3.5. Base Application (AppID: 0x0)

Support of the Base Application is mandatory. It provides the minimal set of functionality required to support the OCCP protocol. This includes the ability to read and write data, as well as to provide status.

The Base Application has the following messages:

- GET_VERSION: Get the version of the Base application supported by the target.
- GET_STATUS: Get the status of the target.
- WRITE_DATA: Write data to the target.
- READ_DATA: Read data from the target.

5.2.3.5.1. GET_VERSION (MsgID: 0x0)

The GET_VERSION messages are defined in Section 5.2.3.4.

5.2.3.5.2. GET_STATUS (MsgID: 0x1)

The GET_STATUS message requests the status of the target.

Bit Location	Field Name	Type	Description
7-0	AppID	uint	0x0
15-8	MsgID	uint	0x1
20-16	Flags	uint	0x0

Bit Location	Field Name	Type	Description
31-21	MsgLen	uint	2
47-32	StatID	uint	**Status ID** • 0x0-0xff: Status IDs for status of each application on the target device. A get_status with an application ID returns the overall status for the application. This may not apply to all applications, and is primarily useful in applications that maintain internal state. Applications that do not support status will return the Unsupported status ID error. Status ID 0x0 for the base application must be implemented by all targets, and is used to report the overall status of the target device. • 0x100-0x7fff: reserved by OCCP • 0x8000-0xffff: implementation specific use.

Table 7. GET_STATUS Request

The GET_STATUS response without error is defined in Table 8.

Bit Location	Field Name	Type	Description
7-0	AppID	uint	0x0
15-8	MsgID	uint	0x1
20-16	Flags	uint	0x0
31-21	MsgLen	uint	4
39-32	MsgTyp	uint	**Message Type** - The type of the message, like error, warning, and status. • 0x1: informational • 0x8: warning • 0xf: error • remaining values in 0x0=0x7f: reserved for OCCP uses • 0x80-0xff: implementation specific uses
47-40	Source	uint	**Source** - The source of the status message. This indicates the source of the status message, such as the application or the part of the SiP that generated the status message. The encoding is implementation defined.
63-48	Status	uint	Status value

Table 8. GET_STATUS Response

The GET_STATUS response with error is defined in Table 4.

5.2.3.5.3. WRITE_DATA (MsgID: 0x2)

The WRITE_DATA message writes data to the target's memory-mapped registers or memories.

Bit Location	Field Name	Type	Description
7-0	AppID	uint	0x0
15-8	MsgID	uint	0x2
20-16	Flags	uint	0x0
31-21	MsgLen	uint	WLen + 12
95-32	Addr	uint	**Write Address** - Address to write to. This is a 64 bit address, and can be used to write to any address space that the target supports. The address space is determined by the 'write attr' field, which allows for different address spaces to be used.
106-96	WLen	uint	**Write Length** - The length is in bytes, and is not aligned to any specific boundary. The length must be less than or equal to the maximum message length supported by the target. WLen being 0 is considered as a corrupt-data error, as defined in Table 4.
111-107	AddrAttr	uint	**Address Attributes** - Address attributes can be used to define different address spaces in the target (like configuration and MMIO). They can also be used to indicate the attribute of the transaction (like posted and non-posted). Standard definition to be added when needed. • bit[109-107]: Reserved • bit[111-110]: Implementation defined
127-112	Reserved	-	Reserved
127+WLen*8-128	WData	int	**Write Data** - Data payload to write

Table 9. WRITE_DATA Request

The WRITE_DATA response without error is defined in Table 10.

Bit Location	Field Name	Type	Description
7-0	AppID	uint	0x0
15-8	MsgID	uint	0x2
20-16	Flags	uint	0x0
31-21	MsgLen	uint	0

Table 10. WRITE_DATA Response

The WRITE_DATA response with error is defined in Table 4.

5.2.3.5.4. READ_DATA (MsgID: 0x3)

The READ_DATA message reads data from the target's memory-mapped registers or memories.

Bit Location	Field Name	Type	Description
7-0	AppID	uint	0x0
15-8	MsgID	uint	0x3
20-16	Flags	uint	0x0

Bit Location	Field Name	Type	Description
31-21	MsgLen	uint	10
95-32	Addr	uint	**Read Address** - Address to read from. This is a 64 bit address, and can be used to write to any address space that the target supports. The address space is determined by the 'write attr' field, which allows for different address spaces to be used.
106-96	RLen	uint	**Read Length** - The length is in bytes, and is not aligned to any specific boundary. The length must be less than or equal to the maximum message length supported by the target. RLen being 0 is considered as a corrupt-data error, as defined in Table 4.
111-107	AddrAttr	uint	**Address Attributes** - Address attributes can be used to define different address spaces in the target (like configuration and MMIO). They can also be used to indicate the attribute of the transaction (like posted and non-posted). Standard definition to be added when needed. • bit[109-107]: Reserved • bit[111-110]: Implementation defined

Table 11. READ_DATA Request

The READ_DATA response without error is defined in Table 12.

Bit Location	Field Name	Type	Description
7-0	AppID	uint	0x0
15-8	MsgID	uint	0x3
20-16	Flags	uint	0x0
31-21	MsgLen	uint	Message Length
(MsgLen*8+31)-32	RData	int	**Read Data** - Data payload for read

Table 12. READ_DATA Response

The READ_DATA response with error is defined in Table 4.

5.2.3.6. Boot application (AppID: 0x1)

The Boot Application is used to boot the target device. It is used along with the Base Application to provide a minimal set of functionality required to boot the target device. The Base functionality is used to write the boot image to the target device. The Boot Application is used to boot the target device once the boot image has been written. Since target devices can vary widely in how firmware is started and authenticated, the Boot Application provides abstracted functionality to boot the target device. This allows the initiating device to boot the target device without needing to know the specifics of how the target device boots, and allows for a common boot process across different target devices.

The Base Application has the following messages:

- GET_VERSION: Get the version of the Boot application supported by the target.
- EXECUTE_IMAGE: Notify the target to execute the loaded software image.
- AUTHENTICATE_IMAGE: Write data to the target.

- TRAIN_D2D: Read data from the target.

5.2.3.6.1. GET_VERSION (MsgID: 0x0)

The GET_VERSION messages are defined in Section 5.2.3.4.

5.2.3.6.2. EXECUTE_IMAGE (MsgID: 0x1)

This message tells the target to directly execute the loaded image, like setting CPU PC to image entry point. This is used to boot the target device after the boot image has been loaded. This boot method is the simplest and supports non-secure booting of the target device. Upon receiving this message, the target device will execute the image that has been written to it at the address specified in the message. Whether to support this message depends on target's choice.

Bit Location	Field Name	Type	Description
7-0	AppID	uint	0x1
15-8	MsgID	uint	0x1
20-16	Flags	uint	0x0
31-21	MsgLen	uint	10
95-32	StartAddr	uint	**Start Address** - Start address of the loaded software image. The target shall derive the execution entry point into the software image accordingly.
103-96	CPUID	uint	**CPU ID** - This identifies which CPU to execute the image. It's up to the target to map different ID values to different CPUs.
106-104	Reserved	-	Reserved
111-107	AddrAttr	uint	**Address Attributes** - Start Address's attributes can be used to define different address spaces in the target (like configuration and MMIO). They can also be used to indicate the attribute of the transaction (like posted and non-posted). Standard definition to be added when needed. • bit[109-107]: Reserved • bit[111-110]: Implementation defined

Table 13. EXECUTE_IMAGE Request

The EXECUTE_IMAGE response without error is defined in Table 14.

Bit Location	Field Name	Type	Description
7-0	AppID	uint	0x1
15-8	MsgID	uint	0x1
20-16	Flags	uint	0x0
31-21	MsgLen	uint	0

Table 14. EXECUTE_IMAGE Response

The EXECUTE_IMAGE response with error is defined in Table 4.

The status of the execution can be queried using the GET_STATUS message in the Base Application. This allows the initiating device to determine if the image was successfully executed or if there was an error during execution. If there

was an error, the GET_STATUS message will provide more information about the error.

5.2.3.6.3. AUTHENTICATE_IMAGE (MsgID: 0x2)

The AUTHENTICATE_IMAGE request message asks the target to verify the software image's authenticity and integrity and execute it when the check passes. If the authentication fails, the target device will not execute the boot image and will provide an error status via the GET_STATUS message in the Base Application. The authentication process is specific to the target device and may involve checking a signature, hash, or other method of verifying the integrity of the boot image.

Bit Location	Field Name	Type	Description
7-0	AppID	uint	0x1
15-8	MsgID	uint	0x2
20-16	Flags	uint	0x0
31-21	MsgLen	uint	10
95-32	StartAddr	uint	**Start Address** - Start address of the loaded software image. The target shall derive the execution entry point into the software image accordingly.
103-96	CPUID	uint	**CPU ID** - This identifies which CPU to authenticate and execute the image. It's up to the target to map different ID values to different CPUs.
106-104	Reserved	-	Reserved
111-107	AddrAttr	uint	**Address Attributes** - Start Address's attributes can be used to define different address spaces in the target (like configuration and MMIO). They can also be used to indicate the attribute of the transaction (like posted and non-posted). Standard definition to be added when needed. • bit[109-107]: Reserved • bit[111-110]: Implementation defined

Table 15. AUTHENTICATE_IMAGE Request

The AUTHENTICATE_IMAGE response without error is defined in Table 16.

Bit Location	Field Name	Type	Description
7-0	AppID	uint	0x1
15-8	MsgID	uint	0x2
20-16	Flags	uint	0x0
31-21	MsgLen	uint	0

Table 16. AUTHENTICATE_IMAGE Response

The AUTHENTICATE_IMAGE response with error is defined in Table 4.

The status of the authentication and execution can be queried using the GET_STATUS message in the Base Application. This allows the initiating device to determine if the image was successfully executed or if there was an error during execution. If there was an error, the GET_STATUS message will provide more information about the error.

5.2.3.6.4. TRAIN_D2D (MsgID: 0x3)

The TRAIN_D2D request message asks the target to start training the high-bandwidth D2D connections, when training is needed. This request is issued to both D2D sides after both are loaded with the necessary training software.

Bit Location	Field Name	Type	Description
7-0	AppID	uint	0x1
15-8	MsgID	uint	0x3
20-16	Flags	uint	0x0
31-21	MsgLen	uint	1
39-32	D2DID	uint	**D2D ID** - This identifies which D2D connection to train. • 0x0 - 0xFE: ID of individual D2D connections • 0xFF: All D2D connections

Table 17. TRAIN_D2D Request

The TRAIN_D2D response without error is defined in Table 18.

Bit Location	Field Name	Type	Description
7-0	AppID	uint	0x1
15-8	MsgID	uint	0x3
20-16	Flags	uint	0x0
31-21	MsgLen	uint	0

Table 18. TRAIN_D2D Response

The TRAIN_D2D response with error is defined in Table 4.

The status of the D2D training can be queried using the GET_STATUS message in the Base Application. This allows the initiating device to determine if the training was successful or if there was an error during the training. If there was an error, the GET_STATUS message will provide more information about the error.

5.2.4. OCCP I3C/I2C Binding

This section describes how the OCCP protocol is bound to the I3C/I2C transport. The following discussion focuses on I3C, but it is also applicable to I2C.

5.2.4.1. I3C Addressing

OCCP messages are sent between devices using the device I3C addresses. Each device on the I3C bus is assigned a unique address, which is used to identify the destination of messages. The OCCP protocol uses the I3C addressing scheme to route messages between the initiating device and the target device. The SiP integrator assigns unique I3C addresses to different chiplets and records them in a data structure that's shared with software.

5.2.4.2. I3C Message Format

OCCP messages are encapsulated within I3C messages by adding an I3C wrapper. The I3C message format adds an I3C destination address, header CRC, flags, and body CRC with OCCP message inserted between flags and the body CRC. The OCCP header has a separate CRC from the OCCP message body. This allows the header to be validated before the body is processed, which is important for error handling in unreliable transports.

The OCCP I3C wrapper, OCCP header, and OCCP body are all little-endian structures in memory. These are transported as an ordered byte stream across the I3C bus. Bit ordering within bytes follows the i2c/i3c standard.

The I3C messages are defined in Table 19, Table 20, and Table 21 depending on the size of the OCCP messages.

Bit Location	Field Name	Type	Description
7-0	HCRC	uint	**Header CRC** - 8-bit CRC (polynomial: 0xd3) over the OCCP message header
15-8	Flag	uint	**I3C Flags** • bit[8] - BODY-CRC-Present flag to indicate that OCCP message body, if exists, has dedicated CRC • bit[15-9] - Reserved
31-16	Reserved	-	Reserved
63-32	MsgHeader	uint	OCCP Message Header

Table 19. I3C Binding Packet for Messages with MsgLen being 0

Bit Location	Field Name	Type	Description
7-0	HCRC	uint	**Header CRC** - 8-bit CRC (polynomial: 0xd3) over the OCCP message header
15-8	Flag	uint	**I3C Flags** • bit[8] - BODY-CRC-Present flag to indicate that OCCP message body, if exists, has dedicated CRC • bit[15-9] - Reserved
31-16	Reserved	-	Reserved
63-32	MsgHeader	uint	OCCP Message Header
(MsgLen*8+63)-64	MsgBody	uint	OCCP Message Body
(MsgLen*8+71)-(MsgLen*8+64)	BCRC	uint	8-bit CRC (polynomial: 0xd3) over the OCCP message body

Table 20. I3C Binding Packet for Messages with MsgLen larger than 0 but smaller than 14

Bit Location	Field Name	Type	Description
7-0	HCRC	uint	**Header CRC** - 8-bit CRC over 32-bit OCCP message header (polynomial: 0xd3)
15-8	Flag	uint	**I3C Flags** • bit[8] - BODY-CRC-Present flag to indicate that OCCP message body, if exists, has dedicated CRC • bit[15-9] - Reserved
31-16	Reserved	-	Reserved
63-32	MsgHeader	uint	OCCP Message Header
(MsgLen*8+63)-64	MsgBody	uint	OCCP Message Body
(MsgLen*8+95)-(MsgLen*8+64)	BCRC	uint	32-bit CRC (polynomial: 0x992c1a4c) over the OCCP message body

Table 21. I3C Binding Packet for Messages with MsgLen larger 13

OCCP messages are generally either small enough to be protected by an 8 bit CRC, or large enough to require a 32 bit CRC, as is the case for large writes used for loading firmware images.

5.2.4.3. I3C Message Response

For I3C transport the requestor gets the response to a request by doing a 'read' transaction on the I3C bus. The I3C target provides the response message in this read transaction.

In cases where transport error is detected during the reception of an I3C message, the OCCP response message carries an error code being 0x6.

If the target is not ready to provide the response immediately, it shall return all-0 data back.

5.2.4.4. I3C Message Ordering

Between a pair of I3C controller and I3C Target, there shall be at most only one outstanding OCCP request. A new OCCP request shall not be issued until the OCCP response of the previous request has been read by the controller.

5.2.5. Open Chiplet Time Synchronization (OCTS)

OCTS defines a scheme to synchronize all system timers in the same SiP. OCTS covers the protocol and transport scheme.

5.2.5.1. System Timer Preset

[REQ-IC-18] A chiplet with a System Timer shall implement a 64-bit System Timer Preset register.

[REQ-IC-19] The software responsible for System Timer synchronization shall use the System Management Channel to distribute the preset value to all System Timers that are not synchronized to the Primary System Timer.

5.2.5.2. System Timer Credit

[REQ-IC-20] A chiplet with a System Timer shall implement a System Timer Credit register that specifies the maximum number of steps the counter can increase within a given Synchronization Time Window (Section 5.2.5.4).

The System Timer Credit value is determined by the ratio of the primary System Timer clock frequency to TMR_CNT_CREDIT's frequency (Section 5.2.5.4).

5.2.5.3. TMR_SYNC_LOAD

[REQ-IC-21] A System Time Management Primary Chiplet shall generate a global signal (TMR_SYNC_LOAD). When the primary System Timer Counter matches its System Timer Preset value, TMR_SYNC_LOAD shall be asserted high for a minimum duration of 5 ns.

[REQ-IC-22] A System Time Management Secondary Chiplet shall take in TMR_SYNC_LOAD as an input. On the rising edge of the TMR_SYNC_LOAD signal (Section 5.2.5.3), if a secondary System Timer is not yet in sync with the primary, the secondary System Timer Counter shall synchronize to the System Timer Preset value. Once the secondary System Timer has received the preset value, it synchronizes with the primary. Afterwards, it will ignore TMR_SYNC_LOAD until it gets out of synchronization due to events like a reset.

5.2.5.4. TMR_CNT_CREDIT

[REQ-IC-23] A System Time Management Primary Chiplet shall generate a global signal, TMR_CNT_CREDIT, by dividing the primary System Timer clock by a factor like 10 or 20.

The interval between consecutive rising edges of TMR_CNT_CREDIT defines the Synchronization Time Window.

[REQ-IC-24] A System Time Management Secondary Chiplet shall take in TMR_CNT_CREDIT as an input. Once all credits defined by Section 5.2.5.2 are exhausted, the secondary System Timer Counter shall halt for the remainder of the Synchronization Time Window. If any credits remain at the end of the Synchronization Time Window, the secondary System Timer Counter shall advance to a value that fully utilizes the remaining credits.

5.2.5.5. Software Requirements

[REQ-IC-25] When a System Time Management Secondary Chiplet needs to sync with the System Time Management Primary Chiplet after a reset, for example, the software shall select a counter value ahead of the current primary System Timer Counter value. The chosen value shall allow sufficient margin for the software to program it into the System Timer Preset registers of both the primary and secondary System Timers before the primary System Timer Counter reaches the selected counter value.

5.2.6. Open Chiplet Cross Triggering (OCCT)

OCCT provides a scheme to transfer cross triggers from an arbitrary chiplet to all other chiplets in the same package. It also provides an interface to connect with package-external agents. OCCT covers the protocol and transport scheme. For added flexibility, this logic is provisioned to operate in either a point-to-point (P2P) mode or a low-cost wire-OR mode. The OCA OCCT allows the SiP integrator to choose which mode to employ at the package-level since OCA architecture compliant chiplets can support either mode.

The mode that a cross trigger operates in can be determined at the package level. This cross trigger hardware, as well as the corresponding pads, gets programmed differently, depending on the mode.

[REQ-IC-26] Each cross trigger shall have XTRIG_PORT_CNT bidirectional cross trigger ports. In the P2P mode, upon receiving a cross trigger, a chiplet shall forward it to all outgoing ports of the same cross trigger, as well as any internal consumer. In the wire-OR mode, only a single pin is employed at the package level.

In the P2P mode, a minimum XTRIG_PORT_CNT of two ensures that all chiplets can be connected through at least a single cross-triggering daisy chain. Increasing the number of cross trigger ports reduces the number of hops to reach the entire cross-triggering network, thereby also reducing cross-triggering latency. In the wire-OR mode, there is only a single, shared trace for each cross trigger and any chiplet can assert the cross trigger.

Forwarding a cross trigger to a chiplet's internal logic is implementation-defined.

[REQ-IC-27] A single bidirectional cross trigger port connecting two chiplets shall consist of the following four signals:

Name	Function - P2P Mode	Function - Wire-OR Mode
CT_Req_out	Outbound CT request	Bidirectional Wire-OR
CT_Ack_in	Inbound CT acknowledge	tie-off
CT_Req_in	Inbound CT request	tie-off
CT_Ack_out	Outbound CT acknowledge	N/C

Table 22. Cross Triggering Modes

The Point-to-Point Mode: The Req signal initiates a cross trigger event from the source chiplet. It is asserted when a trigger occurs and remains asserted until the corresponding Ack signal is received. The Ack signal provides handshake feedback to the source, indicating that the Req has been received and the source may safely de-assert Req. The Ack signal is asserted when the Req signal is asserted and de-asserted when the Req signal is de-asserted. The timing of the Req and Ack signal transitions depends on propagation delay, clock frequencies of the source and destination, and the number of synchronizer stages in the cross trigger logic.

The Wire-OR Mode: Only the CT_Req_out pads are used for each cross trigger, with all CT_Req_out pads from participating chiplets connected together at the package level. This configuration allows multiple chiplets to share a common wire-OR signal, forming a parallel drop-bus. Network performance may decrease as the number of connected chiplets increases, and is also affected by the physical topology (such as daisy-chain versus star) and the overall size of the network. The wire-OR signal can also be routed to a package-level pad for external (board-level) control or observation.

The handshake scheme in the P2P mode guarantees that a single cycle pulse in the source clock domain will cause a single cycle pulse in the destination domain. A benefit of this circuit is that it functions regardless of the clock frequency relationship between the source and destination clock domains.

In wire-OR mode, there is no handshake mechanism to ensure all consumers reliably receive a cross trigger event. To mitigate this, a stretch circuit extends a single-cycle pulse into a programmable-width assertion. The stretch duration should be set according to the source and slowest destination clock frequencies, as well as the propagation delay of the cross trigger signals.

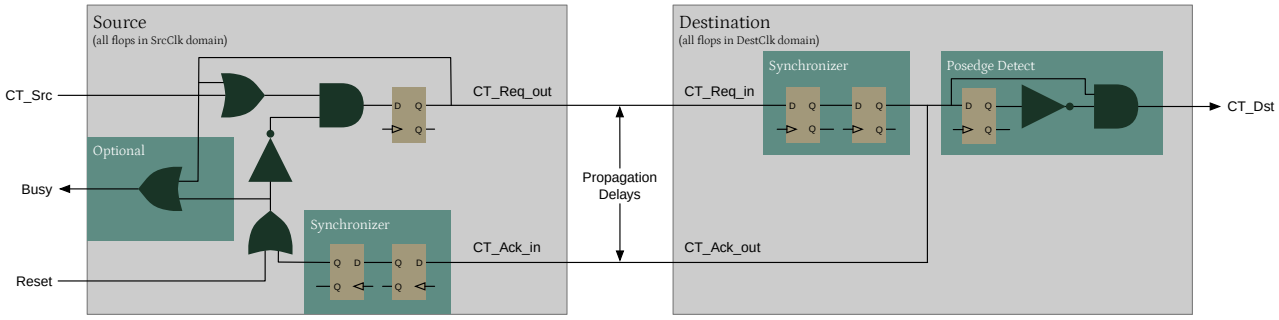

Figure 20. Point-to-Point Cross Triggering Circuit

Figure 20 shows an example implementation of the point-to-point cross triggering functionality in the OCA architecture. The minimum period between cross trigger events requires two loops around the path. One loop for the Req/Ack to assert and a second loop for the Req/Ack to de-assert. Optionally, it is possible to identify hazard cases where a second cross trigger assertion in the source domain occurs before a prior cross trigger is complete, while the Busy signal is still true.

Figure 21 shows the interface signal timing diagram while Table 23 summarizes each stage of the handshake.

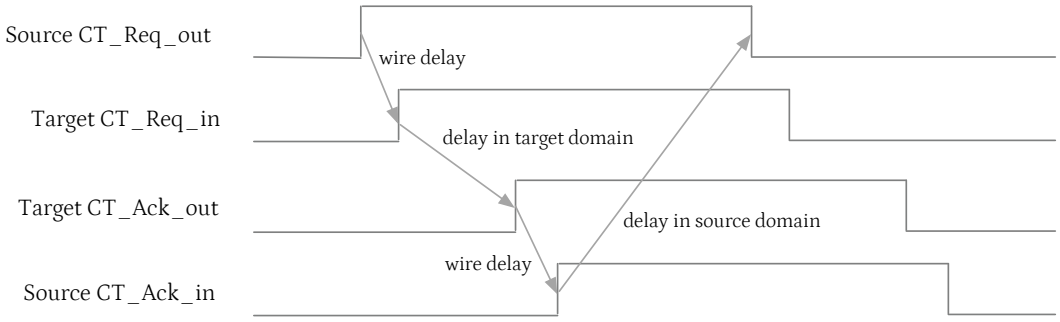

Figure 21. Point-to-Point Cross Triggering Timing Diagram

Source CT_Req_out	Target CT_Req_in	Target CT_Ack_out	Source CT_Ack_in	Comments
0	0	0	0	Quiescent state
1	0	0	0	Cross trigger request sent, not received at destination yet.
1	1	0	0	Cross trigger request sent, received at destination input.
1	1	1	0	Cross trigger request received at destination input and acknowledge sent back, but not received at source input yet.

Source CT_Req_out	Target CT_Req_in	Target CT_Ack_out	Source CT_Ack_in	Comments
1	1	1	1	Cross trigger acknowledge received at source input.
0	1	1	1	Cross trigger event complete in source, with request de-asserted, but de-assertion not received by destination yet.
0	0	1	1	Cross trigger event complete in source, with request de-asserted, received at destination.
0	0	0	1	Cross trigger event complete in source and destination, with acknowledge de-asserted at destination, but de-assertion not received by source yet.
0	0	0	0	Cross trigger transmission complete and system is back in quiescent state again.

Table 23. Point-to-Point Cross Triggering Handshake

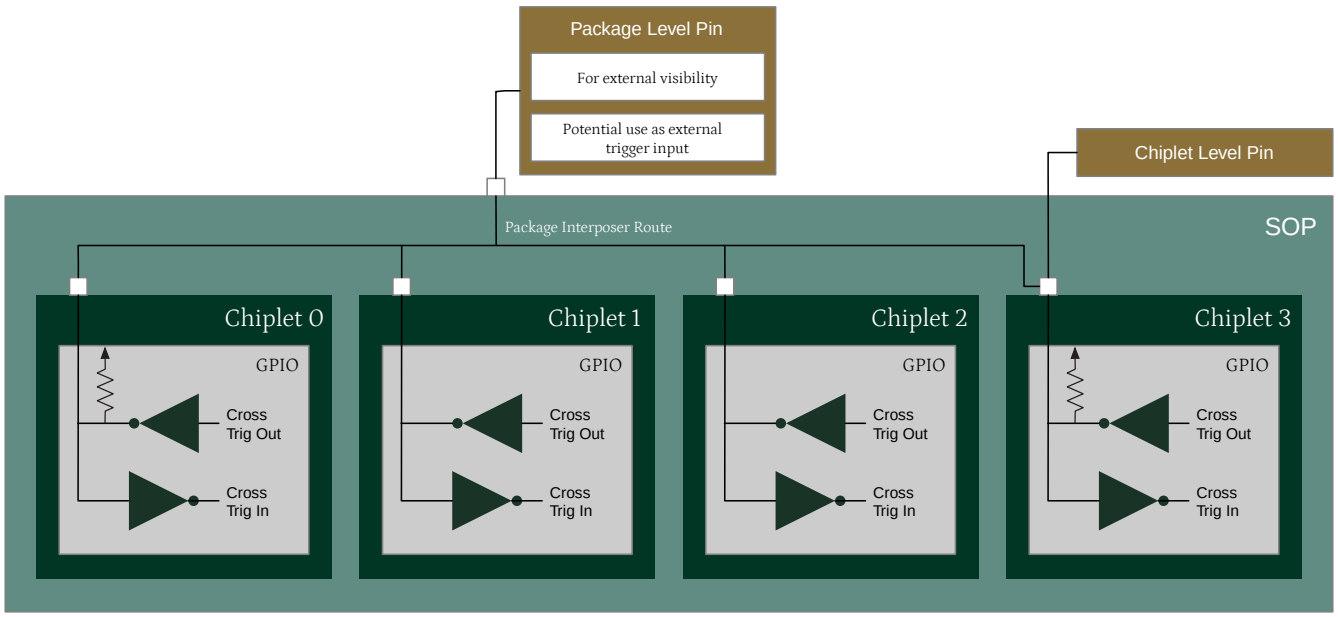

Figure 22. Wire-OR Cross Triggering

Figure 22 shows the wire-OR scheme with an example 4 chiplet package. The package-level wire is negative-true, but the signaling is positive-true internally. The wire is connected to all participating chiplets and can also be tied to a package-level pin for external control/observe. The pad types are GPIO. The pull-ups shown could be within the package or could be realized within the pads themselves by programming, depending on the pad capabilities. The wire-OR function is achieved by having the signal being pulled up, where any permutation of chiplets could be asserting by driving low.

[REQ-IC-28] The mode bit shall reset to wire-OR mode to ensure there are no drive-fights for instances that are connected for point-to-point mode at the package level.

5.2.6.1. Integration Examples

The simplest way of connecting multiple chiplets in a package in point-to-point mode is to have two separate cross trigger ports on each chiplet and daisy-chaining all chiplets together in a single chain. With two cross trigger ports per chiplet, for a single cross trigger, this would require eight bumps, four bumps per port. This would have a minimal bump cost, but latency would not be optimal. Latency would vary depending on where in the chain any given chiplet is and would increase with more chiplets.

By adding additional ports on chiplets, it is possible to reduce latency by implementing fan-out connectivity in

parallel. Figure 23 shows how a subset of chiplets could have additional ports to achieve reduced latency.

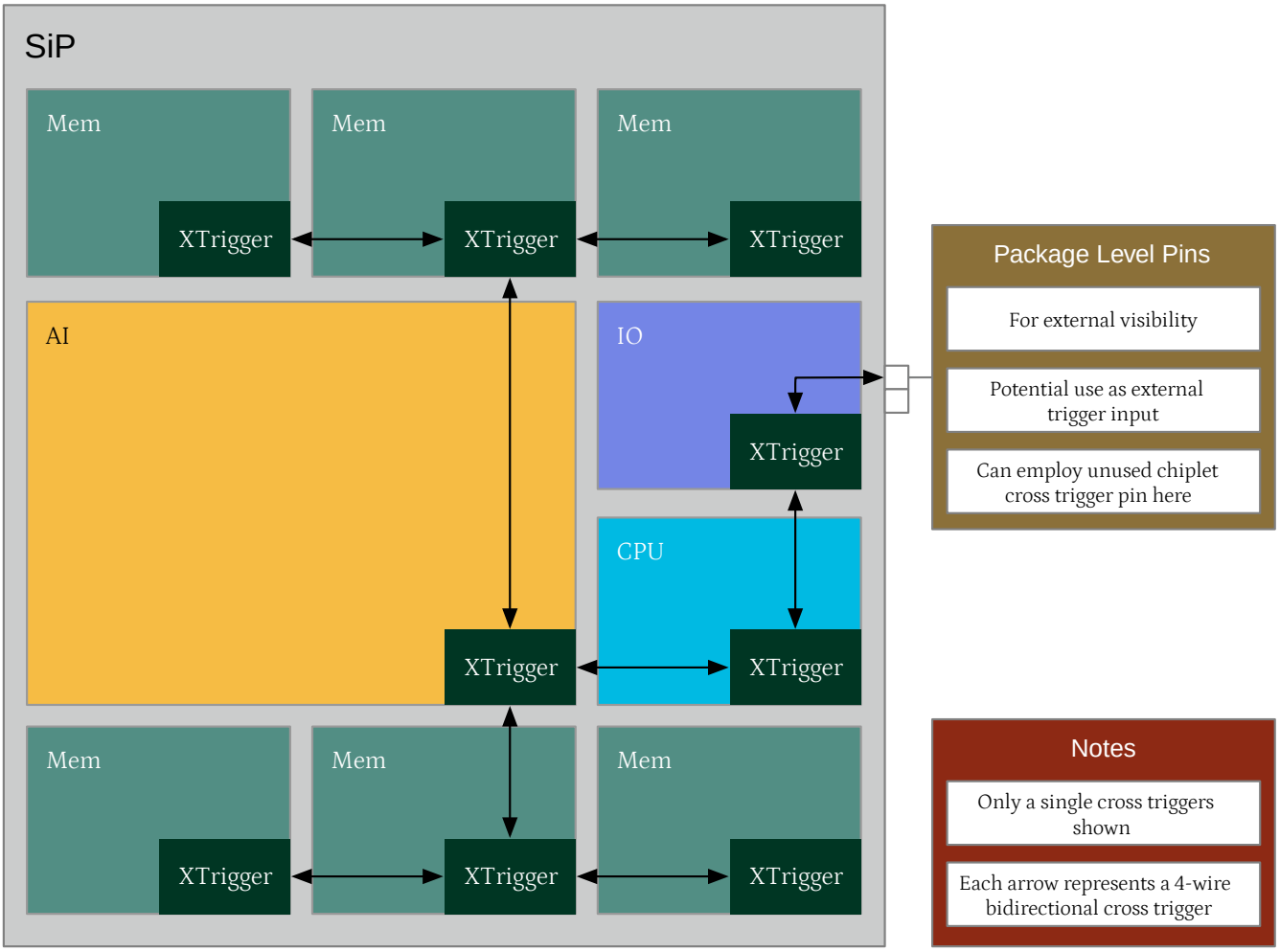

Figure 23. Point-to-Point Multiple Chiplet Connectivity

Internal to a chiplet, multiple destinations can be connected in parallel. To connect multiple destinations in parallel, feedback Ack signals can be ANDed together, where the slowest destination clock domain would determine when the final aggregate Ack signal is de-asserted. Care must be taken in cases where a participant can be power-gated. In these cases, the Ack should be properly negated with power state.

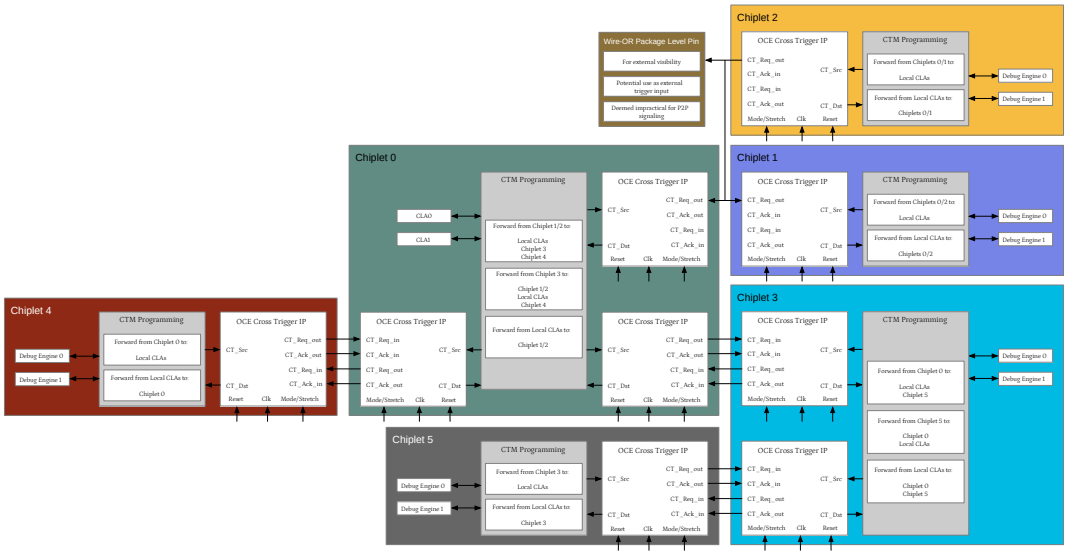

Figure 24. Multiple Chiplet Connectivity With Point-to-Point and Wire-OR Chiplets

Figure 24 shows a package composed of five chiplets, where three of them are connected in a wire-OR and three connections are point-to-point, with chiplet 0 acting as a bridge.

5.2.7. Open Chiplet Packetized Scan (OCPS)

It is usually not possible to pinout the parallel scan ports from every chiplet of a SiP. OCPS defines a solution which uses a parallel scan network to connect all chiplets in a SiP, possibly from different vendors. OCPS covers a standardized parallel scan interface and a protocol for performing SiP-level scan testing of chiplets.

OCPS meets the following requirement:

- A standardized inter-chiplet interface for parallel scan.
- Constant and deterministic flit cycle latency from originating scan input to originating scan output.
- Sufficient scalability to cover all chiplets in a SiP.

5.2.7.1. Inter-Chiplet Packetized Scan Interface

The inter-chiplet packetized scan interface used to receive test traffic from upstream chiplet is referred to as the Parallel Primary Test Access Port (PPTAP). The port used to forward test traffic to another chiplet is called the Parallel Secondary Test Access Port (PSTAP).

[REQ-IC-29] The inter-chiplet packetized scan interface shall be a parallel OCPS Interface (OCPS_INTF[65:0]) with OCPS_INTF[65:0] = {OCPS_CLK_UP, OCPS_DATA_UP[31:0], OCPS_CLK_DW, OCPS_DATA_DW[31:0]}. OCPS_CLK_DW and OCPS_DATA_DW[31:0] are the clock and data bus sent downward from PSTAP to PPTAP. OCPS_CLK_UP and OCPS_DATA_UP[31:0] are the clock and data bus returned upward from PPTAP to PSTAP.

Table 24 summarizes the OCPS Bus.

Index	Field Name	Description
31:0	OCPS_DATA_DW[31:0]	Data bus sent downward from PSTAP to PPTAP
32	OCPS_CLK_DW	Clock sent downward from PSTAP to PPTAP
64:33	OCPS_DATA_UP[31:0]	Data bus returned upward from PPTAP to PSTAP
65	OCPS_CLK_UP	Clock returned upward from PPTAP to PSTAP

Table 24. OCPS Bus

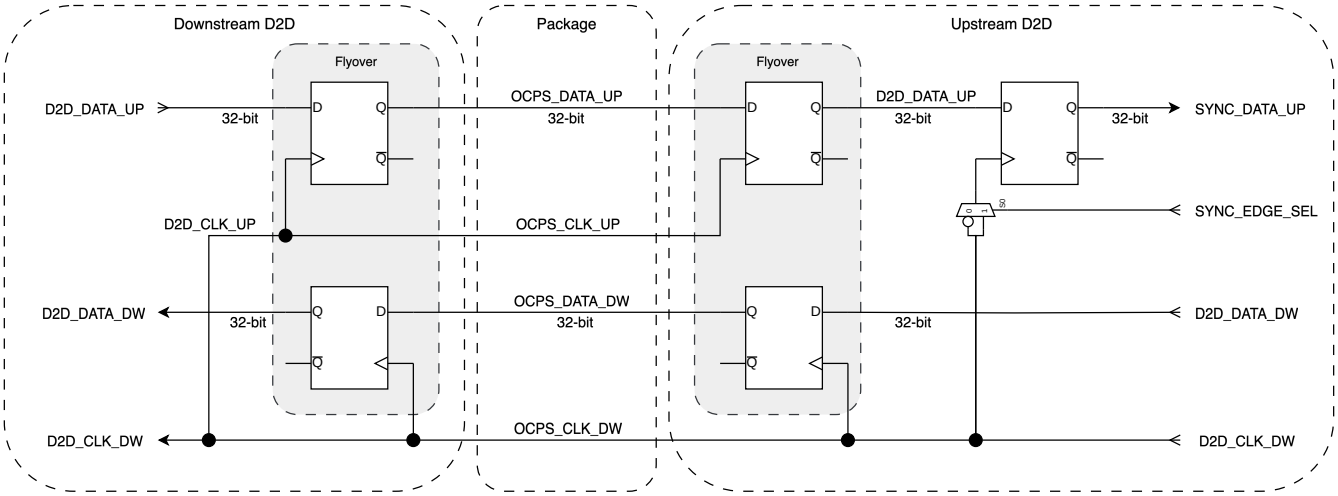

Figure 25. Flyover interconnection of two chiplets.

Figure 25 illustrates an OCPS-compliant D2D flyover interconnection between two chiplets.

[REQ-IC-30] The OCPS Bus shall use the flyover mechanism of the high-bandwidth D2D interconnect as the transport mechanism to connect two chiplets.

5.2.7.2. SiP Packetized Scan Interface

An SiP's packetized scan interface is the parallel test port that is exposed at the package level for the external tester to communicate with the SiP.

[REQ-IC-31] An SiP that supports parallel scan shall integrate at least one chiplet with SiP packetized scan interface(s).

The exact transport mechanism of the SiP packetized scan interface is implementation defined. It is usually different than the flyover of D2D as D2D interfaces are often not exposed at the package level. Instead, the SiP packetized scan interface can be implemented with GPIOs or other high-bandwidth interfaces, like the high-bandwidth external memory interface. It is the chiplet designer's decision if a chiplet supports SiP packetized scan interfaces.

If a chiplet supports SiP packetized scan interface, the security requirements defined in Section 4.6.2 apply.

5.2.7.3. Test Packet Routing

As test packets enter a chiplet via a test entry point, like a PPTAP or a SiP packetized scan interface, the OCPS network routes them to either adaptors to scan-test local logic or a PSTAP to scan test a downstream chiplet. Between the test entry point and the adaptors or the PSTAPs, test packets can go through OCPS switches for OCPS network routing. PPTAPs and PSTAPs can also implement the switch function and become OCPS switches.

[REQ-IC-32] The routing configuration of each OCPS switch shall be controlled by a Test Access Register (OCPS_CTRL_TDR) that is accessible by the serial test channel (Section 4.9.1).

[REQ-IC-33] The scan path length between a test entry point, like a PPTAP or a SiP packetized scan interface, and a PSTAP shall be deterministic and constant under a given routing configuration.

5.2.7.4. Scan Data Synchronization

[REQ-IC-34] To ensure predictable and constant scan depth from the tester's perspective, the following shall be satisfied:

- OCPS_CLK_DW and OCPS_CLK_UP shall be treated as mesochronous (identical frequency, unknown phase).
- OCPS_DATA_DW shall be synchronous to OCPS_CLK_DW as it exits the upstream chiplet.
- OCPS_DATA_UP shall be synchronous to OCPS_CLK_UP as it exits the downstream chiplet.
- OCPS_DATA_UP shall be resynchronized to OCPS_CLK_DW after entering the upstream chiplet.
- If the chiplet vendor provides the option, the SiP integrator may select the rising or falling clock edge for OCPS_DATA_UP resynchronization, whichever provides the greatest timing margin.
- The upper and lower bounds of the signal propagation delay from the upstream chiplet's synchronizer flop clock pin for SYNC_DATA_UP to the bump-out for OCPS_CLK_DW shall be provided to the SiP integrator across specified PVT corners.
- The upper and lower bounds of the signal propagation delay from the upstream chiplet's bump-out for OCPS_CLK_UP to the synchronizer flop D-pin for SYNC_DATA_UP shall be provided to the SiP integrator across specified PVT corners.
- The upper and lower bounds of the signal propagation delay between the downstream chiplet's bump-out for OCPS_CLK_DW and OCPS_CLK_UP shall be provided to the SiP integrator across specified PVT corners.

SiP integrators need the upper and lower bounds of the internal signal propagation delay for OCPS_CLK_DW and OCPS_CLK_UP to determine the maximum scan frequency, appropriate synchronization clock edge, and the effective depth of the scan chain.

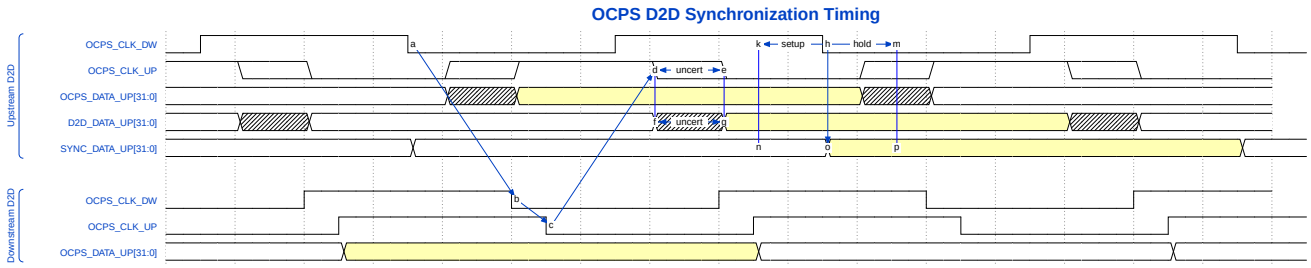

Figure 26. Flyover interface timing and synchronization.

Figure 26 illustrates the timing of OCPS_DATA_UP synchronization across an OCPS-compliant interface. a → b → d shows the round-trip propagation delay of the scan clock across the interface. d → e and f → g show the clock uncertainty window on OCPS_CLK_UP due to jitter and the influence of PVT corners on the round-trip propagation delay. k ← h → m shows the setup and hold window surrounding the selected sampling edge of OCPS_CLK_DW.

The SiP integrator performs timing closure on the OCPS interface using the internal timing information provided by the chiplet vendor(s) and a timing analysis of the flyover channel. OCPS_DATA_DW is launched source-synchronous with OCPS_CLK_DW and OCPS_DATA_UP is launched source-synchronous with OCPS_CLK_UP. The D2D_DATA_UP scan data bus must meet setup and hold timing relative to the synchronizer flop clocked with OCPS_CLK_DW. The SiP integrator must consider the clock uncertainty window on the arrival of D2D_DATA_UP along with the setup and hold window of the synchronizer flop. These two windows must not overlap. If the chiplet vendor allows selection of the clock edge for synchronization, the SiP integrator may adjust this to assist in timing closure.

5.3. Transport Mechanisms

5.3.1. UCIe and BoW

The OCA architecture adopts both UCIe and BoW as transport options for COH_TRN and NCOH_TRN, the transport scheme for the high bandwidth D2D connections.

Both UCIe and BoW are recognized as transport options because both are actively used in the industry. They are supported by the OCA architecture in respective profiles.

5.3.2. I3C

The OCA architecture adopts I3C as a transport option for BOOT_TRN, SM_TRN, FUSA_TRN, the transport scheme for lower-bandwidth D2D connections.

I3C has desirable properties like multi-drop and a limited amount of wires. The In-Band Interrupts (IBI) to allow targets to initiate communication to the controller.

The OCA architecture does not mandate a specific I3C speed and deems it a part of the boot and system management latencies and product specific.

5.3.3. I2C

The OCA architecture adopts I2C with point-to-point wired interrupts as another transport option for BOOT_TRN, SM_TRN, FUSA_TRN, the transport scheme for lower-bandwidth D2D connections.

With back compatibility from I3C, I2C is an accepted protocol for the OCA architecture.

I2C doesn't support IBI. To allow targets initiate communication to the controller, point-to-point wired interrupts are

needed.

5.3.4. Mix and Match of I3C and I2C

Both I3C and I2C are supported by the OCA architecture for their features, compatibility and availability. Therefore, the architecture ensures chiplet designers have flexibility to choose either I2C or I3C based on their IP and foundry constraints, without causing fragmentation in the ecosystem.

While I3C is designed with backward compatibility to I2C, not all features are fully supported in every configuration. The OCA architecture expects implementations to comply with the I2C and I3C specifications. To avoid redundancy and potential discrepancies, this specification does not repeat requirements already defined in those standards. Instead, the following additional requirements and recommendations address OCA-specific considerations:

1. **Addressing**: The OCA architecture uses static addressing and has no usage of dynamic addressing and does not guarantee compatibility when dynamic addressing is used.
2. **Hot Join**: The OCA architecture has no usage of hot join and does not guarantee compatibility when hot-join is used.
3. **Speed**: Speed of the Transport Layer significantly affects boot latency, although the OCA architecture does not mandate specific targets. Chiplet designers and SiP integrators are advised to minimize boot latency by considering options such as using I3C for higher bus speeds, splitting boot images to utilize faster transport mechanisms, and other optimizations.
4. **Signaling**: The OCA architecture expects an I3C controller to support both I3C targets (push-pull signaling) and I2C targets (open-drain signaling) on the same bus, as defined by the I3C specification.
5. **Multi-controller**: The OCA architecture has no usage of multi-controller and does not guarantee compatibility when multi-controller is used.

[REQ-IC-35] A chiplet with an I3C target shall support an I2C target mode or a separate I2C target interface to ensure compatibility in mixed I3C/I2C environments.

The I2C target mode or interface must meet the expectations on an standalone I2C target in accordance with the I2C specification, such as wired interrupts, static addressing, glitch filtering, and clock stretching. The OCA architecture does not extend specific requirements along those lines to avoid redundancy.

[REQ-IC-36] To enable the In-Band Interrupt (IBI) feature and wired interrupts to ensure compatibility in mixed I3C/I2C environments:

1. A chiplet with an I3C controller shall support taking at least one wired interrupt in parallel with IBI.
2. A chiplet with an I2C controller shall support taking at least one wired interrupt.
3. A chiplet with an I2C target shall support generating one wired interrupt.

With the support of one wired interrupt in a controller, a SiP integrator could wire 'OR' all interrupts to the controller at minimum. Alternatively, if a controller provides more interrupt inputs, each wired interrupt could be connected individually for a better resolution.

[REQ-IC-37] A chiplet with an I2C target shall support disabling clock stretching to ensure compatibility in mixed I3C/I2C environments.

[OPT-IC-5] A chiplet with an I2C target should include a glitch filter to improve signal integrity and reduce false triggering.

5.4. Interfaces

Physical communication interfaces and traffic types of a chiplet depend on the chiplet role (primary vs. secondary),

location in the topology (root, leaf, or somewhere in the middle), special application needs (FuSa, RAS, etc.), cache coherency requirements, and bandwidth requirements.

A chiplet can be designed to be more flexible, supporting multiple roles and satisfying different requirements in parallel. Correspondingly, complexity increases. It is up to chiplet designers to find a balance between flexibility and complexity.

The OCA architecture focuses on a basic set of interfaces:

- **Non-Fully-Coherent Interface** - A high-bandwidth interface that can be used for non-coherent or IO-coherent memory access, interrupt delivery, configuration, etc.
- **Coherent Interface** - A high-bandwidth interface for fully coherent memory access.
- **Out-of-Band (OOB) Boot Interface** - A low-bandwidth, highly available interface for Boot and Security. It is called out-of-band relative to the high-bandwidth D2D interfaces.
- **OOB Mgmt Interface** - A low-bandwidth, highly available interface for System Management. It is called out-of-band relative to the high-bandwidth D2D interfaces.
- **Timer Sync Interface** - Dedicated interface for system timer synchronization.
- **Cross Trigger Interface** - Dedicated interface for cross triggering.
- **Serial Test Interface** - Test interface for in-package test network.
- **Wired Interrupts Interface** - Dedicated interface for wired interrupts, serving Functional Safety and System Management.
- **FuSa Message Interface** - Dedicated messaging interface for Functional Safety.

[**REQ-IC-38**] Chiplets shall follow the specifications in Table 25 to implement interfaces and map architectural Channels accordingly.

Certain architectural Channels may be mapped to the same interface, requiring them to share the same protocol and transport scheme. Conversely, some Channels may be mapped to multiple interfaces, mainly because certain interfaces may not always be available. In such cases, these Channels need to switch to alternative interfaces as needed.

Interface	Role	Protocol/ Transport	Channels	Notes
Non-Fully-Coherent	Initiator and Receiver (bi-direction)	NCOH_PRO/ NCOH_TRN	Non-Coherent; IO-Coherent; Interrupt; Address Translation; System Management; Security	High-bandwidth D2D connection; Number of the Non-Fully-Coherent interfaces per chip (NCOH_CNT) is implementation-defined; Isolation of the Security Channel from other channels, if needed, is achieved by cryptography; Can merge with the Coherent interface if NCOH_TRN equals COH_TRN and NCOH_PRO equals COH_PRO.
Coherent	Initiator and Receiver (bi-direction)	COH_PRO/C OH_TRN	Coherent	High-bandwidth D2D connection; Number of the Coherent interfaces per chip(COH_CNT) is implementation-defined.
Out-of-Band (OOB) Boot	Controller for Boot/Security Primary chiplet; Target for Boot/Security Secondary chiplet	OCCP (rec.)/BOOT_ TRN	Boot; Security	One OOB Boot interface per chiplet to support the Boot and Security Channels

Interface	Role	Protocol/ Transport	Channels	Notes
Out-of-Band (OOB) Mgmt	Controller for System Management Primary chiplet; Target for System Management Secondary chiplet	OCCP (rec.)/SM_TRN	System Management	One OOB Management interface per chiplet to support the System Management Channel
Timer Sync	Transmitter for System Time Primary chiplet; Receiver for System Time Secondary chiplet	OCTS	Timer Sync	Needed when chiplet has a System Timer
Cross Trigger	Transmitter and Receiver	OCCT	Cross Trigger	XTRIG_CNT bi-directional cross triggers
Serial Test	Receiver	IEEE 1838	Serial Test	At least 1 SPTAP per chiplet
Serial Test	Initiator	IEEE 1838	Serial Test	SSTAP_CNT SSTAPs per chiplet
Parallel Test	Receiver	OCPS	Parallel Test	At least 1 PPTAP per chiplet
Parallel Test	Initiator	OCPS	Parallel Test	PSTAP_CNT PSTAPs per chiplet
Wired Interrupts	Initiator	Level-sensitive interrupt	FuSa Interrupts	2 wired interrupts (1 fatal + 1 non-fatal) per chiplet for the Safety Interrupt Channel
Wired Interrupts	Receiver	Level-sensitive interrupt	FuSa Interrupts	(FUSA_FATAL_CNT + FUSA_NONFATAL_CNT) wired interrupts per chiplet for the Safety Interrupt Channel, if FuSa is needed
FuSa Message	Controller for FuSa Primary chiplet; Target for FuSa Secondary chiplet	OCSP (rec.)/FUSA_TRN	Safety Message	1 dedicated interface per chiplet for safety message exchange, if FuSa is needed.

Table 25. Chiplet Interfaces

5.5. Traffic Attribute

Some traffic attributes were specified earlier in this specification like Static Security Identifier (SSID) and Dynamic Security Identifier (DSID). This section specifies additional traffic attributes.

Profile definition in Chapter 7 specifies which traffic attributes are adopted.

5.5.1. Quality of Service (QoS)

The OCA architecture supports End-to-End QoS in a multi-chiplet system. It is assumed that each chiplet's NoC has the following attributes:

- Provides means to assign QoS requirements to transactions;
- Designed to correctly meet the QoS requirements indicated in transactions.

[**REQ-IC-39**] The COH_PRO and NCOH_PRO protocols shall support QOS_WIDTH QoS bits in their transaction requests. QOS_WIDTH is profile-dependent.

The management of flow control credits is specific to each protocol.

5.5.2. Performance Observability

The OCA architecture provides the capability to observe end-to-end performance for post-silicon performance debugging.

[**REQ-IC-40**] The COH_PRO and NCOH_PRO protocols shall carry a 1-bit perfID value across D2D connections.

The perfID is generated in the chiplet. The field is set to 0b1 in a traffic flow of interest to indicate that this transaction is of interest. This information is subsequently used by trace infrastructure to decide which transactions to record for performance debugging purposes.

The mechanism to tag values is implementation-defined. It is typically derived based on the source of the transaction which for a CPU may also include an application/virtual machine.

Chapter 6. Software Layer

This chapter defines the interoperability at the software layer. Challenge to software interoperability comes from interaction between software that are designed by different vendors. Like hardware interoperability, software interoperability needs to deal with the inter-chiplet issues. In addition, software interoperability also sees challenges on the same chiplet. This is because chiplet designers and SiP integrators can be different vendors and SiP integrators' software needs to interact with chiplet designers' hardware and software.

6.1. Open Chiplet Management Interface (OCMI)

TBD in later versions of OCA architecture.

6.2. Open Chiplet Configuration Data Structure (OCCDS)

This section defines a set of properties that describe chiplet hardware capabilities and configurations that are related to interoperability. The Open Chiplet Configuration Data Structure (OCCDS) helps the software running on various chiplets to understand the hardware it runs on. The data structure describes each hardware resource from its local chiplet's perspective. Further processing can be done to parse the data structure and construct the view of the whole SiP from each software entity's view.

6.2.1. Common Connection Properties

In general, software needs to understand the overall topology of the SiP and where its own chiplet sits in the topology.

- **Physical topology**: the graph description of the physical connection of the chiplets in a SiP.
- **Logical topology**: for each functional domain, the architecture uses an Acyclic Directed Graph (ADG) topology to describe the interconnection of chiplets in a SiP (Section 3.2). In addition to the ADG, there is a domain primary chiplet, which is the main controller of the functional domain.

6.2.2. Functional Domain Specific Properties

For Addressing, an address domain appears to other chiplets as a set of address segments. Each segment has its own size. For each memory-mapped D2D connection of a chiplet, which address segments are accessible and the sizes of them should be included in the OCCDS.

Some functional domains, such as interrupts (Section 4.7.1), require software on one chiplet to communicate with hardware on another. To enable standard communication between software and hardware, a hardware configuration structure (e.g., a set of registers) is defined. This structure resides within a specific address segment at a defined offset. Accordingly, both the address segment and the address offset of the hardware configuration structure should be included in OCCDS.

For Boot, the concept of self-boot chiplets is introduced to facilitate hierarchical boot requirements. Accordingly, the Boot ADG should identify which chiplets are self-boot chiplets and which downstream chiplets are related to each self-boot chiplets.

For System Management, the management hierarchy can be further divided into sub-domains according to how resources, like power rails, clocks, and resets, are shared among chiplets. The upper-most chiplet of a sub-domain in the System Management ADG is responsible for the management of the shared resources. Therefore, the software needs to understand which chiplets are in each sub-domain and where the upper-most chiplet for each sub-domain sits.

For Security, a chiplet may have security domains shared with other chiplets, while each chiplet can have their own assignment of SSIDs (Section 4.6.1). Software would need to understand the SSIDs assigned to the same shared security domain in different chiplets to configure the SSID remapping.

6.2.3. An Example of OCCDS

The example is about the final SiP view in the format of the OCCDS. Some properties are provided and defined by a chiplet designer, e.g., local address segments; while some others may only be resolved after SiP integration, for example, Domain-specific ADG and Domain primary chiplet.

1. Common Connection Properties
    a. Topology graph to describe the physical connection of D2D interfaces
2. Common Properties for each functional domain:
    a. For each functional domain:
        i. Domain-specific ADG
        ii. Domain primary chiplet
3. Domain specifics:
    a. Communication Channels
        i. N/A
    b. Addressing
        i. For each D2D inbound interface:
            A. Address domains that are accessible by the D2D interface
        ii. For each address domain:
            A. Address segments and sizes presented through the corresponding D2D
    c. Boot
        i. The address segment and the address offset for boot status reports
        ii. Self-boot chiplets in the Boot ADG
    d. System Management
        i. The address segment and the address offsets for the OCMI message buffers and door bells
        ii. Sub-domains in the System Management ADG
    e. System Time Management
        i. The address segments and the address offsets for each system timer (TMR_SYNC_LOAD and TMR_CNT_CREDIT)
    f. Security
        i. Shared security domains across different chiplets with their SSIDs in each chiplet
    g. Interrupts
        i. The address segment and the address offset for MSI configuration
    h. Design for Debug
        i. N/A
    i. Design for Test
        i. N/A
    j. Functional Safety
        i. N/A
    k. Reliability Availability and Serviceability
        i. N/A

6.3. Software Image Format (SIF)

A formatting standard for composing comprehensible software images for distribution, validation and execution, within and around an OCA profile compliant SiP. Chiplet software artifacts cover a wide range of formats, architectures, encodings, compression, and encryption. In addition, chiplet suppliers will each have their own approach to many aspects of hardware and software configuration of their chiplets with no common descriptors. This format seeks to make software artifacts in an OCA profile compliant system recognizable and manageable without a complete understanding of the vendor's software contents.

The format defined below, generically wraps a chiplet vendor or other 3rd party supplied software with a minimal amount of additional metadata. The provided metadata included with the enveloped, vendor specific image makes it, at a minimum, universally parsable by OCA profile compliant systems. In addition, SiP integration specific fields provide configuration support for deployment and management in the context of specific SiP environment. SIF compatible images may be released directly by vendors or created/repackaged by SiP integrators depending on the application needs. Images packaged for a particular SiP integrator may then be read and processed by boot management related chiplets and delivered to downstream chiplets for execution or further processing.

[REQ-SWPKG-1] Any software artifact managed or transmitted by an OCA profile compliant chiplet shall be packaged according to the OCA Software Image Format (SIF). It is the responsibility of the SiP integrator to verify the correctness of any packaged software intended to be used in their SiP environment.

The package format is comprised of three main sections; the HEADER, the VENDOR BUNDLE, and the PACKAGE_HASH. The HEADER is further broken down into a number of subfields. The general formatting and field definitions are given:

Field Name	Size (Bytes)	Context	Header Component	Content	Example
OCA_IDENTIFIER	7	GLOBAL	Yes	Common OCA SW Identifier Value	"OCACHPL"
OCA_SW_PACKAGE_VERSION	1	GLOBAL	Yes	Version of OCA Software Image Format	0x01
TARGET_CHIPLET_ID	8	SIP	Yes	Identifiers for in SiP Distribution	Integration Specific
PACKAGE_VERSION	16	SIP	Yes	Human Readable Version String	Integration Specific
SW_PACKAGE_CONFIGURATION	8	SIP	Yes	SiP Specific Configuration Flags for This Package	Integration Specific
PACKAGE_SIZE	8	GLOBAL	Yes	Size of HEADER + VENDOR BUNDLE	0x48 Bytes + <Size of Vendor Bundle>
VENDOR_BUNDLE	<(2^8)-HEADER_SIZE	CHIPLET	No	Native Vendor Chiplet Software Artifact	Chiplet Vendor Defined

Field Name	Size (Bytes)	Context	Header Component	Content	Example
PACKAGE_HASH	32	GLOBAL	No	Cryptographic Hash of OCA Header Fields + VENDOR BUNDLE	See example below

Table 26. Image Format Description

The CONTEXT of each field is the overall level of comprehension that is expected in order to fully process and act upon the field information. - GLOBAL context means that any OCA profile compliant tool or chiplet implementation can interpret these fields. - SIP context refers to a particular integration of chiplets. These fields are to be interpreted by boot control, administrative, or security related chiplets in a particular implementation in order to drive the configuration and boot process of SiP environment. The content of each field is at the discretion of the SiP integrator. - CHIPLET context is the proprietary or other externally opaque configuration and executable content of any one chiplet. The CHIPLET context includes vendor executable binaries, configuration, and other assets beyond the scope of this packaging standard. Content in this context should only be modified by the chiplet manufacturer or boot software vendor. The data here shall not be modified by the SiP integrator as part of the packaging process.

[REQ-SWPKG-2] Any chiplet or OCA software tool shall be capable of parsing the SIF and processing fields marked as having GLOBAL context. SIP and CHIPLET context fields may be read and reported, but no interpretation should be applied.

Below is an extended description of each field:

6.3.1. OCA Identifier

A fixed 7-byte string of the ASCII character "OCACHPL" intended to clearly mark a collection of bytes as an OCA SIF binary. Any tool or chiplet reading these bytes should recognize it can parse the bytes following this identifier to, at least, the GLOBAL context.

6.3.2. OCA_SW_PKG_VER

A single byte identifier of the version of OCA SIF is used. The current version defined here is 0x01. The content, size, and context of any header field after this one or any field following the vendor bundle may be modified based on this value. This value shall monotonically increase as new revisions are released. SiP integrators should maintain the same version number between all packages when assembling software artifacts for a given environment. Note that the OCA architecture specification may move forward in versioning without this value changing.

6.3.3. TARGET_CHIPLET_ID

An 8-byte field that should be used by SiP integrators to indicate which chiplet in system the package is intended for. The exact scheme for specifying downstream chiplets, D2D links, and other hierarchical details is at the discretion of the SiP integrator. Generic SIF packages may be released without populating this field

TBD - Recommendations to SiP integrators on how to format this field for optimal environment distribution

[REQ-SWPKG-3] The TARGET_CHIPLET_ID field shall be comprised of 8 bytes representing the intended recipient chiplet of the package software image. The SiP integrator is responsible for the identification scheme and interpretation of the scheme for distribution of software at boot time within a given SiP environment.

6.3.4. PACKAGE_VERSION

A 16-byte human readable ASCII string indicating version information for the software this package represents. This version information may apply to both the vendor bundle and the SIF header itself. A SiP integrator may overwrite this string if repackaging an image from another implementation or a non-SIP specific package. The readable string portion of the field is expected to be in the lowest necessary bytes and padded with null bytes. Any OCA SIF compatible tool may be able to report this value in its human readable state, but not interpret the actual version scheme.

TBD - Recommendations to SiP integrators on how to format this field for clear, concise versioning of SIF images

[REQ-SWPKG-4] The PACKAGE_VERSION field shall be comprised of up to 16 ASCII characters representing the version of the image according to the SiP integrator designated scheme. The SiP integrator is responsible for the version scheme and any interpretation of it between images.

Examples:

```
// "0.1.0"
{0x30, 0x2e, 0x31, 0x2e, 0x30, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00}
// "c7fe172f77c03916"
{0x63, 0x37, 0x66, 0x65, 0x31, 0x37, 0x32, 0x66, 0x37, 0x37, 0x63, 0x30, 0x33, 0x39,
0x31, 0x36}
// 123
{0x31, 0x32, 0x33, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00}
```

6.3.5. SW_PACKAGE_CONFIGURATION

An 8-byte field intended for additional SiP specific configuration information. This field is intended to allow SiP integrators to embed additional usage or config flags for a boot controller or administrative chiplet to use in the distribution and boot of downstream or other subordinate chiplets. This field does not replace or supersede configuration in the vendor bundle. Instead it is intended for marking similar software packages that have different usage models such as "primary" vs "recovery" or "development" vs "production". The content and organization of this field is at the discretion of the SiP integrator

TBD - Recommendations to SiP integrators on how to format this field to account for common use cases including primary/recovery, development/production, secured/unsecured.

[OPT-SWPKG-5] SiP Integrator should use the TARGET_CHIPLET_ID, PACKAGE_VERSION SW_PACKAGE_CONFIGURATION in order to determine how to deliver and execute software on downstream chiplets during system boot.

6.3.6. PKG_SIZE

An 8-byte value representing a little endian unsigned 64-bit integer. This integer is the number of bytes in the complete header and vendor bundle. This value should match the number of bytes to be processed when calculating the PACKAGE_HASH.

6.3.7. VENDOR BUNDLE

A collection of bytes that represents the native software artifact intended to run on a target chiplet. This content is intended to be delivered to a downstream chiplet that can act on it and further the boot process of the SiP. The downstream chiplet should process this information while running from a bootROM, a secondary-bootloader, or via direct execution.

The composition of the vendor bundle is entirely at the discretion of the chiplet owner. Contents may include:

- executable code for a chiplets processor architecture
- configuration for hardware or software components
- lifecycle control information
- additional software manifests and embedded software image specific to a vendor chiplet
- runtime assets
- encrypted material
- cryptographic authentication material (hashes, public keys, signatures)

[REQ-SWPKG-6] The SiP integrator shall not modify the content of the vendor bundle as part of the SiF image generation process.

6.3.8. PACKAGE_HASH

A 32-byte cryptographic hash intended for optional integrity checking and attestation operation. The hash of a SIF packages should be verified by an administrative or boot manager chiplet if the system requires strong integrity guarantees on either image being transferred within the SIP, read or written from non-volatile storage or otherwise maintaining integrity at rest. This value is not intended for authenticity checking and is not replacement for code-signing and secure boot techniques in each chiplet. That capability should be handled within the vendor bundle.

In addition, the PACKAGE_HASH may be passed to security related chiplets that may require software hashes for runtime attestation tracking.

An example of calculating the package hash for a given SIF image:

```
// 48 Bytes Header (including the overall size of 2096 (0x0830) bytes)
OCA_IDENTIFIER = uint8_t{0x4F, 0x43, 0x41, 0x43, 0x48, 0x50, 0x4C} //"OCACHPL"
OCA_SW_PACKAGE_VERSION = uint8_t{0x01}  // SIF Version 1
TARGET_CHIPLET_ID = uint8_t{0xFF, 0xEE, 0xDD, 0xCC, 0xBB, 0xAA, 0x99, 0x88}  // SiP
Integrator Defined Placeholder Data
PACKAGE_VERSION = uint8_t{0x30, 0x31, 0x32, 0x33, 0x34, 0x35, 0x36, 0x37, 0x38, 0x39,
0x41, 0x42, 0x43, 0x44, 0x45, 0x45}  // Human readable SiP integrator defined software
string
SW_PACKAGE_CONFIGURATION = uint8_t{0x08, 0x19, 0x2A, 0x3B, 0x4C, 0x5D, 0x6E, 0x7F} //
SiP Integrator Defined Placeholder Data
PACKAGE_SIZE = uint8_t{0x30, 0x08, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00} // Number of
bytes for HEADER + VENDOR_BUNDLE
```

```
HEADER = uint8_t{0x4F, 0x43, 0x41, 0x43, 0x48, 0x50, 0x4C, 0x01,
                 0xFF, 0xEE, 0xDD, 0xCC, 0xBB, 0xAA, 0x99, 0x88,
                 0x30, 0x31, 0x32, 0x33, 0x34, 0x35, 0x36, 0x37,
                 0x38, 0x39, 0x41, 0x42, 0x43, 0x44, 0x45, 0x46,
```

```
                    0x08, 0x19, 0x2A, 0x3B, 0x4C, 0x5D, 0x6E, 0x7F,
                    0x30, 0x08, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00}

// Vendor Bundle that is 2KB of 0x00
VENDOR_BUNDLE = uint8_t{0x00 * 2048 }

// Compute the hash over the HEADER and VENDOR_BUNDLE
PACKAGE_HASH = sha256sum(HEADER+VENDOR_BUNDLE, PACKAGE_SIZE)

print(PACKAGE_HASH)
c65ce7d59aa7c3d3c5a5530222bad91d7d983937efba19c8e2dd02ca410bae7d

// PACKAGE_HASH = uint8_t{0xc6, 0x5c, 0xe7, 0xd5, 0x9a, 0xa7, 0xc3, 0xd3,
//                        0xc5, 0xa5, 0x53, 0x02, 0x22, 0xba, 0xd9, 0x1d,
//                        0x7d, 0x98, 0x39, 0x37, 0xef, 0xba, 0x19, 0xc8,
//                        0xe2, 0xdd, 0x02, 0xca, 0x41, 0x0b, 0xae, 0x7d}
```

[**REQ-SWPKG-7**] For OCA SIF v1, the header and vendor bundle contents must have a 32-byte cryptographic computed using SHA-256 as defined in NIST standard FIPS 180-4

[**REQ-SWPKG-8**] SIF packaged images shall not be modified in the SiP environment once packaged. This includes in-place changes to all HEADER fields, the VENDOR_BUNDLE, and the PACKAGE_HASH. SIF images may only be updated at their designated stored location if replaced in their entirety with a different SIF image with a correctly calculated PACKAGE_HASH.

6.4. Tooling and Software Support

- Package generation and image inspection tooling - TBD
- Reference code for inspecting, verifying, copying, and operating over SIF images at the chiplet level.

Chapter 7. Profiles

An OCA profile is a specific configuration of the OCA architecture that tailors its general specifications to meet particular application needs, performance targets, or market segments.

Chiplets conforming to the same profile are guaranteed to be interoperable.

- A selection of required functional domains like Addressing, Boot, etc.
- A refinement of the general architecture defined in Chapter 4 and Chapter 5 by assigning specific values to variables;

A profile must specify which functional domains are required and which are recommended. For each required domain, all of its requirements become mandatory within the profile, while recommendations are optional unless explicitly made stricter. For recommended domains, their requirements and recommendations are both optional. If a non-required functional domain is adopted, all of its requirements must be implemented, while its recommendations remain optional. If a recommendation is adopted, every aspect of it must be fulfilled.

Different profiles may adopt different variable settings and/or functional domain selections. Chiplets from different profiles are not guaranteed to interoperate. However, if a chiplet needs to support multiple profiles, it can be designed to support a superset, with one profile enabled at a time.

7.1. Automotive Profile

The automotive profile is designed to enable a device for cost-sensitive automotive applications.

7.1.1. System Layer for Automotive Chiplets

Referenced per subsections in Chapter 4, the Automotive profile has the following functional domains:

[REQ-PFAU-1] The following functional domains shall be implemented:

1. Communication Channels
2. Addressing
3. Boot
4. System Management
5. Design for Debug
6. Design for Test
7. Functional Safety

[OPT-PFAU-1] The following functional domains should be implemented:

1. System Time Management
2. Security
3. Interrupts

[REQ-PFAU-2] The profile-dependent parameter SAD_ADDR_WIDTH shall be 64.

[REQ-PFAU-3] The profile-dependent parameter D2D_ADDR_WIDTH shall be 64.

[REQ-PFAU-4] The profile-dependent parameter SYS_TIME_FREQ shall be 1 GHz.

[REQ-PFAU-5] The profile-dependent parameter SSID_WIDTH shall be 3.

[REQ-PFAU-6] The profile-dependent parameter DSID_WIDTH shall be 4.

One of the recommended usage is that 1 bit of SSID can be used as secure world ID of main CPU's Trusted Execution Environment (TEE); and the other 2 bits of SSID are used to indicate privileges of coprocessors (e.g., security processor).

[REQ-PFAU-7] The profile-dependent parameters XTRIG_CNT shall be 1 and XTRIG_PORT_CNT shall be no less than [2, NCOH_CNT + COH_CNT + 1], whichever is larger.

[REQ-PFAU-8] The profile-dependent parameter SSTAP_CNT shall be no less than 1.

[REQ-PFAU-9] The profile-dependent parameter PSTAP_CNT shall be no less than [1, NCOH_CNT + COH_CNT], whichever is larger.

[REQ-PFAU-10] The profile-dependent parameter FUSA_FATAL_CNT shall be no less than [1, NCOH_CNT + COH_CNT], whichever is larger.

[REQ-PFAU-11] The profile-dependent parameter FUSA_NONFATAL_CNT shall be no less than [1, NCOH_CNT + COH_CNT], whichever is larger.

7.1.2. Transport Layer and Protocol Layer for Automotive Chiplets

[REQ-PFAU-12] All requirements in the Transport and the Protocol layers shall be implemented.

[REQ-PFAU-13] The QoS traffic attributes shall be implemented.

[REQ-PFAU-14] The Performance Observability traffic attributes shall be implemented.

[REQ-PFAU-15] The profile-dependent parameters NCOH_TRN shall be UCIe, NCOH_PRO shall be AXI over UCIe (Section 5.2.1) and QOS_WIDTH shall be 2.

[REQ-PFAU-16] The profile-dependent parameter D2D_ROUTING_WIDTH shall be 10 when applied to each AXI ID field in the AXI over UCIe transport and shall be 11 when applied to CHI-C2C Node ID.

[REQ-PFAU-17] The profile-dependent parameters COH_TRN shall be UCIe, COH_PRO shall be CHI_C2C (Section 5.2.2) and QOS_WIDTH shall be 2.

[REQ-PFAU-18] The profile-dependent parameter SAD_ROUTING_WIDTH when applied to CHI-C2C Node ID shall be 11.

AXI over UCIe does not utilize a node ID and its address handling is specified by the relevant addressing requirements.

[REQ-PFAU-19] The profile-dependent parameter BOOT_TRN shall be I3C or I2C.

[REQ-PFAU-20] The profile-dependent parameter SM_TRN shall be I3C or I2C.

7.2. Data Center Profile

This profile is designed to enable creation of chiplet-based SoCs used for data centers, cloud computing, high-performance computing, and other similar computing usages.

7.2.1. System Layer of Data Center Chiplets

Referenced per subsections in Chapter 4, the Data Center profile has the following functional domains:

[REQ-PFDC-1] The following functional domains shall be implemented:

1. Communication Channels
2. Addressing
3. Boot
4. System Management
5. Design for Debug
6. Design for Test
7. Reliability Availability and Serviceability

[OPT-PFDC-1] The following functional domains should be implemented:

1. System Time Management
2. Security
3. Interrupts

[REQ-PFDC-2] The profile-dependent parameter SAD_ADDR_WIDTH shall be 64.

[REQ-PFDC-3] The profile-dependent parameter D2D_ADDR_WIDTH shall be 64.

[REQ-PFDC-4] The profile-dependent parameter SYS_TIME_FREQ shall be 1 GHz.

[REQ-PFDC-5] The profile-dependent parameter SSID_WIDTH shall be 8.

[REQ-PFDC-6] The profile-dependent parameter DSID_WIDTH shall be 16.

One of the recommended usage is that 1 bit of SSID can be used as secure world ID of main CPU's Trusted Execution Environment (TEE); and the other 2 bits of SSID are used to indicate privileges of coprocessors (e.g., security processor).

[REQ-PFDC-7] The profile-dependent parameters XTRIG_CNT shall be 2 and XTRIG_PORT_CNT shall be no less than [2, NCOH_CNT + COH_CNT + 1], whichever larger.

[REQ-PFDC-8] The profile-dependent parameter SSTAP_CNT shall be no less than [1, NCOH_CNT + COH_CNT], whichever larger.

[REQ-PFDC-9] The profile-dependent parameter PSTAP_CNT shall be no less than [1, NCOH_CNT + COH_CNT], whichever larger.

[REQ-PFDC-10] The recommendations of RAS error detection, response, report, and containment shall be implemented.

[REQ-PFDC-11] The profile-dependent parameter RAS_NUM_REDUNDANT_LANES shall be no less than 2.

There shall be at least two redundant lanes in the data center profile but we recommend four.

7.2.2. Transport Layer and Protocol Layer for Data Center Chiplets

[REQ-PFDC-12] All requirements in the Transport and the Protocol layers shall be implemented.

[REQ-PFDC-13] The QoS traffic attributes shall be implemented.

[REQ-PFDC-14] The Performance Observability traffic attributes shall be implemented.

[REQ-PFDC-15] The profile-dependent parameters NCOH_TRN shall be UCIe, NCOH_PRO shall be AXI over UCIe (Section 5.2.1) and QOS_WIDTH shall be 2.

[REQ-PFDC-16] The profile-dependent parameter D2D_ROUTING_WIDTH shall be 10 when applied to each AXI ID field in the AXI over UCIe transport and shall be 11 when applied to CHI-C2C Node ID.

[REQ-PFDC-17] The profile-dependent parameters COH_TRN shall be UCIe, COH_PRO shall be CHI_C2C (Section 5.2.2) and QOS_WIDTH shall be 2.

[REQ-PFDC-18] The profile-dependent parameter SAD_ROUTING_WIDTH when applied to CHI-C2C Node ID shall be 11.

AXI over UCIe does not utilize a node ID and its address handling is specified by the relevant addressing requirements.

[REQ-PFDC-19] The profile-dependent parameter BOOT_TRN shall be I3C or I2C.

[REQ-PFDC-20] The profile-dependent parameter SM_TRN shall be I3C or I2C.

7.3. Data Center Profile Using Bunch of Wires

This profile is designed to enable creation of chiplet-based SoCs used for data centers, cloud computing, high-performance computing, and other similar computing usages. This profile utilizes Bunch of Wires (BOW) as the physical layer alternative to UCIe for high-bandwidth, low-latency inter-chiplet communication in data center environments.

7.3.1. System Layer of Data Center Chiplets

Referenced per subsections in Chapter 4, the Data Center profile has the following functional domains:

[REQ-PFDCB-1] The following functional domains shall be implemented:

1. Communication Channels
2. Addressing
3. Boot
4. System Management
5. Design for Debug
6. Design for Test
7. Reliability Availability and Serviceability

[OPT-PFDCB-2] The following functional domains should be implemented:

1. System Time Management
2. Security
3. Interrupts

[REQ-PFDCB-3] The profile-dependent parameter SAD_ADDR_WIDTH shall be 64.

[REQ-PFDCB-4] The profile-dependent parameter D2D_ADDR_WIDTH shall be 64.

[REQ-PFDCB-5] The profile-dependent parameter SYS_TIME_FREQ shall be 1 GHz.

[REQ-PFDCB-6] The profile-dependent parameter SSID_WIDTH shall be 8.

[REQ-PFDCB-7] The profile-dependent parameter DSID_WIDTH shall be 16.

One of the recommended usage is that 1 bit of SSID can be used as secure world ID of main CPU's Trusted Execution Environment (TEE); and the other 2 bits of SSID are used to indicate privileges of coprocessors (e.g., security processor).

[REQ-PFDCB-8] The profile-dependent parameters XTRIG_CNT shall be 2 and XTRIG_PORT_CNT shall be no less than [2, NCOH_CNT + COH_CNT + 1], whichever larger.

[REQ-PFDCB-9] The profile-dependent parameter SSTAP_CNT shall be no less than [1, NCOH_CNT + COH_CNT], whichever larger.

[REQ-PFDCB-10] The profile-dependent parameter PSTAP_CNT shall be no less than [1, NCOH_CNT + COH_CNT], whichever larger.

[REQ-PFDCB-11] The recommendations of RAS error detection, response, report, and containment shall be implemented.

[REQ-PFDCB-12] The profile-dependent parameter RAS_NUM_REDUNDANT_LANES shall be no less than 2 in advanced packaging.

In standard BOW packaging, redundant lane support is optional and therefore recommended in this profile. In advanced BOW packaging, the inclusion of at least two redundant lanes is mandatory and required by this profile.

7.3.2. Transport Layer and Protocol Layer for Data Center Chiplets

[REQ-PFDCB-13] All requirements in the Transport and the Protocol layers shall be implemented.

Different from UCIe and AXI over UCIe, the BOW and Quasar NoC Protocol (QNP) require some developments in the QoS attributes and Performance Observability attributes. Therefore, those requirements are omitted in the profile.

[REQ-PFDCB-14] The profile-dependent parameters NCOH_TRN shall be BOW, NCOH_PRO shall be both AXI-lite over BOW and QNP over BOW.

Non-coherent transactions can be transmitted over BOW using either AXI-lite or QNP protocols. The selection of protocol depends on the transaction type and the sender.

The Bunch of Wires specification is released separately. The protocol of AXI-lite over BOW is released separately.

[REQ-PFDCB-15] The profile-dependent parameter D2D_ROUTING_WIDTH shall be 10-bit when applied to each AXI ID field in the AXI over BOW; and shall be 6-bit source and 6-bit destination when applied to QNP address.

Although AXI-lite does not define an ID field, it is common practice in AXI NoC implementations to repurpose the ID field for routing purposes.

[REQ-PFDCB-16] The profile-dependent parameters COH_TRN shall be BOW, COH_PRO shall be CHI over BOW.

Please note there are ongoing discussions about what to use for inter-chiplet coherent transport protocol. CHI is the leading option while other alternatives such as CHI over QNP over BOW are also being evaluated.

The protocol of Quasar NoC Protocol is released separately. The protocol of QNP over BOW is released separately.

[REQ-PFDCB-17] The profile-dependent parameter SAD_ROUTING_WIDTH shall be 10-bit when applied to each AXI ID field in the AXI over BOW; and shall be 6-bit source and 6-bit destination when applied to QNP address.

[REQ-PFDCB-18] The profile-dependent parameter BOOT_TRN shall be I3C or I2C.

[REQ-PFDCB-19] The profile-dependent parameter SM_TRN shall be I3C or I2C.

Appendix A: Interim Specification License

Important Notice—Read before copying, reviewing, or using this specification:

This license agreement is a legal agreement between You and Tenstorrent Holdings Inc. and its subsidiaries ("Tenstorrent") and governs Your use of the specification attached to this license ("this Specification"). This Specification comprises the entire base specification plus the entirety of any optional extension which a user chooses to implement. This Specification is a part of the Open Chiplet Atlas™ Architecture.

This Agreement can be accepted only by an adult of legal age of majority in the country in which this Specification is used. If you don't have the required age or authority to accept this Agreement, or if you don't accept all the terms and conditions of this Agreement, do not download or use this Specification.

If you are entering into this Agreement on behalf of a company or other legal entity, you represent that you have the legal authority to bind the entity to this Agreement, in which case "You" and "Your" will mean the entity you represent and its affiliates.

This is a draft specification. Changes are anticipated and likely. Do not rely on or design to this non-final draft. Tenstorrent cannot design an open chiplet ecosystem alone. A robust ecosystem is best built when interested parties such as You participate. Accordingly, Tenstorrent provides this draft to You for purposes of your edification and to invite Feedback. If you wish to submit Feedback about this Specification as allowed by Section 3, please reach out to Tenstorrent generally or feedback@openchipletatlas.org.

1. **Ownership**
   a. Tenstorrent or its licensors hold all rights, title and interest in and to this Specification and its drafts (such as this one), modifications and derivative works, including their respective intellectual property rights, subject to Your license rights under Section 2 ("License"). This Specification may include materials from Tenstorrent's licensors, and these licensors are intended third party beneficiaries that may enforce this Agreement with respect to their intellectual property rights.
   b. This Agreement does not transfer any or license any rights, title and interest in and to works which You have created which relate or conform to this Specification, including any respective intellectual property rights, subject to Tenstorrent's rights under Section 1.a ("Ownership") and Section 4.a ("Feedback").
2. **License**
   a. Tenstorrent grants You a limited, revocable, non-exclusive, non-transferable, royalty free, worldwide copyright license to:
      i. display, distribute, and copy this Specification only within Your organization solely for the purpose of edification and providing Feedback per Section 4.
   b. All of Tenstorrent's licenses to You are subject to the following restrictions:
      i. no right is granted to You to sublicense Your rights under this Agreement; and
      ii. while this Specification is open to any licensees accepting these terms, for the avoidance of doubt, there are no rights to modify or distribute this Specification; and
      iii. neither the whole nor any part of the information contained in this Specification and related collateral may be modified, adapted or reproduced in any material form except with the prior written permission of Tenstorrent; and
      iv. For the avoidance of doubt, this license does not extend to the implementation of other published specifications or third-party technologies even if referred to in this Specification; and
      v. You agree to cooperate with Tenstorrent and provide reasonably requested information to verify Your compliance with this Agreement.
   c. No other license is granted, either expressly or impliedly, to any intellectual property owned or controlled by Tenstorrent or any other party. For the avoidance of doubt, this license does not extend to technologies not owned by Tenstorrent, including other standards such as PCIe, BoW, RISC-V, I3C, and UCIe. Licensee is responsible to get independent licenses to non-Tenstorrent technologies such as these.

3. **No Patent Targeting**
    a. You agree that You will not use this Specification for purposes of acquiring, asserting, modifying, filing, continuing, drafting, continuing in part patent applications which read on implementations of this Specification or portions thereof. Tenstorrent is entitled to seek injunctive relief to enforce any violation of this Section 3.
4. **Feedback**
    a. You may, but don't have to, provide to Tenstorrent suggestions, feature requests or other feedback regarding this Specification, including possible enhancements or modifications to this Specification ("Feedback"). For any Feedback that You voluntarily provide, You hereby grant Tenstorrent and its affiliates a perpetual, non-exclusive, worldwide, irrevocable license to use, reproduce, modify, license, sublicense (through multiple tiers of sublicensees), and distribute (through multiple tiers of distributors) it without the payment of any royalties or fees to You. This means, You grant to Tenstorrent a perpetual, irrevocable, non-exclusive, royalty-free, worldwide copyright license, with the right to directly and indirectly sublicense, to copy, perform, display, and distribute the Feedback in any way, and to prepare derivative works that are based on or incorporate all or part of the Feedback. Further, if by including Your Feedback in the Open Chiplet Atlas™ architecture or collateral, a patent claim or patent application that is owned or controlled by You or Your affiliate becomes necessary to implement any portion of Open Chiplet Atlas™ architecture, then You hereby grant to all Open Chiplet Atlas™ architecture implementers a royalty-free, worldwide, non-exclusive license under such necessary claim, to make, have made, use, import, offer to sell, lease, sell, promote and otherwise distribute implementations of the Open Chiplet Atlas™ architecture. Feedback will be treated as non-confidential and may be freely disclosed by Tenstorrent. Tenstorrent will use or not use Feedback at its sole discretion.
5. **No Warranties**
    a. THIS SPECIFICATION AND ANY ACCOMPANYING COLLATERAL MAY NOT BE QUALIFIED TO WORK IN CERTAIN ENVIRONMENTS OR AT ALL. THIS SPECIFICATION AND ANY ACCOMPANYING COLLATERAL IS PROVIDED "AS IS" AND POSSIBLY WITH FAULTS. TENSTORRENT DISCLAIMS ANY AND ALL WARRANTIES AND GUARANTEES, EXPRESS, IMPLIED OR OTHERWISE, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, NONINFRINGEMENT, OR FITNESS FOR A PARTICULAR PURPOSE. Tenstorrent does not warrant or assume responsibility for the accuracy or completeness of any information, text, graphics, links or other items contained within this Specification or accompanying collateral. You assume all liability, financial or otherwise, associated with the use of products designed to this Specification.
6. **Limitation of Liability**
    a. NEITHER TENSTORRENT NOR ITS LICENSORS, RESELLERS, DEALERS, OR SUPPLIERS SHALL HAVE ANY LIABILITY TO YOU OR ANY OTHER PERSON OR ENTITY FOR ANY INDIRECT, INCIDENTAL, SPECIAL, CONSEQUENTIAL OR ANY OTHER SIMILAR DAMAGES WHATSOEVER, INCLUDING BUT NOT LIMITED TO, LOSS OF REVENUE OR PROFIT, LOST OR DAMAGED DATA, LOST OR DAMAGED COMPUTER SYSTEMS, OR OTHER COMMERCIAL OR ECONOMIC LOSS, EVEN IF YOU HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES, OR THEY ARE FORESEEABLE. TENSTORRENT IS ALSO NOT RESPONSIBLE FOR CLAIMS BY A THIRD PARTY. THIS SPECIFICATION IS IN CONSTANT DEVELOPMENT AND REDEVELOPMENT: IT IS NOT FAULT-TOLERANT AND IS NOT INTENDED FOR USE IN HAZARDOUS ENVIRONMENTS REQUIRING FAIL-SAFE PERFORMANCE IN WHICH THE FAILURE OF THE ENGINEERING SAMPLE COULD LEAD DIRECTLY TO DEATH, PERSONAL INJURY, OR SEVERE PHYSICAL OR ENVIRONMENTAL DAMAGE.
        i. THIS SPECIFICATION IS PROVIDED TO YOU FREE. ACCORDINGLY, TENSTORRENT'S MAXIMUM AGGREGATE LIABILITY, AND THAT OF TENSTORRENT'S LICENSORS, DEALERS, RESELLERS, SHALL NOT EXCEED $10.00 USD.
        ii. YOU ACKNOWLEDGE THAT THE LIMITATIONS OF LIABILITY IN THIS CONTRACT ARE AN ESSENTIAL PART OF THE CONTRACT.
7. **Termination**
    a. This License shall remain in force until terminated by You or by Tenstorrent. You may terminate this license immediately by discontinuing use of this Specification. Tenstorrent may, at any time, terminate this Agreement at any time upon notice to You where notice may be given, among other methods, via a posting on the same webpage where You downloaded this Specification.

b. Upon termination by Tenstorrent, You shall stop reviewing the relevant this Specification. Upon expiry or termination, the provisions of Sections 1 (“Ownership”), 3 (“No Patent Targeting”), 4 (“Feedback”), 5 (“No Warranties”), 6 (“Limitation of Liability”), 7 (“Termination”), and 8 (“General”) survive.

8. General
   a. **Assignment**: You do not have rights to assign this agreement. Any attempted assignment shall be void and of no effect. Tenstorrent may assign, delegate or transfer this Agreement at any time and for any reason.
   b. **Choice of Law**: This Agreement will be governed in all respects by the laws of the United States and of the State of California, without regard to the conflicts of laws principles. The United Nations Convention on Contracts for the International Sale of Goods is specifically disclaimed. The state or federal courts residing in Santa Clara County, California shall have exclusive jurisdiction over any dispute or claim arising out of this Agreement. Notwithstanding this, You agree that Tenstorrent shall still be allowed to apply for injunctive remedies or an equivalent type of urgent legal relief in any jurisdiction.
   c. **Export Compliance**: You are responsible to ensure that Your use of this Specification complies fully with any relevant export laws and regulations.
   d. **Severability**: If any court of competent jurisdiction determines that any provision of this Agreement is illegal, invalid or unenforceable, such provision will be construed as limited to the extent necessary to be consistent with and fully enforceable under the law and the remaining provisions will remain in full force and effect. Unless otherwise specified, remedies are cumulative.
   e. **Independent Parties**. Each party acknowledges and agrees that the other is an independent contractor in the performance of this Agreement.

Appendix B: Terminology

All product names, logos, and brands are property of their respective owners. All company, product and service names used in this document are for identification purposes only. Use of these names, logos, and brands does not imply endorsement.

B.1. Initialism

- *ATC* - Address Translation Cache
- *AVSBus* - Adaptive Voltage Scaling Bus
- *BMC* - Board Management Controller
- *BoW* - Bunch of Wires
- *D2D* - Die-To-Die
- *DFT* - Design for Test
- *DSID* - Dynamic Security Identifier
- *FuSa* - Functional Safety
- *I3C* - Improved Inter-Integrated Circuit
- *MSI* - Message Signaled Interrupt
- *OCA* - Open Chiplet Atlas
- *OCCP* Open Chiplet Configuration Protocol
- *OCMI* - Open Chiplet Management Interface
- *OCSP* - Open Chiplet Safety Protocol
- *OCE* - Open Chiplet Ecosystem
- *PMBus* - Power Management Bus
- *PPA* - Power, Performance and Area
- *PTAP* - Primary Test Access Port
- *QoS* - Quality of Service
- *RAS* - Reliability, Availability, Serviceability
- *RMA* - Return Merchandise Authorization
- *ROM* - Read-Only Memory
- *RoT* - Root of Trust
- *SiP* - System in Package
- *SMC* - System Management Controller
- *SPI* - Serial Peripheral Interface
- *SSID* - Static Security Identifier
- *STAP* - Secondary Test Access Port
- *TEE* - Trusted Execution Environment
- *TLB* - Translation Lookup Buffer
- *UCIe* - Universal Chip Interconnect Express.
- *VR* - Voltage Regulator

B.2. Nomenclature

- *Address Domain* - A range of memory addresses that a processor or a software process uses to access memory or memory-mapped devices.

- *Base Chiplet* - A chiplet that integrates essential functions, like essential compute elements, interconnect fabric, safety, security, and other management features. It is usually configured as the primary chiplet for all applicable domains.
- *Boot* - The process of starting a system and bringing it to a fully functional state, including hardware initialization and software loading.
- *Channel* - An abstracted mechanism for inter-chiplet communication.
- *Cluster* - A group of compute cores that coordinate in a single or multiple chiplets.
- *Device Tree* - A data structure used to describe a hardware system, as defined by devicetree.org.
- *Expansion Chiplet* - An auxiliary chiplet that connects to the base chiplet to extend functionality or increase performance, such as adding additional processing units (e.g. CPU, GPU, NPUs), memory, interfaces, or accelerators.
- *Function Domain* - A specific functional area that encapsulates a distinct aspect of silicon architecture or implementation, like boot, system management, security, etc.
- *Hub Chiplet* - A specialized chiplet that primarily functions as a high-bandwidth interconnect or data routing node, enabling efficient communication among chiplets, like CPUs, accelerators, memory, or I/O. Hub chiplets usually integrate functionalities to act as the primary chiplet for all applicable domains.
- *OCA Functional Domain* - A specific functional area defined within the OCA framework, encompassing a related group of features and functionalities.
- *OCA Profile* - Refinement of the OCA basic definition through selection of functional domains and the assignment of parameter values, tailored for specific families of chiplets.
- *Primary Chiplet* - A chiplet that serves as the main control element of a particular function domain. Different function domains may designate different primary chiplets.
- *Reset* - A mechanism that initializes circuits to a predefined state.
- *Secondary Chiplet* - A chiplet that doesn't serve as the primary chiplet of a given function domain.
- *RAS Agent* - A component responsible for implementing Reliability, Availability, and Serviceability (RAS) features within a chiplet.
- *Profile* - A refinement of the OCA basic definition through selection of functional domains and the assignment of parameter values, tailored for specific families of chiplets.

B.3. Conventions

- *Shall* - Indicates mandatory requirements that need to be strictly followed in order to conform to the standard.

[REQ-EX-1] This shall be an example requirement.

- *Should* - Indicates a recommendation without excluding other possibilities. It signifies that there may be valid reasons to deviate from the specified behavior, but that careful consideration should be given before doing so.

[OPT-EX-1] This should be an example recommendation.

Appendix C: High Level Final License Terms (for Comment Only)

Draft for Comment Only: Non-Final Non-Binding TT Open Chiplet Atlas™ High Level Terms & Conditions

Important Notice—This Appendix C, describing high level terms and conditions for a possible future license agreement, is not a binding agreement and is provided here solely for your review and comment. All of the following is non-binding and solely for comment.

1. **Background.**
   a. Tenstorrent's goal in developing the Open Chiplet Atlas™ Architecture ("**OCA Architecture**" as further defined in Section 2 below) is to provide an open and thorough solution to chiplet design such that compliant off the shelf designs will be able to interoperate when integrated in a system in package. It is designed to address the challenges across multiple layers in the hardware and software stack in an ISA and IP neutral way.
   b. OCA Architecture specifications should be made available to anyone who wants to design products which implement the OCA Architecture as written ("**OCA Implementers**") via a licensing scheme that promotes broad, fast, and low-cost adoption in a transparent manner. Tenstorrent also will provide the ability for others to design a product based on a modified OCA Architecture specification (such members are referred to herein as "**OCA Forkers**"). OCA Implementers and OCA Forkers are referred to collectively as "**OCA Adopters**".
   c. Tenstorrent contemplates that OCA Adopters would enter into a license agreement for each release of an OCA Profile (defined below) to ensure clarity of the scope of licenses granted and received by all parties.
2. **Content**. Tenstorrent contemplates maintaining the following items:
   a. The OCA Architecture, which is defined by several specifications, further comprises:
      i. "**OCA Functional Domains**" include the core functionality of OCA (e.g., security, RAS, etc.).
      ii. "**OCA Profiles**" are normative specifications comprised of OCA Functional Domains that have been configured and augmented to address certain segments that have specific use cases and requirements (e.g., automotive, data center).
   b. Additionally, Tenstorrent plans to separately license in an open manner:
      i. "**OCA Harness**" are RTL and related design materials which can act as a starting point for implementations of OCA as well as other IP commonly needed for chiplet designs.
3. **Feedback**. Tenstorrent maintains the OCA Profiles, OCA Functional Domains, and collateral. Tenstorrent highly encourages feedback on any part of the OCA Architecture or OCA Harness. To provide feedback under the OCA Contributor Agreement please contact Tenstorrent at feedback@openchipletatlas.org.
4. **Implementation License.**
   a. **Copyright**. Tenstorrent would grant OCA Implementers a non-exclusive, worldwide, non-sublicensable, non-transferable, royalty free (zero dollar) license under its copyrights in an OCA Profile to reproduce, distribute, display, perform and create derivative works for the sole purpose of developing products that implement and are compliant with the normative elements of that OCA Profile.
   b. **Patent**. Tenstorrent would grant OCA Implementers a non-exclusive, worldwide, non-sublicensable, non-transferable, royalty free (zero dollar) license under its necessary patent claims that read on the normative elements of an OCA Profile to make, use, sell and import portions of products that implement and are compliant with the normative elements of an OCA Profile ("**OCA Compliant Portions**").
5. **Forking License.**
   a. **Copyright**. Tenstorrent would grant OCA Forkers a non-exclusive, worldwide, non-sublicensable, non-transferable, royalty free (zero dollar) license under its copyrights in an OCA Profile to reproduce, distribute, display, perform and create derivative works for the sole purpose of developing products that implement any element of that OCA Profile ("**OCA Forks**"). This license would extend only to implementations of elements of the OCA Profile as licensed by Tenstorrent and not to modifications or additions thereto.

   b. **Patent**. Tenstorrent would grant OCA Forkers a non-exclusive, worldwide, non-sublicensable, non-transferable, royalty free (zero dollar) license under its necessary patent claims that read on the normative elements of an OCA Profile to make, use, sell and import portions of products that implement any element of that OCA Profile. This license would extend only to implementations of elements of the OCA Profile as licensed by Tenstorrent and not to modifications or additions thereto.

6. **Reciprocal Patent License**. As a necessary condition to receive the preceding licenses to an OCA Profile, OCA Adopters and their affiliates would grant a license under their necessary patent claims that read on the normative elements of the licensed OCA Profile to Tenstorrent and OCA Implementers to make, use, sell and import portions of products that implement and comply with the normative elements of any OCA Profile.
7. **Additional Terms.**
   a. OCA Adopters would be prohibited from using any OCA Architecture specifications or collateral to file, modify or otherwise target patent applications or claims.
   b. Tenstorrent may, at any time, terminate its license with any OCA Implementer or OCA Forker that commences, maintains or participates in any legal proceeding against any OCA Implementer with respect to any OCA Architecture implementation.